

MINA

Product documentation

Mediation, Integration and Automation

Version 1.0.3 - 97 activities, 11 groups, 85 functions and 12 shared connection types.

This manual and the searchable web guide share the same content. Activity tables reflect base editor contracts; selected schemas and provider modes can add fields or requirements. Read the Security section before shared or production deployment.

Use the PDF bookmarks or the linked table of contents to navigate. Content does not certify external providers, guarantee performance, or establish production security readiness.

Contents

MINA documentation	11
Tasks, sub-tasks and event starters	12
Input mapping, schemas and expressions	13
AI-assisted task and project development	14
Workspace, projects and samples	15
Start	16
Scheduler	17
Call Sub Task	19
End	21
Read File	22
Write File	25
List Files	28
Delete File	31
Rename File	33
Copy File	35
File Poller	37
FTP Get	39
FTP Put	41
FTP Delete	43
FTP Directory	45
FTP Change Directory	47
SFTP Get	49
SFTP Put	52
SFTP Delete	55
SFTP Directory	58
SFTP Change Directory	61

HTTP Listener	64
HTTP Request	66
HTTP Response	69
REST API Receiver	71
REST API Invoke	73
SOAP Service	76
SOAP Request Reply	78
EMS Queue Receiver	81
EMS Topic Subscriber	85
EMS Queue Sender	89
EMS Topic Publisher	93
EMS Request Reply	96
EMS Reply	101
Get JMS Queue Message	106
JMS Receive Message	108
JMS Request Reply	111
JMS Send Message	115
Reply to JMS Message	118
Wait for JMS Request	122
Kafka Receive Message	125
Kafka Send Message	131
Kafka Get Messages	137
Pub/Sub Subscriber	143
Pub/Sub Publisher	146
Dynamic Connection	149
IDoc Acknowledgment	154
IDoc Confirmation	157

IDoc Converter	160
IDoc Listener	163
IDoc Parser	167
IDoc Reader	171
Post IDoc to SAP	174
IDoc Renderer	178
RFC BAPI Listener	181
Invoke RFC BAPI in SAP	185
Reply from RFC BAPI in SAP	189
Read Table	192
JDBC Create	196
JDBC Update	199
JDBC Query	202
JDBC Truncate	206
JDBC Delete	209
Stored Procedure	212
Dynamic SQL	215
Snowflake Insert	219
Snowflake Query	223
Snowflake Update	226
Snowflake Delete	229
Snowflake Bulk Load	232
AMQP Receive Message	237
AMQP Get Message	241
AMQP Send Message	245
AMQP Dead Letter Message	249
Parse XML	252

Render XML	254
Parse JSON	256
Render JSON	258
Parse Data	260
Render Data	263
Read Excel Workbook	265
Mapper	268
Mediation	270
Transform	272
Log	274
Catch Exception	276
Throw Exception	278
Rethrow Exception	280
Confirm Message	281
Java Activity	283
Python Invoke	286
External Command	289
Assign	292
Checkpoint	294
Sleep	296
Get Shared Variable	298
Set Shared Variable	300
If	302
While True	303
For Each	304
Iterate	306
Repeat Until True	308

Repeat on Error Until True	309
Scope	310
None	311
Critical Section	312
JDBC Transaction	313
Pick First	314
concat	315
trim	316
normalizeSpace	317
upperCase	318
lowerCase	319
capitalize	320
stringLength	321
substring	322
substringBefore	323
substringAfter	324
replace	325
matches	326
startsWith	327
endsWith	328
contains	329
compare	330
translate	331
padLeft	332
padRight	333
tokenize	334
currentDate	335

currentTime	336
currentDateTime	337
parseDate	338
formatDate	339
formatDateTime	340
addDays	341
addMonths	342
addHours	343
addMinutes	344
addSeconds	345
dateDifference	346
year	347
month	348
day	349
hour	350
minute	351
second	352
timezoneFromDateTime	353
number	354
integer	355
round	356
roundHalfToEven	357
floor	358
ceiling	359
abs	360
sqrt	361
power	362

modulo	363
clamp	364
min	365
max	366
sum	367
average	368
count	369
distinctValues	370
sort	371
reverse	372
first	373
last	374
indexOf	375
subsequence	376
flatten	377
join	378
split	379
deepEqual	380
string	381
boolean	382
jsonParse	383
jsonRender	384
render-xml	385
coalesce	386
default	387
exists	388
empty	389

not	390
ifThenElse	391
uuid	392
base64Encode	393
base64Decode	394
hexEncode	395
hexDecode	396
urlEncode	397
urlDecode	398
hash	399
JDBC shared connection	400
SNOWFLAKE shared connection	403
AMQP shared connection	406
HTTP shared connection	409
FTP shared connection	411
SFTP shared connection	412
EMS shared connection	414
JMS shared connection	416
KAFKA shared connection	418
PUBSUB shared connection	421
SAP shared connection	423
SAP_TID shared connection	426
Environment property defaults	427
Mediation: business messages without transformation scripts	443
MINA documentation site and PDF	446
Activity debugging and testing	447
MINA Studio debugging	449

Data and file utilities	450
Runtime logging	452
MINA Control Plane guide	453
Control Plane operator workspace	457
Python archive modes	458
Vendor driver installation	460
MINA environment variables	461
SAP ECC integration runtime guide	465
EMS/JMS sender reuse in Run mode	468
Kafka publishing performance	469
MINA security review - 2026-09-22	470
Web dependency security update	474

MINA documentation

Getting started

Overview

Mediation, Integration and Automation. This installed reference covers Studio, activities, tasks, groups, mapping functions, shared connections, runtime, deployment and Control Plane. It is available offline and contains no project data or credentials.

Activity fields are extracted from the Studio editor contracts. A shared field marked optional may still be needed for a selected mode or provider. Dynamic schemas are project-specific; the guide explains how Studio derives them rather than inventing their fields.

Use the tree or search to choose a topic. Activity pages have Configuration, Input, Output, Advanced and Errors tabs. Download the complete PDF for the same reference in a portable format.

Scope: this is a product reference, not a claim of provider certification or production security readiness. Read the Security section before shared or production deployment.

Tasks, sub-tasks and event starters

Studio

Configuration

A project contains Starter Tasks and callable Sub Tasks. Give each task a unique name. Start and End define task boundaries; an event receiver replaces manual Start for event-driven tasks. Call Sub Task selects a target and may resolve a dynamic override at runtime. Keep an explicit fallback when packaging must discover a called task.

Create activities on the canvas and connect transitions. Conditions use input, environment properties and earlier activity output. Validate Task and Validate Project before Run or Debug. Nested calls are shown in the debug process/activity tree.

Input

Declare the task input using an inline schema or a project schema. Call Sub Task maps into that interface. Event starters receive data from their configured system; they do not require mapping an inbound event from a preceding activity. Real providers still require valid connection and listener configuration.

Output

Map the End activity to the task output interface. A waiting Call Sub Task publishes the called result. Spawn without waiting does not provide the synchronous business result. Project-selected schemas determine actual child fields.

Advanced

Run and ordinary Debug keep continuous starters ready for events until stopped. Activity Testing opens a paused session without starting event listeners. Debug stepping and breakpoints affect the active job; test-bench Run once executes only the selected activity, not a complete process.

Errors

Connect specific error paths or use Catch and Rethrow. Unhandled sub-task errors propagate to the caller. Missing task references, broken mappings, unavailable providers and malformed group graphs require correction before reliable execution.

Input mapping, schemas and expressions

Studio

Mapping workflow

Select an activity and open Input. Drag Data fields, Functions or Constants onto a target, or type an expression. The expanded input editor provides more space and applies edits only after Apply mappings. For typed suggestions enter a dot in a data expression, or press Ctrl+Space; Enter or Tab accepts a suggestion.

```
${input.customer.id}  
${properties.routing.destination}  
${Previous-Activity.result}  
${vars.currentElement}
```

Use the Data tree for the actual saved reference names. A constant string is different from an expression; keep datatype conversion and quoting appropriate for the target. Structural schema nodes are populated through child fields; supported union/scalar targets accept whole values.

Repeating targets support For Each and For Each Group. Conditional mappings support If, When/Otherwise and Choose. Project functions use named parameters in expressions; built-in function pages list signatures and insertion templates, not guaranteed outputs for every input datatype.

JDBC named parameters such as :orderId create Input parameters.orderId after SQL analysis. Set the parameter datatype before mapping. SAP IDoc fields derive from fetched metadata; XML/JSON/flat data fields derive from the chosen schema. Call Sub Task fields derive from the selected task interface. Refresh metadata and validate after changing contracts.

AI-assisted task and project development

Studio

Workflow

AI Build accepts a requirement for the current Task or Complete Project. Review the design preview and generated definition before Apply. Generation alone does not deploy or run the application. Without a configured model key, the local blueprint builder is used.

Do not enter credentials or confidential payloads into AI prompts. The current prompt instruction to exclude credentials is not an enforced redaction boundary. Validate and test generated designs, and review the security report before enabling shared access.

Workspace, projects and samples

Studio

Workspace

Use the project tree to manage Tasks, schemas, resources and environment properties. The palette and searchable activity picker add activities to the canvas. Select and move activities, edit their display names, and configure transition conditions. Use Window for the XML/JSON viewers and comparison workspace.

Project export/import preserves editable design artifacts. Deployment export packages an executable application; these are different workflows. Installed samples open as editable project copies. Samples marked connection setup need your own provider configuration and credentials.

Themes include Plain Classic and Plain Studio with light/dark variants. Layout panes can be shown or hidden from menus. The status/footer area exposes background work and execution state.

Start

Activities / Starters & Tasks

Overview

Defines the task input boundary.

For Sub Tasks, the Start schema is the callable input contract. Starter Tasks normally use one external event activity instead.

Activity type: start; operation: start.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

No additional fields declared for this section.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The Task input schema replaces the generic payload boundary when configured.

Field / key	Type / required	Details
Starter payload payload	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The Task input schema replaces the generic payload boundary when configured.

Field / key	Type / required	Details
Initial task payload payload	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

No additional fields declared for this section.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

Scheduler

Activities / Starters & Tasks

Overview

Runs Scheduler on the task execution path.

Activity type: timer; operation: schedule.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Schedule mode scheduleMode	select Not marked required by editor	Choices: dateTime, cron
Scheduled date and time scheduledDateTime	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Cron expression cronExpression	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Time zone timezone	select Not marked required by editor	Choices: local, UTC
Repeat after first run repeatEnabled	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Interval interval	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Unit unit	select Not marked required by editor	Choices: seconds, minutes, hours, days
Run once immediately when started locally runOnceOnLocalStart	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

No additional fields declared for this section.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Scheduled time scheduledTime	dateTime Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Actual fire time actualTime	dateTime Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Sequence sequence	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
SCHEDULER	The schedule expression or interval is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

Call Sub Task

Activities / Starters & Tasks

Overview

Invokes a reusable Sub Task and waits for its result.

Mapped values become the Sub Task Start input. Dynamic Sub Task ID/name is resolved per invocation; the selected Sub Task is its design-time schema and runtime fallback. The Sub Task End output returns to this activity.

Activity type: call_task; operation: call.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Sub Task taskId	task Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic Sub Task ID/name expression dynamicTaskId	text Not marked required by editor	Resolved for every invocation. Use a literal Sub Task ID/name or an expression such as <code>\${properties.routing.targetTask}</code> ; the selected Sub Task remains the fallback.
Spawn without waiting spawn	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The selected Sub Task Start and End schemas replace generic payload/result fields.

Field / key	Type / required	Details
Sub Task input payload	object Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic Sub Task ID/name expression dynamicTaskId	string Not marked required by editor	Resolved for every invocation. Use a literal Sub Task ID/name or an expression such as <code>\${properties.routing.targetTask}</code> ; the selected Sub Task remains the fallback.
Spawn without waiting spawn	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The selected Sub Task Start and End schemas replace generic payload/result fields.

Field / key	Type / required	Details
Invoked task ID taskId	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Sub Task output output	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Spawn status spawned	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
TASK_NOT_FOUND	The selected or dynamic Sub Task does not exist.
SUBTASK_ERROR	The invoked Sub Task returned an error.
VALIDATION	Required input or mapped data is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

End

Activities / Starters & Tasks

Overview

Defines the task output boundary.

Mapped values become the final task result. For a Sub Task, this result is returned to its Call Sub Task activity.

Activity type: end; operation: end.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

No additional fields declared for this section.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The Task output schema determines the result mapping.

Field / key	Type / required	Details
Task result result	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The Task output schema determines the result mapping.

Field / key	Type / required	Details
Task output result	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

No additional fields declared for this section.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

Read File

Activities / File

Overview

Performs the selected filesystem operation.

File paths and operation-specific values accept constants, property expressions, or mappings from earlier execution-path outputs. Runtime output contains operation-specific metadata and content.

Activity type: file; operation: read.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Filename / directory path	text Not marked required by editor	Absolute paths and project-property expressions are supported.
Include timestamp metadata includeTimestamp	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Read as readAs	select Not marked required by editor	Choices: Text, Binary
Exclude file content excludeFileContent	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Text encoding encoding	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Filename path	string Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Text encoding encoding	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Include timestamp metadata includeTimestamp	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Read as readAs	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Exclude file content excludeFileContent	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Text file content textContent	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Binary file content binaryContent	base64 Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Full filename fileInfo.fullName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Filename fileInfo.fileName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Location fileInfo.location	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
File type fileInfo.type	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Read protected fileInfo.readProtected	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Write protected fileInfo.writeProtected	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Size fileInfo.size	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Last modified fileInfo.lastModified	dateTime Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
FILE_NOT_FOUND	The configured source path does not exist.
FILE_IO	The runtime cannot read, write, move, or delete the file.
PERMISSION	The runtime lacks filesystem permission.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

Write File

Activities / File

Overview

Performs the selected filesystem operation.

File paths and operation-specific values accept constants, property expressions, or mappings from earlier execution-path outputs. Runtime output contains operation-specific metadata and content.

Activity type: file; operation: write.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Filename / directory path	text Not marked required by editor	Absolute paths and project-property expressions are supported.
Include timestamp metadata includeTimestamp	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Write as writeAs	select Not marked required by editor	Choices: Text, Binary
Text encoding encoding	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Append to existing file append	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Overwrite existing file overwrite	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Create non-existing directories createDirectories	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Add line separator addLineSeparator	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Compression compression	select Not marked required by editor	Choices: None, GZip

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Filename path	string Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Text content textContent	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Binary content binaryContent	base64 Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Text encoding encoding	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Add line separator addLineSeparator	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Include timestamp metadata includeTimestamp	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Write as writeAs	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Append to existing file append	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Overwrite existing file overwrite	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Create non-existing directories createDirectories	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Compression compression	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Affected path path	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Operation result success	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
FILE_NOT_FOUND	The configured source path does not exist.
FILE_IO	The runtime cannot read, write, move, or delete the file.
PERMISSION	The runtime lacks filesystem permission.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

List Files

Activities / File

Overview

Performs the selected filesystem operation.

File paths and operation-specific values accept constants, property expressions, or mappings from earlier execution-path outputs. Runtime output contains operation-specific metadata and content.

Activity type: file; operation: list.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Filename / directory path	text Not marked required by editor	Absolute paths and project-property expressions are supported.
Include timestamp metadata includeTimestamp	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
File name pattern / wildcard pattern	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Recursive recursive	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
List listType	select Not marked required by editor	Choices: Files and Directories, Only Files, Only Directories
Sort by sortBy	select Not marked required by editor	Choices: Name, Size, Last Modified
Sort order sortOrder	select Not marked required by editor	Choices: Ascending, Descending

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Directory / wildcard path	string Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Recursive recursive	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Include timestamp metadata includeTimestamp	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
File name pattern / wildcard pattern	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
List listType	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Sort by sortBy	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Sort order sortOrder	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Matched file information files	array Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
File count count	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Detected change eventType	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
FILE_NOT_FOUND	The configured source path does not exist.
FILE_IO	The runtime cannot read, write, move, or delete the file.
PERMISSION	The runtime lacks filesystem permission.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose

sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: `frontend/src/ActivityEditor.tsx`

Delete File

Activities / File

Overview

Performs the selected filesystem operation.

File paths and operation-specific values accept constants, property expressions, or mappings from earlier execution-path outputs. Runtime output contains operation-specific metadata and content.

Activity type: file; operation: delete.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Filename / directory path	text Not marked required by editor	Absolute paths and project-property expressions are supported.
Include timestamp metadata includeTimestamp	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Remove non-empty directory recursively recursive	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Ignore missing file ignoreMissing	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Filename / directory path	string Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Recursive recursive	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Include timestamp metadata includeTimestamp	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Ignore missing file ignoreMissing	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Affected path path	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Operation result success	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
FILE_NOT_FOUND	The configured source path does not exist.
FILE_IO	The runtime cannot read, write, move, or delete the file.
PERMISSION	The runtime lacks filesystem permission.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

Rename File

Activities / File

Overview

Performs the selected filesystem operation.

File paths and operation-specific values accept constants, property expressions, or mappings from earlier execution-path outputs. Runtime output contains operation-specific metadata and content.

Activity type: file; operation: rename.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Filename / directory path	text Not marked required by editor	Absolute paths and project-property expressions are supported.
Include timestamp metadata includeTimestamp	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
To filename destination	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Overwrite existing file overwrite	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Create non-existing directories createDirectories	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
From filename path	string Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
To filename destination	string Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Include timestamp metadata includeTimestamp	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Overwrite existing file overwrite	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Create non-existing directories createDirectories	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Affected path path	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Operation result success	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
FILE_NOT_FOUND	The configured source path does not exist.
FILE_IO	The runtime cannot read, write, move, or delete the file.
PERMISSION	The runtime lacks filesystem permission.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

Copy File

Activities / File

Overview

Performs the selected filesystem operation.

File paths and operation-specific values accept constants, property expressions, or mappings from earlier execution-path outputs. Runtime output contains operation-specific metadata and content.

Activity type: file; operation: copy.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Filename / directory path	text Not marked required by editor	Absolute paths and project-property expressions are supported.
Include timestamp metadata includeTimestamp	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Destination filename / directory destination	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Overwrite existing file overwrite	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Create non-existing directories createDirectories	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Preserve file attributes preserveAttributes	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Copy directory recursively recursive	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Source filename / directory path	string Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Destination filename / directory destination	string Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Recursive recursive	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Include timestamp metadata includeTimestamp	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Overwrite existing file overwrite	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Create non-existing directories createDirectories	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Preserve file attributes preserveAttributes	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Affected path path	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Operation result success	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
FILE_NOT_FOUND	The configured source path does not exist.
FILE_IO	The runtime cannot read, write, move, or delete the file.
PERMISSION	The runtime lacks filesystem permission.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

File Poller

Activities / File

Overview

Performs the selected filesystem operation.

File paths and operation-specific values accept constants, property expressions, or mappings from earlier execution-path outputs. Runtime output contains operation-specific metadata and content.

Activity type: file; operation: poll.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Filename / directory path	text Not marked required by editor	Absolute paths and project-property expressions are supported.
Include timestamp metadata includeTimestamp	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
File name pattern / wildcard pattern	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Change type eventType	select Not marked required by editor	Choices: Created, Modified, Deleted, Any
Poll interval (seconds) pollInterval	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Include existing files includeExisting	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Recursive recursive	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
After processing postAction	select Not marked required by editor	Choices: None, Delete, Move
Move processed file to moveTo	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

No additional fields declared for this section.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Matched file information files	array Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
File count count	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Detected change eventType	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
FILE_NOT_FOUND	The configured source path does not exist.
FILE_IO	The runtime cannot read, write, move, or delete the file.
PERMISSION	The runtime lacks filesystem permission.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

FTP Get

Activities / FTP

Overview

Performs the selected FTP operation through a shared FTP connection.

Connection defaults come from the resource; paths, names, content, and transfer settings may be mapped dynamically for each invocation.

Activity type: ftp; operation: get.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Shared connection resourceId	resource Not marked required by editor	Connection type: ftp
Remote path remotePath	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Working directory workingDirectory	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Binary transfer binary	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Timeout seconds timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Remote path remotePath	string Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Working directory workingDirectory	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Binary transfer binary	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Timeout seconds timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Downloaded content contentBase64	base64 Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Size size	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
AUTHENTICATION	Remote authentication failed.
REMOTE_NOT_FOUND	The remote path does not exist.
TRANSFER	The remote transfer failed.
CONNECTIVITY	The configured shared connection is unavailable.
RETRY_EXHAUSTED	The reconnection or retry policy was exhausted.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

FTP Put

Activities / FTP

Overview

Performs the selected FTP operation through a shared FTP connection.

Connection defaults come from the resource; paths, names, content, and transfer settings may be mapped dynamically for each invocation.

Activity type: ftp; operation: put.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Shared connection resourceId	resource Not marked required by editor	Connection type: ftp
Remote path remotePath	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Working directory workingDirectory	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Binary transfer binary	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Timeout seconds timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Content content	binary string Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Remote path remotePath	string Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Working directory workingDirectory	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Binary transfer binary	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Timeout seconds timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Remote path remotePath	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Operation result success	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
AUTHENTICATION	Remote authentication failed.
REMOTE_NOT_FOUND	The remote path does not exist.
TRANSFER	The remote transfer failed.
CONNECTIVITY	The configured shared connection is unavailable.
RETRY_EXHAUSTED	The reconnection or retry policy was exhausted.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

FTP Delete

Activities / FTP

Overview

Performs the selected FTP operation through a shared FTP connection.

Connection defaults come from the resource; paths, names, content, and transfer settings may be mapped dynamically for each invocation.

Activity type: ftp; operation: delete.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Shared connection resourceId	resource Not marked required by editor	Connection type: ftp
Remote path remotePath	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Working directory workingDirectory	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Binary transfer binary	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Timeout seconds timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Remote path remotePath	string Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Working directory workingDirectory	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Binary transfer binary	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Timeout seconds timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Remote path remotePath	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Operation result success	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
AUTHENTICATION	Remote authentication failed.
REMOTE_NOT_FOUND	The remote path does not exist.
TRANSFER	The remote transfer failed.
CONNECTIVITY	The configured shared connection is unavailable.
RETRY_EXHAUSTED	The reconnection or retry policy was exhausted.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

FTP Directory

Activities / FTP

Overview

Performs the selected FTP operation through a shared FTP connection.

Connection defaults come from the resource; paths, names, content, and transfer settings may be mapped dynamically for each invocation.

Activity type: ftp; operation: dir.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Shared connection resourceId	resource Not marked required by editor	Connection type: ftp
Remote path remotePath	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Working directory workingDirectory	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Binary transfer binary	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Timeout seconds timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Remote path remotePath	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Working directory workingDirectory	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Binary transfer binary	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Timeout seconds timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Directory entries entries	array Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Current directory directory	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
AUTHENTICATION	Remote authentication failed.
REMOTE_NOT_FOUND	The remote path does not exist.
TRANSFER	The remote transfer failed.
CONNECTIVITY	The configured shared connection is unavailable.
RETRY_EXHAUSTED	The reconnection or retry policy was exhausted.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

FTP Change Directory

Activities / FTP

Overview

Performs the selected FTP operation through a shared FTP connection.

Connection defaults come from the resource; paths, names, content, and transfer settings may be mapped dynamically for each invocation.

Activity type: ftp; operation: change_dir.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Shared connection resourceId	resource Not marked required by editor	Connection type: ftp
Target directory remotePath	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Working directory workingDirectory	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Binary transfer binary	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Timeout seconds timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Remote path remotePath	string Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Working directory workingDirectory	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Binary transfer binary	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Timeout seconds timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Remote path remotePath	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Operation result success	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
AUTHENTICATION	Remote authentication failed.
REMOTE_NOT_FOUND	The remote path does not exist.
TRANSFER	The remote transfer failed.
CONNECTIVITY	The configured shared connection is unavailable.
RETRY_EXHAUSTED	The reconnection or retry policy was exhausted.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

SFTP Get

Activities / SFTP

Overview

Performs the selected secure file-transfer operation.

Authentication and host-key policy come from the SFTP shared connection. Operation inputs remain dynamically mappable at runtime.

Activity type: sftp; operation: get.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Shared connection resourceId	resource Not marked required by editor	Connection type: sftp
Remote path remotePath	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Working directory workingDirectory	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Binary transfer binary	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Timeout seconds timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Verify host key verifyHostKey	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Remote path remotePath	string Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Working directory workingDirectory	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Binary transfer binary	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Timeout seconds timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Verify host key verifyHostKey	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Downloaded content contentBase64	base64 Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Size size	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
AUTHENTICATION	Remote authentication failed.
REMOTE_NOT_FOUND	The remote path does not exist.
TRANSFER	The remote transfer failed.
CONNECTIVITY	The configured shared connection is unavailable.
RETRY_EXHAUSTED	The reconnection or retry policy was exhausted.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose

sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: `frontend/src/ActivityEditor.tsx`

SFTP Put

Activities / SFTP

Overview

Performs the selected secure file-transfer operation.

Authentication and host-key policy come from the SFTP shared connection. Operation inputs remain dynamically mappable at runtime.

Activity type: sftp; operation: put.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Shared connection resourceId	resource Not marked required by editor	Connection type: sftp
Remote path remotePath	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Working directory workingDirectory	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Binary transfer binary	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Timeout seconds timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Verify host key verifyHostKey	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Content content	binary string Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Remote path remotePath	string Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Working directory workingDirectory	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Binary transfer binary	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Timeout seconds timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Verify host key verifyHostKey	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Remote path remotePath	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Operation result success	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
AUTHENTICATION	Remote authentication failed.
REMOTE_NOT_FOUND	The remote path does not exist.
TRANSFER	The remote transfer failed.
CONNECTIVITY	The configured shared connection is unavailable.
RETRY_EXHAUSTED	The reconnection or retry policy was exhausted.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

SFTP Delete

Activities / SFTP

Overview

Performs the selected secure file-transfer operation.

Authentication and host-key policy come from the SFTP shared connection. Operation inputs remain dynamically mappable at runtime.

Activity type: sftp; operation: delete.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Shared connection resourceId	resource Not marked required by editor	Connection type: sftp
Remote path remotePath	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Working directory workingDirectory	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Binary transfer binary	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Timeout seconds timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Verify host key verifyHostKey	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Remote path remotePath	string Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Working directory workingDirectory	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Binary transfer binary	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Timeout seconds timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Verify host key verifyHostKey	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Remote path remotePath	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Operation result success	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
AUTHENTICATION	Remote authentication failed.
REMOTE_NOT_FOUND	The remote path does not exist.
TRANSFER	The remote transfer failed.
CONNECTIVITY	The configured shared connection is unavailable.
RETRY_EXHAUSTED	The reconnection or retry policy was exhausted.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose

sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: `frontend/src/ActivityEditor.tsx`

SFTP Directory

Activities / SFTP

Overview

Performs the selected secure file-transfer operation.

Authentication and host-key policy come from the SFTP shared connection. Operation inputs remain dynamically mappable at runtime.

Activity type: sftp; operation: dir.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Shared connection resourceId	resource Not marked required by editor	Connection type: sftp
Remote path remotePath	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Working directory workingDirectory	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Binary transfer binary	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Timeout seconds timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Verify host key verifyHostKey	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Remote path remotePath	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Working directory workingDirectory	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Binary transfer binary	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Timeout seconds timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Verify host key verifyHostKey	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Directory entries entries	array Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Current directory directory	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
AUTHENTICATION	Remote authentication failed.
REMOTE_NOT_FOUND	The remote path does not exist.
TRANSFER	The remote transfer failed.
CONNECTIVITY	The configured shared connection is unavailable.
RETRY_EXHAUSTED	The reconnection or retry policy was exhausted.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose

sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

SFTP Change Directory

Activities / SFTP

Overview

Performs the selected secure file-transfer operation.

Authentication and host-key policy come from the SFTP shared connection. Operation inputs remain dynamically mappable at runtime.

Activity type: sftp; operation: change_dir.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Shared connection resourceId	resource Not marked required by editor	Connection type: sftp
Target directory remotePath	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Working directory workingDirectory	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Binary transfer binary	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Timeout seconds timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Verify host key verifyHostKey	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Remote path remotePath	string Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Working directory workingDirectory	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Binary transfer binary	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Timeout seconds timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Verify host key verifyHostKey	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Remote path remotePath	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Operation result success	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
AUTHENTICATION	Remote authentication failed.
REMOTE_NOT_FOUND	The remote path does not exist.
TRANSFER	The remote transfer failed.
CONNECTIVITY	The configured shared connection is unavailable.
RETRY_EXHAUSTED	The reconnection or retry policy was exhausted.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose

sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

HTTP Listener

Activities / HTTP & APIs

Overview

Starts a Starter Task when an inbound HTTP request matches its configured method and path.

Run or Debug deploys the task and publishes its live Studio endpoint. The request body, headers, query parameters, method, path, and path parameters become the activity output. Host, port, authentication, and TLS are inherited from the selected HTTP shared connection.

Activity type: http_listener; operation: listen.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
HTTP shared connection resourceId	resource Not marked required by editor	Connection type: http
Listener path path	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Allowed HTTP methods methods	methods Not marked required by editor	Select every HTTP method accepted by this listener.
Expected content type contentType	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Authentication policy authentication	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

No additional fields declared for this section.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Request body body	object string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Request headers headers	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Query parameters query	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Path parameters pathParameters	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
HTTP method method	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
HTTP_BAD_REQUEST	The inbound HTTP request is invalid.
HTTP_UNAUTHORIZED	Listener authentication rejected the request.
HTTP_NOT_FOUND	No matching listener resource exists.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

HTTP Request

Activities / HTTP & APIs

Overview

Sends an outbound HTTP request and exposes the response to downstream activities.

The shared HTTP connection supplies base address, security, TLS, proxy, and timeout defaults. Activity values and mapped inputs override defaults at execution time.

Activity type: http; operation: request.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
HTTP shared connection resourceId	resource Not marked required by editor	Connection type: http
Method method	select Not marked required by editor	Choices: GET, POST, PUT, PATCH, DELETE, HEAD
Endpoint URL url	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Timeout seconds timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Follow redirects followRedirects	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Headers headers	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Query parameters query	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Request body body	object string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Method method	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Endpoint URL url	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Timeout seconds timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Follow redirects followRedirects	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Status code statusCode	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Response headers headers	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Response body body	object string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
HTTP_CONNECTIVITY	The HTTP endpoint could not be reached.
HTTP_TIMEOUT	The request exceeded its timeout.
HTTP_CLIENT_ERROR	The endpoint returned a 4xx response.

Fault	Description
HTTP_SERVER_ERROR	The endpoint returned a 5xx response.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

HTTP Response

Activities / HTTP & APIs

Overview

Completes the active inbound HTTP exchange.

Status, headers, and body are returned to the caller that triggered the listener. Place it on the same execution path as the inbound HTTP, REST, or SOAP event.

Activity type: http_response; operation: response.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Status code statusCode	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Content type contentType	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Response headers headers	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Response body body	object string Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Status code statusCode	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Content type contentType	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Sent status code statusCode	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Response sent sent	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
HTTP_RESPONSE	The listener response could not be written.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

REST API Receiver

Activities / HTTP & APIs

Overview

Hosts or invokes a REST operation, depending on the selected operation.

A Receiver starts the task from an inbound request. Invoke performs an outbound call. Request and response structures remain available as hierarchical mapping trees.

Activity type: rest; operation: receiver.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
HTTP shared connection resourceId	resource Not marked required by editor	Connection type: http
Resource path path	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Allowed HTTP methods methods	methods Not marked required by editor	Select every HTTP method exposed by this REST resource.
Request content type contentType	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Response content type responseType	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Resource path path	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Allowed HTTP methods methods	string Not marked required by editor	Select every HTTP method exposed by this REST resource.
Request content type contentType	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Response content type responseType	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Request entity body	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Headers headers	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Query parameters query	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
URI parameters pathParameters	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
REST_VALIDATION	The request does not conform to its API/schema contract.
REST_UNSUPPORTED_MEDIA	The request media type is unsupported.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

REST API Invoke

Activities / HTTP & APIs

Overview

Hosts or invokes a REST operation, depending on the selected operation.

A Receiver starts the task from an inbound request. Invoke performs an outbound call. Request and response structures remain available as hierarchical mapping trees.

Activity type: rest; operation: invoke.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
HTTP shared connection resourceId	resource Not marked required by editor	Connection type: http
Method method	select Not marked required by editor	Choices: GET, POST, PUT, PATCH, DELETE, HEAD
Endpoint URL url	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Timeout seconds timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Follow redirects followRedirects	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Headers headers	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Query parameters query	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Request body body	object string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Method method	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Endpoint URL url	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Timeout seconds timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Follow redirects followRedirects	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Status code statusCode	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Response headers headers	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Response body body	object string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
HTTP_CONNECTIVITY	The HTTP endpoint could not be reached.
HTTP_TIMEOUT	The request exceeded its timeout.
HTTP_CLIENT_ERROR	The endpoint returned a 4xx response.

Fault	Description
HTTP_SERVER_ERROR	The endpoint returned a 5xx response.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

SOAP Service

Activities / HTTP & APIs

Overview

Hosts a SOAP service or invokes a request/reply web service.

Service mode publishes the configured contract and begins the task for a matching SOAP request. Request/reply mode uses the selected HTTP connection and WSDL metadata for outbound invocation.

Activity type: soap; operation: service.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
HTTP shared connection resourceId	resource Not marked required by editor	Connection type: http
Service path path	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Service name serviceName	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Operation name operationName	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
WSDL resource wsdl	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Service path path	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Service name serviceName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Operation name operationName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
WSDL resource wsdl	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
SOAP envelope envelope	xml Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
SOAP/HTTP headers headers	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Resolved operation operation	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
SOAP_FAULT	The request generated a SOAP fault.
WSDL_VALIDATION	The message does not conform to the WSDL.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

SOAP Request Reply

Activities / HTTP & APIs

Overview

Hosts a SOAP service or invokes a request/reply web service.

Service mode publishes the configured contract and begins the task for a matching SOAP request. Request/reply mode uses the selected HTTP connection and WSDL metadata for outbound invocation.

Activity type: soap; operation: request_reply.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
HTTP shared connection resourceId	resource Not marked required by editor	Connection type: http
Service endpoint url	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
SOAP action soapAction	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
WSDL resource wsdl	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Timeout seconds timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
SOAP envelope envelope	xml Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
SOAP headers headers	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Service endpoint url	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
SOAP action soapAction	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
WSDL resource wsdl	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Timeout seconds timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
HTTP status statusCode	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
SOAP response envelope body	xml Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Response headers headers	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
SOAP_FAULT	The remote service returned a SOAP fault.
WSDL_VALIDATION	The request or response violates the WSDL.
CONNECTIVITY	The configured shared connection is unavailable.
RETRY_EXHAUSTED	The reconnection or retry policy was exhausted.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

EMS Queue Receiver

Activities / EMS

Overview

Receive a queue event

Use as a task starter for work arriving on a queue; no upstream message-body input is required.

Prerequisites

Install the licensed, version-compatible JMS client JARs under the runtime driver root/jms or set Driver JAR directory. Use the bundled Java bridge/runtime. A direct EMS connection requires URL, username and password; JNDI additionally requires its context factory, provider URL and connection-factory settings. Keep Java class names exactly as provided by the driver. Test from the execution machine, not only the developer laptop.

Sends, receives, browses, or acknowledges EMS messages.

Destination, delivery, selector, acknowledgement, and message properties follow the selected EMS operation. Client acknowledgement handles can be passed to Confirm Message.

Activity type: ems; operation: queue_receiver.

Configuration

Configuration procedure

Select EMS resource, browse the destination property, and configure selector and receive wait. Client acknowledgement exposes a handle, but acknowledgement timing depends on the runtime path. Session count and flow limit can be mapped from properties; neither is a proven bridge concurrency control.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
EMS connection resourceId	resource Required	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: ems
Messaging style messagingStyle	select Not marked required by editor	Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher. Choices: Queue, Topic, Generic
Destination queue	text Not marked required by editor	Queue name, not the broker URL. Browse an environment property to avoid hard-coding environment-specific destinations.
Message type messageType	select Not marked required by editor	Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented. Choices: Text, Bytes, Map, Object, Object Ref, Simple, Stream, XML Text
JMS message selector messageSelector	text Not marked required by editor	Provider-side JMS selector expression over supported headers/properties. It does not query inside the JSON message body.
Field / key	Type / required	Details
Acknowledge mode acknowledgeMode	select Not marked required by	Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read

Field / key	Type / required	Details
	editor	Advanced before relying on rollback/redelivery. Choices: Auto, Client, Dups OK, Explicit Client, Explicit Client Dups OK, Transactional
Session count maxSessions	number Not marked required by editor	Passed as a session-count option by the wrapper; current engine bridge does not create a configurable receiver-session pool from it.
Flow limit flowLimit	number Not marked required by editor	Passed by the wrapper but not enforced as an engine bridge in-flight limit. Do not rely on it as the only backpressure mechanism.
Receive timeout (ms) receiveTimeout	number Not marked required by editor	Receive wait in milliseconds, default 30000 on the engine bridge path. A timeout can return an empty result; it need not throw a fault.

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
queue = ${properties.messaging.inboundQueue}
receiveTimeout = 30000
Downstream message = ${EMS-Receive.body}
Confirm Message ackId = ${EMS-Receive.ackId}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Messaging style messagingStyle	string Not marked required by editor	Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher.
Destination queue	string Not marked required by editor	Queue name, not the broker URL. Browse an environment property to avoid hard-coding environment-specific destinations.
Message type messageType	string Not marked required by editor	Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented.
JMS message selector messageSelector	string Not marked required by editor	Provider-side JMS selector expression over supported headers/properties. It does not query inside the JSON message body.
Acknowledge mode acknowledgeMode	string Not marked required by editor	Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery.
Session count maxSessions	number Not marked required by editor	Passed as a session-count option by the wrapper; current engine bridge does not create a configurable receiver-session pool from it.
Flow limit flowLimit	number Not marked required by editor	Passed by the wrapper but not enforced as an engine bridge in-flight limit. Do not rely on it as the only backpressure mechanism.

Field / key	Type / required	Details
Receive timeout (ms) receiveTimeout	number Not marked required by editor	Receive wait in milliseconds, default 30000 on the engine bridge path. A timeout can return an empty result; it need not throw a fault.

Output

Illustrative runtime output

```
{ "body": { "id": "A100" }, "headers": { "JMSMessageID": "<provider-id>", "properties": {}, "ackId": "<handle>", "count": 1 }
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message body body	object string binary Not marked required by editor	Received message body. The engine attempts JSON decoding for text that contains valid JSON; other text remains a string.
JMS standard headers headers	object Not marked required by editor	Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support.
JMS application properties properties	object Not marked required by editor	Application-defined message properties returned by the broker. Standard JMS headers are returned separately.
Client acknowledgement handle ackId	string Not marked required by editor	Opaque, runtime-local confirmation token. Pass the original value to Confirm Message; it is not the business message ID and cannot be reconstructed after restart.

Advanced

Delivery, transactions and performance

Persistent sender workers are reused for the same resolved connection and execution scope. The cache is bounded; changing credentials/configuration frequently or exceeding the cache can create new workers. Native text messages and string application properties are the supported send path. Do not treat all editor choices as implemented native features. Retry after an ambiguous timeout can duplicate a message.

Implemented behavior and limitations

Important engine limitation: the one-shot receive bridge acknowledges client-mode delivery before closing, then the engine returns a local confirmation token. Confirm Message cannot undo that early provider acknowledgement. The direct Python listener uses a different persistent transport; do not transfer its guarantees to Studio/engine mode.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Troubleshooting procedure

Missing JAR/class/native bridge errors: check the installed driver directory, matching versions and packaged Java bridge.
Authentication/destination failures: verify credentials, queue/topic permissions and the resolved environment destination.
Timeout: inspect empty-receive results as well as faults. A successful send is not proof that another application consumed or processed the message.

Acceptance test

Acceptance: publish while Run is active, verify automatic flow start, stop during processing and check broker redelivery. Do not approve loss-sensitive processing until acknowledgement timing is verified.

Fault	Description
JMSInvalidInputException	The JMS activity input is invalid.
JMSMessageCreateException	The configured JMS message could not be created.
JMSSessionCreateException	A JMS session could not be created.
JMSReceiveException	The JMS message could not be received.
ActivityTimedOutException	The JMS operation reached its configured timeout.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs

EMS Topic Subscriber

Activities / EMS

Overview

Receive a topic event

Use as a starter for topic messages. Topic delivery differs from queue competing-consumer behavior.

Prerequisites

Install the licensed, version-compatible JMS client JARs under the runtime driver root/jms or set Driver JAR directory. Use the bundled Java bridge/runtime. A direct EMS connection requires URL, username and password; JNDI additionally requires its context factory, provider URL and connection-factory settings. Keep Java class names exactly as provided by the driver. Test from the execution machine, not only the developer laptop.

Sends, receives, browses, or acknowledges EMS messages.

Destination, delivery, selector, acknowledgement, and message properties follow the selected EMS operation. Client acknowledgement handles can be passed to Confirm Message.

Activity type: ems; operation: topic_subscriber.

Configuration

Configuration procedure

Select the EMS resource and topic property. Set selector only when necessary. The UI offers durable/shared options, but the native engine uses a regular consumer and does not wire these subscription constructors.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
EMS connection resourceId	resource Required	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: ems
Messaging style messagingStyle	select Not marked required by editor	Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher. Choices: Queue, Topic, Generic
Destination topic	text Not marked required by editor	Broker topic name. Resolve it from the active environment, for example \${properties.messaging.topic}. Consumer topic lists use semicolons.
Message type messageType	select Not marked required by editor	Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented. Choices: Text, Bytes, Map, Object, Object Ref, Simple, Stream, XML Text
JMS message selector messageSelector	text Not marked required by editor	Provider-side JMS selector expression over supported headers/properties. It does not query inside the JSON message body.
Field / key	Type / required	Details
Acknowledge mode acknowledgeMode	select Not marked required by editor	Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery.

Field / key	Type / required	Details
		Choices: Auto, Client, Dups OK, Explicit Client, Explicit Client Dups OK, Transactional
Session count maxSessions	number Not marked required by editor	Passed as a session-count option by the wrapper; current engine bridge does not create a configurable receiver-session pool from it.
Flow limit flowLimit	number Not marked required by editor	Passed by the wrapper but not enforced as an engine bridge in-flight limit. Do not rely on it as the only backpressure mechanism.
Receive timeout (ms) receiveTimeout	number Not marked required by editor	Receive wait in milliseconds, default 30000 on the engine bridge path. A timeout can return an empty result; it need not throw a fault.
Durable subscriber durable	boolean Not marked required by editor	Editor option not wired to a durable subscriber constructor in the native engine bridge. Offline topic retention cannot be assumed.
Durable subscription name subscriptionName	text Not marked required by editor	Intended durable subscription identity; current native engine consumer does not use it to create a durable subscription.
Shared subscription sharedSubscription	boolean Not marked required by editor	Editor option not wired to a shared-subscription constructor in the current native engine bridge.

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
topic = ${properties.messaging.eventsTopic}
Downstream message = ${EMS-Topic.body}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Messaging style messagingStyle	string Not marked required by editor	Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher.
Destination topic	string Not marked required by editor	Broker topic name. Resolve it from the active environment, for example \${properties.messaging.topic}. Consumer topic lists use semicolons.
Message type messageType	string Not marked required by editor	Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented.
JMS message selector messageSelector	string Not marked required by editor	Provider-side JMS selector expression over supported headers/properties. It does not query inside the JSON message body.
Acknowledge mode acknowledgeMode	string Not marked required by editor	Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery.

Field / key	Type / required	Details
Session count maxSessions	number Not marked required by editor	Passed as a session-count option by the wrapper; current engine bridge does not create a configurable receiver-session pool from it.
Flow limit flowLimit	number Not marked required by editor	Passed by the wrapper but not enforced as an engine bridge in-flight limit. Do not rely on it as the only backpressure mechanism.
Receive timeout (ms) receiveTimeout	number Not marked required by editor	Receive wait in milliseconds, default 30000 on the engine bridge path. A timeout can return an empty result; it need not throw a fault.
Durable subscriber durable	boolean Not marked required by editor	Editor option not wired to a durable subscriber constructor in the native engine bridge. Offline topic retention cannot be assumed.
Durable subscription name subscriptionName	string Not marked required by editor	Intended durable subscription identity; current native engine consumer does not use it to create a durable subscription.
Shared subscription sharedSubscription	boolean Not marked required by editor	Editor option not wired to a shared-subscription constructor in the current native engine bridge.

Output

Illustrative runtime output

```
{"body":"event text","headers":{"JMSMessageID":"<provider-id>"},"properties":{},"count":1}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message body body	object string binary Not marked required by editor	Received message body. The engine attempts JSON decoding for text that contains valid JSON; other text remains a string.
JMS standard headers headers	object Not marked required by editor	Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support.
JMS application properties properties	object Not marked required by editor	Application-defined message properties returned by the broker. Standard JMS headers are returned separately.
Client acknowledgement handle ackId	string Not marked required by editor	Opaque, runtime-local confirmation token. Pass the original value to Confirm Message; it is not the business message ID and cannot be reconstructed after restart.

Advanced

Delivery, transactions and performance

Persistent sender workers are reused for the same resolved connection and execution scope. The cache is bounded; changing credentials/configuration frequently or exceeding the cache can create new workers. Native text messages and string application properties are the supported send path. Do not treat all editor choices as implemented native features. Retry after an ambiguous timeout can duplicate a message.

Implemented behavior and limitations

Do not assume messages published while the subscriber is offline are retained. durable, subscriptionName and sharedSubscription are editor-only on the current native engine path. Client acknowledgement has the same early-ack limitation as Queue Receiver.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Troubleshooting procedure

Missing JAR/class/native bridge errors: check the installed driver directory, matching versions and packaged Java bridge. Authentication/destination failures: verify credentials, queue/topic permissions and the resolved environment destination. Timeout: inspect empty-receive results as well as faults. A successful send is not proof that another application consumed or processed the message.

Acceptance test

Acceptance: test online delivery and offline publication separately. Record observed retention and redelivery rather than assuming durable subscription behavior.

Fault	Description
JMSInvalidInputException	The JMS activity input is invalid.
JMSMessageCreateException	The configured JMS message could not be created.
JMSSessionCreateException	A JMS session could not be created.
JMSReceiveException	The JMS message could not be received.
ActivityTimedOutException	The JMS operation reached its configured timeout.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs

EMS Queue Sender

Activities / EMS

Overview

Send a queue message

Use inside a task to send its mapped body to a queue.

Prerequisites

Install the licensed, version-compatible JMS client JARs under the runtime driver root/jms or set Driver JAR directory. Use the bundled Java bridge/runtime. A direct EMS connection requires URL, username and password; JNDI additionally requires its context factory, provider URL and connection-factory settings. Keep Java class names exactly as provided by the driver. Test from the execution machine, not only the developer laptop.

Sends, receives, browses, or acknowledges EMS messages.

Destination, delivery, selector, acknowledgement, and message properties follow the selected EMS operation. Client acknowledgement handles can be passed to Confirm Message.

Activity type: ems; operation: send.

Configuration

Configuration procedure

Select the EMS connection and set queue/destination explicitly. Map message, optional correlationId and application properties. The palette uses operation send; its current base editor contract omits some sender controls, so the supplemental field guide below describes runtime keys as well as editor fields.

Runtime setting	Meaning
destination	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
message	Map the record/message body, not the entire preceding activity envelope unless that envelope is intentionally the payload.
deliveryMode	Persistent or Non-Persistent producer setting. Persistence still depends on broker configuration and is not an exactly-once business guarantee.
priority	JMS priority from 0 to 9; default 4. Ordering and fairness are provider-dependent.
expiration	Message time-to-live in milliseconds; zero means no producer-specified expiry.
correlationId	Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback.
replyTo	Destination attached as JMSReplyTo. Request Reply creates a temporary queue when this is blank.
dynamicProperties	Object/JSON object of application properties. Current messaging code stringifies outgoing property values.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
EMS connection resourceId	resource Required	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL.

Field / key	Type / required	Details
		Connection type: ems

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
message = ${input.order}
destination = ${properties.messaging.outboundQueue}
correlationId = ${input.order.id}
deliveryMode = Persistent
priority = 4
expiration = 0
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message body message	object string binary Required	Map the record/message body, not the entire preceding activity envelope unless that envelope is intentionally the payload.
Dynamic destination override destination	string Not marked required by editor	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
Dynamic JMS properties dynamicProperties	object Not marked required by editor	Object/JSON object of application properties. Current messaging code stringifies outgoing property values.
JMS headers headers	object Not marked required by editor	Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support.

Output

Illustrative runtime output

```
{"messageId": "<provider-id>", "destination": "orders.out", "published": true, "senderReused": true}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message ID messageId	string Not marked required by editor	Identifier returned by the provider, or a locally generated identifier for buffered publishing. It is not proof that a downstream application processed the message.
Resolved destination destination	string Not marked required by editor	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".

Field / key	Type / required	Details
JMS timestamp timestamp	dateTime Not marked required by editor	Timestamp reported by the operation. Kafka native record timestamps are epoch milliseconds; do not assume every connector returns the editor-labelled dateTime representation.

Advanced

Delivery, transactions and performance

Persistent sender workers are reused for the same resolved connection and execution scope. The cache is bounded; changing credentials/configuration frequently or exceeding the cache can create new workers. Native text messages and string application properties are the supported send path. Do not treat all editor choices as implemented native features. Retry after an ambiguous timeout can duplicate a message.

Implemented behavior and limitations

Native send creates a text message; object bodies are JSON-encoded. Message-type, disable-message-ID and disable-timestamp editor options are not fully implemented by the bridge. A warm persistent sender can be faster than the first send; senderReused is diagnostic, not a delivery guarantee.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Troubleshooting procedure

Missing JAR/class/native bridge errors: check the installed driver directory, matching versions and packaged Java bridge. Authentication/destination failures: verify credentials, queue/topic permissions and the resolved environment destination. Timeout: inspect empty-receive results as well as faults. A successful send is not proof that another application consumed or processed the message.

Acceptance test

Acceptance: browse the test queue independently and verify text body, correlation and properties; compare first-send and warm-send timing under the same delivery policy.

Fault	Description
JMSInvalidInputException	The JMS activity input is invalid.
JMSMessageCreateException	The configured JMS message could not be created.
JMSSessionCreateException	A JMS session could not be created.

Fault	Description
JMSSendException	The JMS message could not be sent.
ActivityTimedOutException	The JMS operation reached its configured timeout.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs

EMS Topic Publisher

Activities / EMS

Overview

Publish a topic message

Use inside a task to broadcast a mapped body to the configured topic.

Prerequisites

Install the licensed, version-compatible JMS client JARs under the runtime driver root/jms or set Driver JAR directory. Use the bundled Java bridge/runtime. A direct EMS connection requires URL, username and password; JNDI additionally requires its context factory, provider URL and connection-factory settings. Keep Java class names exactly as provided by the driver. Test from the execution machine, not only the developer laptop.

Sends, receives, browses, or acknowledges EMS messages.

Destination, delivery, selector, acknowledgement, and message properties follow the selected EMS operation. Client acknowledgement handles can be passed to Confirm Message.

Activity type: ems; operation: publish.

Configuration

Configuration procedure

Select the EMS connection, topic/destination property and message mapping. Operation publish selects topic semantics. Its current base editor contract omits some sender controls; use the supplemental runtime-key guide below.

Runtime setting	Meaning
destination	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
message	Map the record/message body, not the entire preceding activity envelope unless that envelope is intentionally the payload.
deliveryMode	Persistent or Non-Persistent producer setting. Persistence still depends on broker configuration and is not an exactly-once business guarantee.
priority	JMS priority from 0 to 9; default 4. Ordering and fairness are provider-dependent.
expiration	Message time-to-live in milliseconds; zero means no producer-specified expiry.
correlationId	Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback.
replyTo	Destination attached as JMSReplyTo. Request Reply creates a temporary queue when this is blank.
dynamicProperties	Object/JSON object of application properties. Current messaging code stringifies outgoing property values.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
EMS connection resourceId	resource Required	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: ems

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
message = ${input.event}
destination = ${properties.messaging.eventsTopic}
deliveryMode = Persistent
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message body message	object string binary Required	Map the record/message body, not the entire preceding activity envelope unless that envelope is intentionally the payload.
Dynamic destination override destination	string Not marked required by editor	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
Dynamic JMS properties dynamicProperties	object Not marked required by editor	Object/JSON object of application properties. Current messaging code stringifies outgoing property values.
JMS headers headers	object Not marked required by editor	Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support.

Output

Illustrative runtime output

```
{"messageId": "<provider-id>", "destination": "events", "published": true}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message ID messageId	string Not marked required by editor	Identifier returned by the provider, or a locally generated identifier for buffered publishing. It is not proof that a downstream application processed the message.
Resolved destination destination	string Not marked required by editor	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
JMS timestamp timestamp	dateTime Not marked required by editor	Timestamp reported by the operation. Kafka native record timestamps are epoch milliseconds; do not assume every connector returns the editor-labelled dateTime representation.

Advanced

Delivery, transactions and performance

Persistent sender workers are reused for the same resolved connection and execution scope. The cache is bounded; changing credentials/configuration frequently or exceeding the cache can create new workers. Native text messages and string application properties are the supported send path. Do not treat all editor choices as implemented native features. Retry after an ambiguous timeout can duplicate a message.

Implemented behavior and limitations

A successful topic publish does not establish delivery to every intended subscriber. Offline retention depends on a genuinely durable provider subscription; the native engine subscriber options do not implement that guarantee. Native text-message restrictions match Queue Sender.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Troubleshooting procedure

Missing JAR/class/native bridge errors: check the installed driver directory, matching versions and packaged Java bridge. Authentication/destination failures: verify credentials, queue/topic permissions and the resolved environment destination. Timeout: inspect empty-receive results as well as faults. A successful send is not proof that another application consumed or processed the message.

Acceptance test

Acceptance: attach two independent online subscribers, publish a unique event and check each body. Test offline behavior separately.

Fault	Description
JMSInvalidInputException	The JMS activity input is invalid.
JMSMessageCreateException	The configured JMS message could not be created.
JMSSessionCreateException	A JMS session could not be created.
JMSSendException	The JMS message could not be sent.
ActivityTimedOutException	The JMS operation reached its configured timeout.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs

EMS Request Reply

Activities / EMS

Overview

Send a request and await a reply

Use when the task needs a synchronous response from another JMS application.

Prerequisites

Install the licensed, version-compatible JMS client JARs under the runtime driver root/jms or set Driver JAR directory. Use the bundled Java bridge/runtime. A direct EMS connection requires URL, username and password; JNDI additionally requires its context factory, provider URL and connection-factory settings. Keep Java class names exactly as provided by the driver. Test from the execution machine, not only the developer laptop.

Sends, receives, browses, or acknowledges EMS messages.

Destination, delivery, selector, acknowledgement, and message properties follow the selected EMS operation. Client acknowledgement handles can be passed to Confirm Message.

Activity type: ems; operation: request_reply.

Configuration

Configuration procedure

Set the request destination and mapped message. Leave replyTo blank for a temporary reply queue. A fixed reply queue requires external correlation isolation; the current bridge does not install a correlation selector on that queue.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
EMS connection resourceId	resource Required	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: ems
Messaging style messagingStyle	select Not marked required by editor	Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher. Choices: Queue, Topic, Generic
Destination destination	text Not marked required by editor	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
Message type messageType	select Not marked required by editor	Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented. Choices: Text, Bytes, Map, Object, Object Ref, Simple, Stream, XML Text
Delivery mode deliveryMode	select Not marked required by editor	Persistent or Non-Persistent producer setting. Persistence still depends on broker configuration and is not an exactly-once business guarantee. Choices: Persistent, Non-Persistent
Priority (0-9) priority	number Not marked required by editor	JMS priority from 0 to 9; default 4. Ordering and fairness are provider-dependent.

Field / key	Type / required	Details
Expiration (ms) expiration	number Not marked required by editor	Message time-to-live in milliseconds; zero means no producer-specified expiry.
JMS correlation ID correlationId	text Not marked required by editor	Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback.
Reply-to destination replyTo	text Not marked required by editor	Destination attached as JMSReplyTo. Request Reply creates a temporary queue when this is blank.
Disable JMS message ID disableMessageId	boolean Not marked required by editor	Editor option not forwarded to the current native sender; do not assume identifiers are disabled.
Disable JMS timestamp disableTimestamp	boolean Not marked required by editor	Editor option not forwarded to the current native sender; do not assume timestamps are disabled.
JMS application / dynamic properties (JSON) dynamicProperties	textarea Not marked required by editor	Object/JSON object of application properties. Current messaging code stringifies outgoing property values.
Request timeout (ms) requestTimeout	number Not marked required by editor	Editor request timeout; current engine bridge reads receiveTimeout for its actual wait, not this field. Verify the effective wait in your deployed archive.
Use temporary reply destination temporaryReplyDestination	boolean Not marked required by editor	Editor flag; the current native request bridge chooses a temporary queue based on replyTo being blank, not this flag.

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
message = ${input.request}
destination = ${properties.messaging.requestQueue}
replyTo = <leave blank for a temporary queue>
correlationId = ${input.request.id}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message body message	object string binary Required	Map the record/message body, not the entire preceding activity envelope unless that envelope is intentionally the payload.
Dynamic destination override destination	string Not marked required by editor	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
Dynamic JMS properties dynamicProperties	object Not marked required by editor	Object/JSON object of application properties. Current messaging code stringifies outgoing property values.

Field / key	Type / required	Details
JMS headers headers	object Not marked required by editor	Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support.
Messaging style messagingStyle	string Not marked required by editor	Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher.
Message type messageType	string Not marked required by editor	Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented.
Delivery mode deliveryMode	string Not marked required by editor	Persistent or Non-Persistent producer setting. Persistence still depends on broker configuration and is not an exactly-once business guarantee.
Priority (0-9) priority	number Not marked required by editor	JMS priority from 0 to 9; default 4. Ordering and fairness are provider-dependent.
Expiration (ms) expiration	number Not marked required by editor	Message time-to-live in milliseconds; zero means no producer-specified expiry.
JMS correlation ID correlationId	string Not marked required by editor	Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback.
Reply-to destination replyTo	string Not marked required by editor	Destination attached as JMSReplyTo. Request Reply creates a temporary queue when this is blank.
Disable JMS message ID disableMessageId	boolean Not marked required by editor	Editor option not forwarded to the current native sender; do not assume identifiers are disabled.
Disable JMS timestamp disableTimestamp	boolean Not marked required by editor	Editor option not forwarded to the current native sender; do not assume timestamps are disabled.
Request timeout (ms) requestTimeout	number Not marked required by editor	Editor request timeout; current engine bridge reads receiveTimeout for its actual wait, not this field. Verify the effective wait in your deployed archive.
Use temporary reply destination temporaryReplyDestination	boolean Not marked required by editor	Editor flag; the current native request bridge chooses a temporary queue based on replyTo being blank, not this flag.

Output

Illustrative runtime output

```
{"received":true,"body":{"accepted":true},"headers":{},"properties":{},"count":1}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Reply message body body	object string binary Not marked required by editor	Received message body. The engine attempts JSON decoding for text that contains valid JSON; other text remains a string.

Field / key	Type / required	Details
Reply JMS headers headers	object Not marked required by editor	Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support.
Reply properties properties	object Not marked required by editor	Application-defined message properties returned by the broker. Standard JMS headers are returned separately.

Advanced

Delivery, transactions and performance

Persistent sender workers are reused for the same resolved connection and execution scope. The cache is bounded; changing credentials/configuration frequently or exceeding the cache can create new workers. Native text messages and string application properties are the supported send path. Do not treat all editor choices as implemented native features. Retry after an ambiguous timeout can duplicate a message.

Implemented behavior and limitations

The engine wrapper uses `receiveTimeout` (default 30000 ms), not the displayed `requestTimeout`, for the actual reply wait. The `temporaryReplyDestination` flag is not consumed; blank `replyTo` controls temporary queue creation. Concurrent callers sharing a fixed reply queue can receive the wrong reply. A timeout can return `received=false/count=0` rather than throw.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Troubleshooting procedure

Missing JAR/class/native bridge errors: check the installed driver directory, matching versions and packaged Java bridge. Authentication/destination failures: verify credentials, queue/topic permissions and the resolved environment destination. Timeout: inspect empty-receive results as well as faults. A successful send is not proof that another application consumed or processed the message.

Acceptance test

Acceptance: test a successful reply, no reply, and concurrent requests. Use temporary reply queues until shared-destination correlation is qualified.

Fault	Description
JMSInvalidInputException	The JMS activity input is invalid.
JMSMessageCreateException	The configured JMS message could not be created.
JMSSessionCreateException	A JMS session could not be created.
JMSSendException	The JMS message could not be sent.
ActivityTimedOutException	The JMS operation reached its configured timeout.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#); [backend/app/runtime.py](#); [backend/app/sap.py](#); [backend/app/java_bridge.py](#); [frontend/scripts/connector-documentation.mjs](#)

EMS Reply

Activities / EMS

Overview

Send a response to a JMS request

Use after receiving a request whose reply destination and correlation information are available.

Prerequisites

Install the licensed, version-compatible JMS client JARs under the runtime driver root/jms or set Driver JAR directory. Use the bundled Java bridge/runtime. A direct EMS connection requires URL, username and password; JNDI additionally requires its context factory, provider URL and connection-factory settings. Keep Java class names exactly as provided by the driver. Test from the execution machine, not only the developer laptop.

Sends, receives, browses, or acknowledges EMS messages.

Destination, delivery, selector, acknowledgement, and message properties follow the selected EMS operation. Client acknowledgement handles can be passed to Confirm Message.

Activity type: ems; operation: reply.

Configuration

Configuration procedure

Map the response body and explicit reply destination. For the engine path this is an outgoing send; it does not automatically derive destination from a listener context. Map the request correlation ID, or request message ID if that is the agreed protocol.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
EMS connection resourceId	resource Required	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: ems
Messaging style messagingStyle	select Not marked required by editor	Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher. Choices: Queue, Topic, Generic
Reply destination destination	text Not marked required by editor	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
Message type messageType	select Not marked required by editor	Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented. Choices: Text, Bytes, Map, Object, Object Ref, Simple, Stream, XML Text
Request correlation ID correlationId	text Not marked required by editor	Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback.
Field / key	Type / required	Details
Delivery mode deliveryMode	select Not marked required by	Persistent or Non-Persistent producer setting. Persistence still depends on broker configuration and is not an exactly-once

Field / key	Type / required	Details
	editor	business guarantee. Choices: Persistent, Non-Persistent
Priority (0-9) priority	number Not marked required by editor	JMS priority from 0 to 9; default 4. Ordering and fairness are provider-dependent.
Expiration (ms) expiration	number Not marked required by editor	Message time-to-live in milliseconds; zero means no producer-specified expiry.
JMS correlation ID correlationId	text Not marked required by editor	Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback.
Reply-to destination replyTo	text Not marked required by editor	Destination attached as JMSReplyTo. Request Reply creates a temporary queue when this is blank.
Disable JMS message ID disableMessageId	boolean Not marked required by editor	Editor option not forwarded to the current native sender; do not assume identifiers are disabled.
Disable JMS timestamp disableTimestamp	boolean Not marked required by editor	Editor option not forwarded to the current native sender; do not assume timestamps are disabled.
JMS application / dynamic properties (JSON) dynamicProperties	textarea Not marked required by editor	Object/JSON object of application properties. Current messaging code stringifies outgoing property values.

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
message = ${input.response}
destination = ${properties.messaging.replyQueue}
correlationId = ${EMS-Receive.headers.JMSCorrelationID}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message body message	object string binary Required	Map the record/message body, not the entire preceding activity envelope unless that envelope is intentionally the payload.
Dynamic destination override destination	string Not marked required by editor	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
Dynamic JMS properties dynamicProperties	object Not marked required by editor	Object/JSON object of application properties. Current messaging code stringifies outgoing property values.
JMS headers headers	object Not marked required by editor	Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support.

Field / key	Type / required	Details
Messaging style messagingStyle	string Not marked required by editor	Queue/topic design hint. The current engine primarily chooses destination kind from the operation; do not use this to turn a queue sender into a topic publisher.
Message type messageType	string Not marked required by editor	Editor message-type choice. The native send implementation creates text messages and JSON-encodes non-string input; the full listed message-type matrix is not implemented.
Request correlation ID correlationId	string Not marked required by editor	Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback.
Delivery mode deliveryMode	string Not marked required by editor	Persistent or Non-Persistent producer setting. Persistence still depends on broker configuration and is not an exactly-once business guarantee.
Priority (0-9) priority	number Not marked required by editor	JMS priority from 0 to 9; default 4. Ordering and fairness are provider-dependent.
Expiration (ms) expiration	number Not marked required by editor	Message time-to-live in milliseconds; zero means no producer-specified expiry.
JMS correlation ID correlationId	string Not marked required by editor	Correlation identifier attached to the outgoing message. For replies use the request correlation ID, or its message ID when the request protocol requires that fallback.
Reply-to destination replyTo	string Not marked required by editor	Destination attached as JMSReplyTo. Request Reply creates a temporary queue when this is blank.
Disable JMS message ID disableMessageId	boolean Not marked required by editor	Editor option not forwarded to the current native sender; do not assume identifiers are disabled.
Disable JMS timestamp disableTimestamp	boolean Not marked required by editor	Editor option not forwarded to the current native sender; do not assume timestamps are disabled.

Output

Illustrative runtime output

```
{"messageId":"<provider-id>","destination":"replies","published":true}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message ID messageId	string Not marked required by editor	Identifier returned by the provider, or a locally generated identifier for buffered publishing. It is not proof that a downstream application processed the message.
Resolved destination destination	string Not marked required by editor	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".

Field / key	Type / required	Details
JMS timestamp timestamp	dateTime Not marked required by editor	Timestamp reported by the operation. Kafka native record timestamps are epoch milliseconds; do not assume every connector returns the editor-labelled dateTime representation.

Advanced

Delivery, transactions and performance

Persistent sender workers are reused for the same resolved connection and execution scope. The cache is bounded; changing credentials/configuration frequently or exceeding the cache can create new workers. Native text messages and string application properties are the supported send path. Do not treat all editor choices as implemented native features. Retry after an ambiguous timeout can duplicate a message.

Implemented behavior and limitations

The request JMSReplyTo value must be usable as a destination name before mapping it. Do not assume the generic headers object sets native JMS correlation/reply headers; use the dedicated correlationId and destination fields. A reply does not prove the requester received it.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Troubleshooting procedure

Missing JAR/class/native bridge errors: check the installed driver directory, matching versions and packaged Java bridge. Authentication/destination failures: verify credentials, queue/topic permissions and the resolved environment destination. Timeout: inspect empty-receive results as well as faults. A successful send is not proof that another application consumed or processed the message.

Acceptance test

Acceptance: send a unique correlation ID through a requester/responder pair and verify it on the response.

Fault	Description
JMSInvalidInputException	The JMS activity input is invalid.
JMSMessageCreateException	The configured JMS message could not be created.
JMSSessionCreateException	A JMS session could not be created.

Fault	Description
JMSSendException	The JMS message could not be sent.
ActivityTimedOutException	The JMS operation reached its configured timeout.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs

Get JMS Queue Message

Activities / JMS

Overview

Runs Get JMS Queue Message on the task execution path.

Activity type: jms; operation: get_queue_message.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
JMS connection resourceId	resource Required	Connection type: jms
Queue name queue	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Message type messageType	select Not marked required by editor	Choices: Text, Bytes, Map, Object, Object Ref, Simple, Stream, XML Text
JMS message selector messageSelector	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Acknowledge mode acknowledgeMode	select Not marked required by editor	Choices: Auto, Client, Dups OK, Explicit Client, Explicit Client Dups OK, Transactional
Receive timeout (ms) receiveTimeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Queue name queue	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Message type messageType	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
JMS message selector messageSelector	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Acknowledge mode acknowledgeMode	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Receive timeout (ms) receiveTimeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message body body	object string binary Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
JMS standard headers headers	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
JMS application properties properties	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Client acknowledgement handle ackId	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
JMSInvalidInputException	The JMS activity input is invalid.
JMSMessageCreateException	The configured JMS message could not be created.
JMSSessionCreateException	A JMS session could not be created.
JMSReceiveException	The JMS message could not be received.
ActivityTimedOutException	The JMS operation reached its configured timeout.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

JMS Receive Message

Activities / JMS

Overview

Runs JMS Receive Message on the task execution path.

Activity type: jms; operation: receive_message.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
JMS connection resourceId	resource Required	Connection type: jms
Messaging style messagingStyle	select Not marked required by editor	Choices: Queue, Topic, Generic
Destination destination	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Message type messageType	select Not marked required by editor	Choices: Text, Bytes, Map, Object, Object Ref, Simple, Stream, XML Text
JMS message selector messageSelector	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Acknowledge mode acknowledgeMode	select Not marked required by editor	Choices: Auto, Client, Dups OK, Explicit Client, Explicit Client Dups OK, Transactional
Maximum sessions maxSessions	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Flow limit flowLimit	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Receive timeout (ms) receiveTimeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Durable subscription durable	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Durable subscription name subscriptionName	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Shared subscription sharedSubscription	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Messaging style messagingStyle	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Destination destination	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Message type messageType	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
JMS message selector messageSelector	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Acknowledge mode acknowledgeMode	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Maximum sessions maxSessions	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Flow limit flowLimit	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Receive timeout (ms) receiveTimeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Durable subscription durable	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Durable subscription name subscriptionName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Shared subscription sharedSubscription	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message body body	object string binary Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
JMS standard headers headers	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
JMS application properties properties	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Client acknowledgement handle ackId	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
JMSInvalidInputException	The JMS activity input is invalid.
JMSMessageCreateException	The configured JMS message could not be created.
JMSSessionCreateException	A JMS session could not be created.
JMSReceiveException	The JMS message could not be received.
ActivityTimedOutException	The JMS operation reached its configured timeout.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

JMS Request Reply

Activities / JMS

Overview

Runs JMS Request Reply on the task execution path.

Activity type: jms; operation: request_reply.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
JMS connection resourceId	resource Required	Connection type: jms
Messaging style messagingStyle	select Not marked required by editor	Choices: Queue, Topic, Generic
Destination destination	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Message type messageType	select Not marked required by editor	Choices: Text, Bytes, Map, Object, Object Ref, Simple, Stream, XML Text
Delivery mode deliveryMode	select Not marked required by editor	Choices: Persistent, Non-Persistent
Priority (0-9) priority	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Expiration (ms) expiration	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
JMS correlation ID correlationId	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Reply-to destination replyTo	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Disable JMS message ID disableMessageId	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Disable JMS timestamp disableTimestamp	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
JMS application / dynamic properties (JSON) dynamicProperties	textarea Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Request timeout (ms) requestTimeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Use temporary reply destination temporaryReplyDestination	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message body message	object string binary Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic destination override destination	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic JMS properties dynamicProperties	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
JMS headers headers	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Messaging style messagingStyle	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Message type messageType	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Delivery mode deliveryMode	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Priority (0-9) priority	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Expiration (ms) expiration	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
JMS correlation ID correlationId	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Reply-to destination replyTo	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Disable JMS message ID disableMessageId	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Disable JMS timestamp disableTimestamp	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Request timeout (ms) requestTimeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Use temporary reply destination temporaryReplyDestination	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Reply message body body	object string binary Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Reply JMS headers headers	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Reply properties properties	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
JMSInvalidInputException	The JMS activity input is invalid.
JMSMessageCreateException	The configured JMS message could not be created.
JMSSessionCreateException	A JMS session could not be created.

Fault	Description
JMSSendException	The JMS message could not be sent.
ActivityTimedOutException	The JMS operation reached its configured timeout.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

JMS Send Message

Activities / JMS

Overview

Runs JMS Send Message on the task execution path.

Activity type: jms; operation: send_message.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
JMS connection resourceId	resource Required	Connection type: jms
Messaging style messagingStyle	select Not marked required by editor	Choices: Queue, Topic, Generic
Destination destination	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Message type messageType	select Not marked required by editor	Choices: Text, Bytes, Map, Object, Object Ref, Simple, Stream, XML Text
Delivery mode deliveryMode	select Not marked required by editor	Choices: Persistent, Non-Persistent
Priority (0-9) priority	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Expiration (ms) expiration	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
JMS correlation ID correlationId	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Reply-to destination replyTo	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Disable JMS message ID disableMessageId	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Disable JMS timestamp disableTimestamp	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
JMS application / dynamic properties (JSON) dynamicProperties	textarea Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message body message	object string binary Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic destination override destination	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic JMS properties dynamicProperties	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
JMS headers headers	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Messaging style messagingStyle	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Message type messageType	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Delivery mode deliveryMode	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Priority (0-9) priority	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Expiration (ms) expiration	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
JMS correlation ID correlationId	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Reply-to destination replyTo	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Disable JMS message ID disableMessageId	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Disable JMS timestamp disableTimestamp	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message ID messageld	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Resolved destination destination	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
JMS timestamp timestamp	dateTime Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
JMSInvalidInputException	The JMS activity input is invalid.
JMSMessageCreateException	The configured JMS message could not be created.
JMSSessionCreateException	A JMS session could not be created.
JMSSendException	The JMS message could not be sent.
ActivityTimedOutException	The JMS operation reached its configured timeout.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

Reply to JMS Message

Activities / JMS

Overview

Runs Reply to JMS Message on the task execution path.

Activity type: jms; operation: reply_message.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
JMS connection resourceId	resource Required	Connection type: jms
Messaging style messagingStyle	select Not marked required by editor	Choices: Queue, Topic, Generic
Reply destination destination	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Message type messageType	select Not marked required by editor	Choices: Text, Bytes, Map, Object, Object Ref, Simple, Stream, XML Text
Request correlation ID correlationId	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Delivery mode deliveryMode	select Not marked required by editor	Choices: Persistent, Non-Persistent
Priority (0-9) priority	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Expiration (ms) expiration	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
JMS correlation ID correlationId	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Reply-to destination replyTo	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Disable JMS message ID disableMessageId	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Disable JMS timestamp disableTimestamp	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
JMS application / dynamic properties (JSON) dynamicProperties	textarea Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message body message	object string binary Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic destination override destination	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic JMS properties dynamicProperties	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
JMS headers headers	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Messaging style messagingStyle	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Message type messageType	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Request correlation ID correlationId	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Delivery mode deliveryMode	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Priority (0-9) priority	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Expiration (ms) expiration	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
JMS correlation ID correlationId	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Reply-to destination replyTo	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Disable JMS message ID disableMessageId	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Disable JMS timestamp disableTimestamp	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message ID messageId	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Resolved destination destination	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
JMS timestamp timestamp	dateTime Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
JMSInvalidInputException	The JMS activity input is invalid.
JMSMessageCreateException	The configured JMS message could not be created.
JMSSessionCreateException	A JMS session could not be created.
JMSSendException	The JMS message could not be sent.
ActivityTimedOutException	The JMS operation reached its configured timeout.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose

sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

Wait for JMS Request

Activities / JMS

Overview

Runs Wait for JMS Request on the task execution path.

Activity type: jms; operation: wait_request.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
JMS connection resourceId	resource Required	Connection type: jms
Messaging style messagingStyle	select Not marked required by editor	Choices: Queue, Topic, Generic
Destination destination	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Message type messageType	select Not marked required by editor	Choices: Text, Bytes, Map, Object, Object Ref, Simple, Stream, XML Text
JMS message selector messageSelector	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Acknowledge mode acknowledgeMode	select Not marked required by editor	Choices: Auto, Client, Dups OK, Explicit Client, Explicit Client Dups OK, Transactional
Receive timeout (ms) receiveTimeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Messaging style messagingStyle	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Destination destination	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Message type messageType	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
JMS message selector messageSelector	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Acknowledge mode acknowledgeMode	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Receive timeout (ms) receiveTimeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
JMS message body body	object string binary Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
JMS standard headers headers	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
JMS application properties properties	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Client acknowledgement handle ackId	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
JMSInvalidInputException	The JMS activity input is invalid.
JMSMessageCreateException	The configured JMS message could not be created.
JMSSessionCreateException	A JMS session could not be created.
JMSReceiveException	The JMS message could not be received.
ActivityTimedOutException	The JMS operation reached its configured timeout.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose

sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: `frontend/src/ActivityEditor.tsx`

Kafka Receive Message

Activities / Kafka

Overview

Start work from Kafka records

Use as an event starter. Run/Debug registers continuous starter processing; one received batch becomes flow input. Pauses/breakpoints can delay downstream work.

Prerequisites

Install the matching confluent-kafka package on the execution host. Configure a Kafka shared connection with reachable bootstrap servers and the required TLS/SASL credentials. Verify advertised broker addresses are reachable from that host; a successful TCP connection to one bootstrap address is not sufficient. These pages describe the Studio/engine runtime; qualify exported direct Python separately.

Produces, receives, or commits Apache Kafka records.

Broker security comes from the shared Kafka connection. Topic, key, headers, value, partitions, offsets, and acknowledgement handles are available for hierarchical mapping.

Activity type: kafka; operation: receive.

Configuration

Configuration procedure

Set topic and groupId explicitly. Select deserializers matching the producer. For Manual mode disable enableAutoCommit and map acknowledgement handles only after processing succeeds. Use a separate test group to avoid consuming production offsets.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Kafka connection resourceId	resource Required	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: kafka
Consumer group ID groupId	text Not marked required by editor	Consumer group used for assignment and stored offsets. Separate applications requiring independent consumption need separate groups.
Topic names topic	text Not marked required by editor	Broker topic name. Resolve it from the active environment, for example \${properties.messaging.topic}. Consumer topic lists use semicolons.
Assign custom partitions assignCustomPartition	boolean Not marked required by editor	Enable explicit partition selection; otherwise the client chooses/subscribes using its normal partitioning and assignment behavior.
Partition IDs / ranges partitionIds	text Not marked required by editor	Comma/semicolon-separated partition IDs or inclusive ranges used for explicit assignment.
Consumer count consumerCount	number Not marked required by editor	Exposed by the editor; the engine receive path creates one consumer per call and does not implement a consumer pool from this field.

Field / key	Type / required	Details
Key deserializer keyDeserializer	select Not marked required by editor	Incoming key decoding; Byte Array exposes base64 text, JSON parses UTF-8 JSON, String decodes text. Choices: String, Byte Array, Avro Schema, JSON
Value deserializer valueDeserializer	select Not marked required by editor	Incoming record-value decoding. Invalid JSON raises a decode error; Avro is not implemented by this engine path. Choices: String, Byte Array, Avro Schema, JSON
Use Schema Registry useRegistry	boolean Not marked required by editor	Editor registry flag; setting this does not activate an Avro serializer in the current engine.
Schema Registry URL schemaRegistryUrl	text Not marked required by editor	Registry endpoint metadata. This engine does not implement registry-backed Avro conversion merely by setting the URL.
Key schema subject : version keySchemaSubject	text Not marked required by editor	Intended registry key subject/version; not resolved by the current engine serializer.
Value schema subject : version valueSchemaSubject	text Not marked required by editor	Intended registry value subject/version; not resolved by the current engine serializer.
Acknowledgement mode acknowledgeMode	select Not marked required by editor	Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery. Choices: Auto, Manual
Enable auto commit enableAutoCommit	boolean Not marked required by editor	Native periodic offset commit toggle. For manual acknowledgement keep this disabled; otherwise offsets can advance before downstream work finishes.
Auto offset reset autoOffsetReset	select Not marked required by editor	Policy used when no valid committed offset exists, commonly earliest or latest. It does not rewind a group that already has committed offsets.
Fetch minimum bytes fetchMinBytes	number Not marked required by editor	Minimum fetch bytes requested from the broker. Larger values can add latency at low traffic.
Maximum poll records maxPollRecords	number Not marked required by editor	Editor batch-size setting. The engine currently passes max.poll.records to its native consumer; this Java-style key may be rejected by the installed native client. Treat this as a known compatibility issue, not a guaranteed supported control.
Session timeout (ms) sessionTimeoutMs	number Not marked required by editor	Consumer failure-detection timeout in milliseconds; must satisfy broker limits.
Heartbeat interval (ms) heartbeatIntervalMs	number Not marked required by editor	Consumer heartbeat interval in milliseconds; coordinate with session timeout.
Additional consumer properties (JSON) additionalProperties	textarea Not marked required by editor	JSON object of native client configuration values merged into the producer/consumer configuration. Native keys differ from Java client keys; invalid keys are rejected by the installed client.

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```

topic = ${properties.messaging.ordersTopic}
groupId = ${properties.messaging.consumerGroup}
valueDeserializer = JSON
acknowledgeMode = Manual
enableAutoCommit = false
Downstream source = ${Kafka-Receive.messages}

```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Consumer group ID groupId	string Not marked required by editor	Consumer group used for assignment and stored offsets. Separate applications requiring independent consumption need separate groups.
Topic names topic	string Not marked required by editor	Broker topic name. Resolve it from the active environment, for example \${properties.messaging.topic}. Consumer topic lists use semicolons.
Assign custom partitions assignCustomPartition	boolean Not marked required by editor	Enable explicit partition selection; otherwise the client chooses/subscribes using its normal partitioning and assignment behavior.
Partition IDs / ranges partitionIds	string Not marked required by editor	Comma/semicolon-separated partition IDs or inclusive ranges used for explicit assignment.
Consumer count consumerCount	number Not marked required by editor	Exposed by the editor; the engine receive path creates one consumer per call and does not implement a consumer pool from this field.
Key deserializer keyDeserializer	string Not marked required by editor	Incoming key decoding; Byte Array exposes base64 text, JSON parses UTF-8 JSON, String decodes text.
Value deserializer valueDeserializer	string Not marked required by editor	Incoming record-value decoding. Invalid JSON raises a decode error; Avro is not implemented by this engine path.
Use Schema Registry useRegistry	boolean Not marked required by editor	Editor registry flag; setting this does not activate an Avro serializer in the current engine.
Schema Registry URL schemaRegistryUrl	string Not marked required by editor	Registry endpoint metadata. This engine does not implement registry-backed Avro conversion merely by setting the URL.
Key schema subject : version keySchemaSubject	string Not marked required by editor	Intended registry key subject/version; not resolved by the current engine serializer.
Value schema subject : version valueSchemaSubject	string Not marked required by editor	Intended registry value subject/version; not resolved by the current engine serializer.
Acknowledgement mode acknowledgeMode	string Not marked required by editor	Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery.
Enable auto commit enableAutoCommit	boolean Not marked required by editor	Native periodic offset commit toggle. For manual acknowledgement keep this disabled; otherwise offsets can advance before downstream work finishes.

Field / key	Type / required	Details
Auto offset reset autoOffsetReset	string Not marked required by editor	Policy used when no valid committed offset exists, commonly earliest or latest. It does not rewind a group that already has committed offsets.
Fetch minimum bytes fetchMinBytes	number Not marked required by editor	Minimum fetch bytes requested from the broker. Larger values can add latency at low traffic.
Maximum poll records maxPollRecords	number Not marked required by editor	Editor batch-size setting. The engine currently passes max.poll.records to its native consumer; this Java-style key may be rejected by the installed native client. Treat this as a known compatibility issue, not a guaranteed supported control.
Session timeout (ms) sessionTimeoutMs	number Not marked required by editor	Consumer failure-detection timeout in milliseconds; must satisfy broker limits.
Heartbeat interval (ms) heartbeatIntervalMs	number Not marked required by editor	Consumer heartbeat interval in milliseconds; coordinate with session timeout.
Additional consumer properties (JSON) additionalProperties	string Not marked required by editor	JSON object of native client configuration values merged into the producer/consumer configuration. Native keys differ from Java client keys; invalid keys are rejected by the installed client.

Output

Illustrative runtime output

```
{
  "messages": [
    {
      "data": {
        "id": "A100",
        "key": "A100",
        "topic": "orders",
        "partition": 0,
        "offset": 42,
        "ackId": "<handle>"
      },
      "count": 1,
      "ackId": "<handle>",
      "ackIds": [ "<handle>" ]
    }
  ]
}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Kafka records messages	array Not marked required by editor	Array of received records with data, key, topic, partition, offset, timestamp, headers and optional ackId.
Record count count	integer Not marked required by editor	Number of records returned by this receive attempt. A zero count means no message was returned, not successful business processing.
Topic topic	string Not marked required by editor	Broker topic name. Resolve it from the active environment, for example \${properties.messaging.topic}. Consumer topic lists use semicolons.
Partition partition	integer Not marked required by editor	Partition number on each returned message. The engine receive result nests it under messages[].
Offset offset	long Not marked required by editor	Kafka offset within one partition; it is not globally ordered across the topic.
Record timestamp timestamp	dateTime Not marked required by editor	Timestamp reported by the operation. Kafka native record timestamps are epoch milliseconds; do not assume every connector returns the editor-labelled dateTime representation.

Field / key	Type / required	Details
Record headers headers	object Not marked required by editor	Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support.
Manual acknowledgement handle ackId	string Not marked required by editor	Opaque, runtime-local confirmation token. Pass the original value to Confirm Message; it is not the business message ID and cannot be reconstructed after restart.

Advanced

Delivery, transactions and performance

Separate local enqueue, broker acknowledgement and downstream processing. Monitor background publisher errors and shutdown draining. Use stable business IDs and idempotent consumers when retries are possible. Producer reuse reduces warm-send overhead, but first connection, TLS, metadata and broker availability still affect latency; no fixed latency is guaranteed.

Implemented behavior and limitations

Runtime records live under `messages[]`, not the editor top-level topic/partition fields. `consumerCount` is not implemented as a pool in the engine receive call. `max.poll.records` is forwarded as a native property and may be rejected by `confluent-kafka`. Manual commits can advance a partition past earlier records, so preserve processing/commit order. Do not claim this receive path is production-qualified without a provider test.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: <code>"\${properties.advanced.logPayload}"</code>

This operation does not expose outbound retry in the common Advanced settings.

Errors

Troubleshooting procedure

Connection/authentication failures: verify DNS, advertised listeners, TLS chain and ACLs on the runtime host. Serialization failures: compare actual payload type with the chosen serializer. Queue full/timeouts: inspect broker latency and producer buffer pressure before increasing limits. Consumer configuration errors may be caused by Java-style options passed to the native client. Capture the resolved non-secret configuration and the full runtime error.

Acceptance test

Acceptance: send two distinguishable records, inspect `messages[]`.data, test failure before confirmation and verify actual committed offsets and redelivery.

Fault	Description
<code>KafkaConnectionException</code>	The Kafka connection or broker is unavailable.
<code>KafkaSerializationException</code>	The record key or value could not be serialized/deserialized.
<code>KafkaProducerException</code>	The producer could not publish the record.
<code>KafkaConsumerException</code>	The consumer could not fetch or acknowledge records.
<code>KafkaSchemaRegistryException</code>	The Avro schema or registry subject could not be resolved.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs

Kafka Send Message

Activities / Kafka

Overview

Publish a record

Use inside a running flow to enqueue or confirm one record on a Kafka topic.

Prerequisites

Install the matching confluent-kafka package on the execution host. Configure a Kafka shared connection with reachable bootstrap servers and the required TLS/SASL credentials. Verify advertised broker addresses are reachable from that host; a successful TCP connection to one bootstrap address is not sufficient. These pages describe the Studio/engine runtime; qualify exported direct Python separately.

Produces, receives, or commits Apache Kafka records.

Broker security comes from the shared Kafka connection. Topic, key, headers, value, partitions, offsets, and acknowledgement handles are available for hierarchical mapping.

Activity type: kafka; operation: send.

Configuration

Configuration procedure

Select the Kafka resource; bind topic to an environment property. Select JSON for an object payload, or String for already-serialized text. Start with explicit acks and waitForDelivery settings. Partition selection is optional; use a stable key before forcing a partition.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Kafka connection resourceId	resource Required	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: kafka
Topic name topic	text Not marked required by editor	Broker topic name. Resolve it from the active environment, for example \${properties.messaging.topic}. Consumer topic lists use semicolons.
Assign custom partition assignCustomPartition	boolean Not marked required by editor	Enable explicit partition selection; otherwise the client chooses/subscribes using its normal partitioning and assignment behavior.
Partition ID partitionId	number Not marked required by editor	Explicit Kafka partition when assignCustomPartition is enabled. It must exist in the target topic.
Key serializer keySerializer	select Not marked required by editor	Outgoing key conversion: String, Byte Array or JSON. Avro selection currently raises a registry-runtime error in the engine. Choices: String, Byte Array, Avro Schema, JSON
Value serializer valueSerializer	select Not marked required by editor	Outgoing value conversion. Select JSON for a structured object; String converts using string representation and is not a JSON encoder. Choices: String, Byte Array, Avro Schema, JSON

Field / key	Type / required	Details
Use Schema Registry useRegistry	boolean Not marked required by editor	Editor registry flag; setting this does not activate an Avro serializer in the current engine.
Schema Registry URL schemaRegistryUrl	text Not marked required by editor	Registry endpoint metadata. This engine does not implement registry-backed Avro conversion merely by setting the URL.
Key schema subject : version keySchemaSubject	text Not marked required by editor	Intended registry key subject/version; not resolved by the current engine serializer.
Value schema subject : version valueSchemaSubject	text Not marked required by editor	Intended registry value subject/version; not resolved by the current engine serializer.
Wait for broker confirmation waitForDelivery	boolean Not marked required by editor	False normally returns after local enqueue into a reused producer. True waits for delivery confirmation. Transactions and one-shot producers also force waiting.
Acknowledgements acks	select Not marked required by editor	Broker acknowledgement policy: 0, 1 or all. This differs from the waitForDelivery switch; acks=0 cannot establish broker durability. Choices: 0, 1, all
Compression type compressionType	select Not marked required by editor	Client batch compression. Select a codec supported by the installed client and broker; compression trades CPU for network bandwidth. Choices: none, gzip, snappy, lz4, zstd
Producer retries retries	number Not marked required by editor	Native producer retry setting. Separate from MINA Advanced retries; application retries can repeat a send after an ambiguous timeout.
Buffer memory (bytes) bufferMemory	number Not marked required by editor	Local producer queue budget in bytes, translated to queue.buffering.max.kbytes. A full queue applies backpressure/errors; it is not durable storage.
Batch size batchSize	number Not marked required by editor	Target producer batch size in bytes, default 16384 in the engine. Larger batches may improve throughput at the expense of latency.
Linger (ms) lingerMs	number Not marked required by editor	Maximum batching delay requested from the producer, default 0 in the engine. Increasing it intentionally adds buffering delay.
Maximum request size maxRequestSize	number Not marked required by editor	Maximum native message size setting, default 1048576 bytes. Broker/topic limits can be lower.
Transactional ID transactionalId	text Not marked required by editor	Enables native transaction initialization/begin/commit for a send and forces confirmation. Must be unique for concurrently active producers; it is not a multi-activity transaction group.
Enable idempotence enableIdempotence	boolean Not marked required by editor	Requests native producer idempotence. Must be compatible with other native settings and does not deduplicate arbitrary repeated business calls.
Additional producer properties (JSON) additionalProperties	textarea Not marked required by editor	JSON object of native client configuration values merged into the producer/consumer configuration. Native keys differ from Java client keys; invalid keys are rejected by the installed client.

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
message = ${input.order}
key = ${input.order.id}
topic = ${properties.messaging.ordersTopic}
valueSerializer = JSON
waitForDelivery = true
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Kafka record value message	object string binary Required	Map the record/message body, not the entire preceding activity envelope unless that envelope is intentionally the payload.
Kafka record key key	string binary Not marked required by editor	Kafka record key. Use a stable business key when partition locality matters; preserve the matching serializer.
Kafka headers headers	object Not marked required by editor	Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support.
Dynamic topic override topic	string Not marked required by editor	Broker topic name. Resolve it from the active environment, for example <code>\${properties.messaging.topic}</code> . Consumer topic lists use semicolons.
Dynamic partition partitionId	integer Not marked required by editor	Explicit Kafka partition when <code>assignCustomPartition</code> is enabled. It must exist in the target topic.
Assign custom partition <code>assignCustomPartition</code>	boolean Not marked required by editor	Enable explicit partition selection; otherwise the client chooses/subscribes using its normal partitioning and assignment behavior.
Key serializer <code>keySerializer</code>	string Not marked required by editor	Outgoing key conversion: String, Byte Array or JSON. Avro selection currently raises a registry-runtime error in the engine.
Value serializer <code>valueSerializer</code>	string Not marked required by editor	Outgoing value conversion. Select JSON for a structured object; String converts using string representation and is not a JSON encoder.
Use Schema Registry <code>useRegistry</code>	boolean Not marked required by editor	Editor registry flag; setting this does not activate an Avro serializer in the current engine.
Schema Registry URL <code>schemaRegistryUrl</code>	string Not marked required by editor	Registry endpoint metadata. This engine does not implement registry-backed Avro conversion merely by setting the URL.
Key schema subject : version <code>keySchemaSubject</code>	string Not marked required by editor	Intended registry key subject/version; not resolved by the current engine serializer.
Value schema subject : version <code>valueSchemaSubject</code>	string Not marked required by editor	Intended registry value subject/version; not resolved by the current engine serializer.
Wait for broker confirmation <code>waitForDelivery</code>	boolean Not marked required by editor	False normally returns after local enqueue into a reused producer. True waits for delivery confirmation. Transactions and one-shot producers also force waiting.

Field / key	Type / required	Details
Acknowledgements acks	string Not marked required by editor	Broker acknowledgement policy: 0, 1 or all. This differs from the waitForDelivery switch; acks=0 cannot establish broker durability.
Compression type compressionType	string Not marked required by editor	Client batch compression. Select a codec supported by the installed client and broker; compression trades CPU for network bandwidth.
Producer retries retries	number Not marked required by editor	Native producer retry setting. Separate from MINA Advanced retries; application retries can repeat a send after an ambiguous timeout.
Buffer memory (bytes) bufferMemory	number Not marked required by editor	Local producer queue budget in bytes, translated to queue.buffering.max.kbytes. A full queue applies backpressure/errors; it is not durable storage.
Batch size batchSize	number Not marked required by editor	Target producer batch size in bytes, default 16384 in the engine. Larger batches may improve throughput at the expense of latency.
Linger (ms) lingerMs	number Not marked required by editor	Maximum batching delay requested from the producer, default 0 in the engine. Increasing it intentionally adds buffering delay.
Maximum request size maxRequestSize	number Not marked required by editor	Maximum native message size setting, default 1048576 bytes. Broker/topic limits can be lower.
Transactional ID transactionalId	string Not marked required by editor	Enables native transaction initialization/begin/commit for a send and forces confirmation. Must be unique for concurrently active producers; it is not a multi-activity transaction group.
Enable idempotence enableIdempotence	boolean Not marked required by editor	Requests native producer idempotence. Must be compatible with other native settings and does not deduplicate arbitrary repeated business calls.
Additional producer properties (JSON) additionalProperties	string Not marked required by editor	JSON object of native client configuration values merged into the producer/consumer configuration. Native keys differ from Java client keys; invalid keys are rejected by the installed client.

Output

Illustrative runtime output

```
{"messageId":"<id>","topic":"orders","queued":true,"deliveryConfirmed":true,"partition":0,"offset":42}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Generated record ID messageId	string Not marked required by editor	Identifier returned by the provider, or a locally generated identifier for buffered publishing. It is not proof that a downstream application processed the message.
Topic topic	string Not marked required by editor	Broker topic name. Resolve it from the active environment, for example \${properties.messaging.topic}. Consumer topic lists use semicolons.

Field / key	Type / required	Details
Accepted by the local Kafka producer queued	boolean Not marked required by editor	True indicates local producer acceptance, not broker durability or downstream consumption.
Broker delivery confirmed before completion deliveryConfirmed	boolean Not marked required by editor	True only when the delivery path has completed its confirmation before returning. Check this instead of inferring delivery from queued.
Publisher activity latency in milliseconds publishLatencyMs	number Not marked required by editor	Publisher activity latency; buffered mode measures enqueue/overhead, not end-to-end broker or consumer latency.
Broker partition (confirmation mode) partition	integer Not marked required by editor	Partition number on each returned message. The engine receive result nests it under messages[].
Broker offset (confirmation mode) offset	long Not marked required by editor	Kafka offset within one partition; it is not globally ordered across the topic.
Record timestamp timestamp	dateTime Not marked required by editor	Timestamp reported by the operation. Kafka native record timestamps are epoch milliseconds; do not assume every connector returns the editor-labelled dateTime representation.

Advanced

Delivery, transactions and performance

Separate local enqueue, broker acknowledgement and downstream processing. Monitor background publisher errors and shutdown draining. Use stable business IDs and idempotent consumers when retries are possible. Producer reuse reduces warm-send overhead, but first connection, TLS, metadata and broker availability still affect latency; no fixed latency is guaranteed.

Implemented behavior and limitations

With `waitForDelivery=false`, `partition/offset` can be absent. The generated ID may be local. Avro Schema selection currently raises a registry-runtime fault even when registry fields are populated. A `transactionalId` starts a transaction for the send; it does not span other activities.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Troubleshooting procedure

Connection/authentication failures: verify DNS, advertised listeners, TLS chain and ACLs on the runtime host. Serialization failures: compare actual payload type with the chosen serializer. Queue full/timeouts: inspect broker latency and producer buffer pressure before increasing limits. Consumer configuration errors may be caused by Java-style options passed to the native client. Capture the resolved non-secret configuration and the full runtime error.

Acceptance test

Acceptance: publish a known ID, verify the exact decoded value/key with an independent consumer, then test unavailable-broker behavior in both buffered and confirmed modes.

Fault	Description
KafkaConnectionException	The Kafka connection or broker is unavailable.
KafkaSerializationException	The record key or value could not be serialized/deserialized.
KafkaProducerException	The producer could not publish the record.
KafkaConsumerException	The consumer could not fetch or acknowledge records.
KafkaSchemaRegistryException	The Avro schema or registry subject could not be resolved.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs

Kafka Get Messages

Activities / Kafka

Overview

Poll records during a flow

Use after Start or another activity to perform a bounded receive attempt, rather than registering a new event starter.

Prerequisites

Install the matching confluent-kafka package on the execution host. Configure a Kafka shared connection with reachable bootstrap servers and the required TLS/SASL credentials. Verify advertised broker addresses are reachable from that host; a successful TCP connection to one bootstrap address is not sufficient. These pages describe the Studio/engine runtime; qualify exported direct Python separately.

Produces, receives, or commits Apache Kafka records.

Broker security comes from the shared Kafka connection. Topic, key, headers, value, partitions, offsets, and acknowledgement handles are available for hierarchical mapping.

Activity type: kafka; operation: get.

Configuration

Configuration procedure

Select the consumer resource/group/topics, maxMessages and poll timeout in seconds. For explicit assignment set assignCustomPartition, partitionIds and seekPosition together. Use customOffset only for custom seek.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Kafka connection resourceId	resource Required	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: kafka
Consumer group ID groupId	text Not marked required by editor	Consumer group used for assignment and stored offsets. Separate applications requiring independent consumption need separate groups.
Topic names topic	text Not marked required by editor	Broker topic name. Resolve it from the active environment, for example \${properties.messaging.topic}. Consumer topic lists use semicolons.
Assign custom partitions assignCustomPartition	boolean Not marked required by editor	Enable explicit partition selection; otherwise the client chooses/subscribes using its normal partitioning and assignment behavior.
Partition IDs / ranges partitionIds	text Not marked required by editor	Comma/semicolon-separated partition IDs or inclusive ranges used for explicit assignment.
Consumer count consumerCount	number Not marked required by editor	Exposed by the editor; the engine receive path creates one consumer per call and does not implement a consumer pool from this field.
Key deserializer keyDeserializer	select Not marked required by editor	Incoming key decoding; Byte Array exposes base64 text, JSON parses UTF-8 JSON, String decodes text. Choices: String, Byte Array, Avro Schema, JSON

Field / key	Type / required	Details
Value deserializer valueDeserializer	select Not marked required by editor	Incoming record-value decoding. Invalid JSON raises a decode error; Avro is not implemented by this engine path. Choices: String, Byte Array, Avro Schema, JSON
Use Schema Registry useRegistry	boolean Not marked required by editor	Editor registry flag; setting this does not activate an Avro serializer in the current engine.
Schema Registry URL schemaRegistryUrl	text Not marked required by editor	Registry endpoint metadata. This engine does not implement registry-backed Avro conversion merely by setting the URL.
Key schema subject : version keySchemaSubject	text Not marked required by editor	Intended registry key subject/version; not resolved by the current engine serializer.
Value schema subject : version valueSchemaSubject	text Not marked required by editor	Intended registry value subject/version; not resolved by the current engine serializer.
Acknowledgement mode acknowledgeMode	select Not marked required by editor	Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery. Choices: Auto, Manual
Enable auto commit enableAutoCommit	boolean Not marked required by editor	Native periodic offset commit toggle. For manual acknowledgement keep this disabled; otherwise offsets can advance before downstream work finishes.
Auto offset reset autoOffsetReset	select Not marked required by editor	Policy used when no valid committed offset exists, commonly earliest or latest. It does not rewind a group that already has committed offsets.
Fetch minimum bytes fetchMinBytes	number Not marked required by editor	Minimum fetch bytes requested from the broker. Larger values can add latency at low traffic.
Maximum poll records maxPollRecords	number Not marked required by editor	Editor batch-size setting. The engine currently passes max.poll.records to its native consumer; this Java-style key may be rejected by the installed native client. Treat this as a known compatibility issue, not a guaranteed supported control.
Session timeout (ms) sessionTimeoutMs	number Not marked required by editor	Consumer failure-detection timeout in milliseconds; must satisfy broker limits.
Heartbeat interval (ms) heartbeatIntervalMs	number Not marked required by editor	Consumer heartbeat interval in milliseconds; coordinate with session timeout.
Additional consumer properties (JSON) additionalProperties	textarea Not marked required by editor	JSON object of native client configuration values merged into the producer/consumer configuration. Native keys differ from Java client keys; invalid keys are rejected by the installed client.
Seek position seekPosition	select Not marked required by editor	Explicit-assignment start offset. The engine maps beginning to -2 and custom to the supplied offset; other selections currently use -1 (end). This is not a reliable resume-current implementation. Choices: current, beginning, end, custom
Custom offset customOffset	number Not marked required by editor	Absolute offset used with custom seek and explicit partition assignment.
Maximum messages maxMessages	number Not marked required by editor	Maximum poll attempts/records requested by one Get call. Each poll can wait for the configured timeout; total latency can grow with this count.

Field / key	Type / required	Details
Poll timeout (seconds) timeout	number Not marked required by editor	Unit depends on the activity: Kafka Get uses seconds; SAP editor fields use milliseconds. Consult the runtime notes because SAP worker calls use timeoutSeconds separately.

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
topic = ${properties.messaging.ordersTopic}
maxMessages = 10
timeout = 1
assignCustomPartition = true
partitionIds = 0
seekPosition = custom
customOffset = 42
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Dynamic topic override topic	string Not marked required by editor	Broker topic name. Resolve it from the active environment, for example \${properties.messaging.topic}. Consumer topic lists use semicolons.
Dynamic partition partition	integer Not marked required by editor	Partition number on each returned message. The engine receive result nests it under messages[].
Dynamic offset offset	long Not marked required by editor	Kafka offset within one partition; it is not globally ordered across the topic.
Consumer group ID groupid	string Not marked required by editor	Consumer group used for assignment and stored offsets. Separate applications requiring independent consumption need separate groups.
Assign custom partitions assignCustomPartition	boolean Not marked required by editor	Enable explicit partition selection; otherwise the client chooses/subscribes using its normal partitioning and assignment behavior.
Partition IDs / ranges partitionIds	string Not marked required by editor	Comma/semicolon-separated partition IDs or inclusive ranges used for explicit assignment.
Consumer count consumerCount	number Not marked required by editor	Exposed by the editor; the engine receive path creates one consumer per call and does not implement a consumer pool from this field.
Key deserializer keyDeserializer	string Not marked required by editor	Incoming key decoding; Byte Array exposes base64 text, JSON parses UTF-8 JSON, String decodes text.
Value deserializer valueDeserializer	string Not marked required by editor	Incoming record-value decoding. Invalid JSON raises a decode error; Avro is not implemented by this engine path.

Field / key	Type / required	Details
Use Schema Registry useRegistry	boolean Not marked required by editor	Editor registry flag; setting this does not activate an Avro serializer in the current engine.
Schema Registry URL schemaRegistryUrl	string Not marked required by editor	Registry endpoint metadata. This engine does not implement registry-backed Avro conversion merely by setting the URL.
Key schema subject : version keySchemaSubject	string Not marked required by editor	Intended registry key subject/version; not resolved by the current engine serializer.
Value schema subject : version valueSchemaSubject	string Not marked required by editor	Intended registry value subject/version; not resolved by the current engine serializer.
Acknowledgement mode acknowledgeMode	string Not marked required by editor	Choose automatic completion or explicit acknowledgement. Guarantees differ by connector and execution path; read Advanced before relying on rollback/redelivery.
Enable auto commit enableAutoCommit	boolean Not marked required by editor	Native periodic offset commit toggle. For manual acknowledgement keep this disabled; otherwise offsets can advance before downstream work finishes.
Auto offset reset autoOffsetReset	string Not marked required by editor	Policy used when no valid committed offset exists, commonly earliest or latest. It does not rewind a group that already has committed offsets.
Fetch minimum bytes fetchMinBytes	number Not marked required by editor	Minimum fetch bytes requested from the broker. Larger values can add latency at low traffic.
Maximum poll records maxPollRecords	number Not marked required by editor	Editor batch-size setting. The engine currently passes max.poll.records to its native consumer; this Java-style key may be rejected by the installed native client. Treat this as a known compatibility issue, not a guaranteed supported control.
Session timeout (ms) sessionTimeoutMs	number Not marked required by editor	Consumer failure-detection timeout in milliseconds; must satisfy broker limits.
Heartbeat interval (ms) heartbeatIntervalMs	number Not marked required by editor	Consumer heartbeat interval in milliseconds; coordinate with session timeout.
Additional consumer properties (JSON) additionalProperties	string Not marked required by editor	JSON object of native client configuration values merged into the producer/consumer configuration. Native keys differ from Java client keys; invalid keys are rejected by the installed client.
Seek position seekPosition	string Not marked required by editor	Explicit-assignment start offset. The engine maps beginning to -2 and custom to the supplied offset; other selections currently use -1 (end). This is not a reliable resume-current implementation.
Custom offset customOffset	number Not marked required by editor	Absolute offset used with custom seek and explicit partition assignment.
Maximum messages maxMessages	number Not marked required by editor	Maximum poll attempts/records requested by one Get call. Each poll can wait for the configured timeout; total latency can grow with this count.
Poll timeout (seconds) timeout	number Not marked required by editor	Unit depends on the activity: Kafka Get uses seconds; SAP editor fields use milliseconds. Consult the runtime notes because SAP worker calls use timeoutSeconds separately.

Output

Illustrative runtime output

```
{"messages": [], "count": 0, "ackId": null, "ackIds": []}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Kafka records messages	array Not marked required by editor	Array of received records with data, key, topic, partition, offset, timestamp, headers and optional ackId.
Record count count	integer Not marked required by editor	Number of records returned by this receive attempt. A zero count means no message was returned, not successful business processing.
Topic topic	string Not marked required by editor	Broker topic name. Resolve it from the active environment, for example <code>\$(properties.messaging.topic)</code> . Consumer topic lists use semicolons.
Partition partition	integer Not marked required by editor	Partition number on each returned message. The engine receive result nests it under <code>messages[]</code> .
Offset offset	long Not marked required by editor	Kafka offset within one partition; it is not globally ordered across the topic.
Record timestamp timestamp	dateTime Not marked required by editor	Timestamp reported by the operation. Kafka native record timestamps are epoch milliseconds; do not assume every connector returns the editor-labelled <code>dateTime</code> representation.
Record headers headers	object Not marked required by editor	Header name/value object. The messaging runtime stringifies outgoing values; this is not a guarantee of arbitrary native binary headers or JMS header setter support.
Manual acknowledgement handle ackId	string Not marked required by editor	Opaque, runtime-local confirmation token. Pass the original value to <code>Confirm Message</code> ; it is not the business message ID and cannot be reconstructed after restart.

Advanced

Delivery, transactions and performance

Separate local enqueue, broker acknowledgement and downstream processing. Monitor background publisher errors and shutdown draining. Use stable business IDs and idempotent consumers when retries are possible. Producer reuse reduces warm-send overhead, but first connection, TLS, metadata and broker availability still affect latency; no fixed latency is guaranteed.

Implemented behavior and limitations

Empty results are normal when no record is available. Each poll can consume the timeout, so the upper wait grows with `maxMessages`. The engine reads `partitionIds/customOffset`; the displayed dynamic `partition/offset` aliases are not equivalent overrides. Current seek maps to end in explicit assignment; do not use it as a resume guarantee. Consumer native-option and Avro limitations are the same as `Receive`.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Troubleshooting procedure

Connection/authentication failures: verify DNS, advertised listeners, TLS chain and ACLs on the runtime host. Serialization failures: compare actual payload type with the chosen serializer. Queue full/timeouts: inspect broker latency and producer buffer pressure before increasing limits. Consumer configuration errors may be caused by Java-style options passed to the native client. Capture the resolved non-secret configuration and the full runtime error.

Acceptance test

Acceptance: seed a test partition with known offsets; check beginning/custom/end behavior and the zero-message branch.

Fault	Description
KafkaConnectionException	The Kafka connection or broker is unavailable.
KafkaSerializationException	The record key or value could not be serialized/deserialized.
KafkaProducerException	The producer could not publish the record.
KafkaConsumerException	The consumer could not fetch or acknowledge records.
KafkaSchemaRegistryException	The Avro schema or registry subject could not be resolved.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py;
backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs

Pub/Sub Subscriber

Activities / GCP Pub/Sub

Overview

Publishes, receives, or acknowledges Google Cloud Pub/Sub messages.

Project and credential defaults come from the shared connection. Message data, attributes, ordering keys, and acknowledgement handles remain available on the execution path.

Activity type: pubsub; operation: subscribe.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Google Pub/Sub connection resourceId	resource Required	Connection type: pubsub
Project ID override projectId	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Subscription ID subscription	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Maximum message fetch count maxMessages	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Acknowledge mode acknowledgeMode	select Not marked required by editor	Choices: Auto, Client
Receive timeout (seconds) receiveTimeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Sequence key expression sequenceKey	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Custom job ID expression customJobId	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Project ID override projectId	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Subscription ID subscription	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Maximum message fetch count maxMessages	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Acknowledge mode acknowledgeMode	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Receive timeout (seconds) receiveTimeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Sequence key expression sequenceKey	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Custom job ID expression customJobId	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Google message ID MessageID	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Publish time PublishTime	dateTime Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Message data Data	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Message attributes Attributes	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Client acknowledgement handle AckID	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
GooglePubSubPluginException	The subscriber could not receive or acknowledge the message.
GooglePubSubConnectionException	The Google Pub/Sub connection is unavailable.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

Pub/Sub Publisher

Activities / GCP Pub/Sub

Overview

Publishes, receives, or acknowledges Google Cloud Pub/Sub messages.

Project and credential defaults come from the shared connection. Message data, attributes, ordering keys, and acknowledgement handles remain available on the execution path.

Activity type: pubsub; operation: publish.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Google Pub/Sub connection resourceId	resource Required	Connection type: pubsub
Project ID override projectId	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Topic ID topic	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Ordering key orderingKey	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Wait for Google confirmation waitForDelivery	boolean Not marked required by editor	Off queues through the persistent batched gRPC publisher for maximum throughput. Enable only when the flow requires Google's message ID.
Publish timeout (seconds) publishTimeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Batch maximum messages batchMaxMessages	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Batch maximum bytes batchMaxBytes	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Batch delay threshold (ms) batchDelayThresholdMilliseconds	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Message data data	string binary Required	Data or at least one attribute is required.
Message attributes attributes	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic Topic ID topic	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Project ID override projectId	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Ordering key orderingKey	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Wait for Google confirmation waitForDelivery	boolean Not marked required by editor	Off queues through the persistent batched gRPC publisher for maximum throughput. Enable only when the flow requires Google's message ID.
Publish timeout (seconds) publishTimeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Batch maximum messages batchMaxMessages	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Batch maximum bytes batchMaxBytes	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Batch delay threshold (ms) batchDelayThresholdMilliseconds	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Published topic name TopicName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Google or local queued message ID MessageID	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Google message ID when confirmation is enabled providerMessageId	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Accepted by local publisher queued	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Confirmed by Google deliveryConfirmed	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Activity enqueue latency (ms) publishLatencyMs	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
GooglePublisherException	The message could not be published.
GooglePubSubConnectionException	The Google Pub/Sub connection is unavailable.
PublisherInvalidInputException	Publisher requires message data or at least one attribute.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

Dynamic Connection

Activities / SAP ECC

Overview

Open or close a stateful SAP session

Use when multiple RFC/BAPI activities must share one JCo context and an explicit commit/rollback boundary.

Prerequisites

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: dynamic_connection.

Configuration

Configuration procedure

Select the SAP resource, connection mode and transactional setting. Bind secrets through properties. On the success path call Dynamic Connection again with terminateConnection=true and commitOnTerminate=true; on the error path terminate without commit.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
SAP ECC shared connection resourceId	resource Not marked required by editor	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: sap
Connection type connectionType	select Not marked required by editor	Connection routing mode: direct application server, message-server logon group, SNC variants or WebSocket configuration. Supply the corresponding shared-connection fields. Choices: dedicated, logongroup, snc, sncwithlogongroup, websocket
Transactional transactional	boolean Not marked required by editor	Whether a stateful transactional context is requested. Only operations sharing the returned live sessionId participate in that context.
Terminate connection terminateConnection	boolean Not marked required by editor	When true, closes the session identified by sessionId instead of creating a new session. Terminating a transactional session rolls back unless commitOnTerminate is enabled.
Commit before termination commitOnTerminate	boolean Not marked required by editor	Commit the live transactional session before closing it. Enable only on the success path; use rollback termination on the error path. Conditional field: (config) => !!config.terminateConnection
Session idle timeout (ms) contextTimeout	number Not marked required by editor	Idle lifetime of a dynamic SAP session in milliseconds. Default is 600000, clamped to at least 1000; later calls renew the session lifetime.

Field / key	Type / required	Details
Timeout (ms) timeout	number Not marked required by editor	Unit depends on the activity: Kafka Get uses seconds; SAP editor fields use milliseconds. Consult the runtime notes because SAP worker calls use timeoutSeconds separately.

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
Open: transactional = true
Invoke RFC Input sessionID = ${SAP-Open.sessionID}
Close Input sessionID = ${SAP-Open.sessionID}
Close: terminateConnection = true
Close success only: commitOnTerminate = true
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
Session ID (required only when terminating) sessionID	string Not marked required by editor	Map the sessionID returned by Dynamic Connection. It identifies a live, process-local JCo context; do not hard-code or reuse it after termination or restart.
Transaction context timeout in milliseconds contextTimeout	integer Not marked required by editor	Idle lifetime of a dynamic SAP session in milliseconds. Default is 600000, clamped to at least 1000; later calls renew the session lifetime.
dedicated, logongroup, snc, sncwithlogongroup, or websocket connectionType	string Not marked required by editor	Connection routing mode: direct application server, message-server logon group, SNC variants or WebSocket configuration. Supply the corresponding shared-connection fields.
Application server host applicationServerHost	string Not marked required by editor	Application-server hostname for direct connection mode; must resolve from the runtime host.
System number systemNumber	string Not marked required by editor	SAP instance/system number for a direct application-server connection, normally a two-character string.
Message server host messageServerHost	string Not marked required by editor	Message-server hostname for logon-group mode.
SAP system ID systemId	string Not marked required by editor	SAP system ID used with message-server/logon-group routing.
SAP logon group logonGroup	string Not marked required by editor	SAP logon-group name configured by the SAP administrator.
SAP client client	string Not marked required by editor	SAP client identifier. Keep leading zeros; provide it as a string.

Field / key	Type / required	Details
SAP user username	string Not marked required by editor	SAP logon user with authorization for the selected RFCs and metadata APIs.
SAP password password	string Not marked required by editor	SAP logon secret. Bind a secret/environment property; do not put credentials into examples, source control or payload logs.
Logon language language	string Not marked required by editor	SAP logon language key.
SAProuter string sapRouter	string Not marked required by editor	Optional SAProuter routing string supplied by the network/SAP administrator.
SNC partner name sncPartnerName	string Not marked required by editor	SNC partner identity expected by the target SAP system.
SNC library path sncLibraryPath	string Not marked required by editor	Native SNC library path on the runtime host, matching OS and architecture.
SNC own name sncMyName	string Not marked required by editor	Optional client SNC identity.
SNC quality of protection sncQop	string Not marked required by editor	SNC quality-of-protection setting accepted by the target system.
Transactional transactional	boolean Not marked required by editor	Whether a stateful transactional context is requested. Only operations sharing the returned live sessionID participate in that context.
Terminate connection terminateConnection	boolean Not marked required by editor	When true, closes the session identified by sessionID instead of creating a new session. Terminating a transactional session rolls back unless commitOnTerminate is enabled.
Commit before termination commitOnTerminate	boolean Not marked required by editor	Commit the live transactional session before closing it. Enable only on the success path; use rollback termination on the error path.
Timeout (ms) timeout	number Not marked required by editor	Unit depends on the activity: Kafka Get uses seconds; SAP editor fields use milliseconds. Consult the runtime notes because SAP worker calls use timeoutSeconds separately.

Output

Illustrative runtime output

```
{"sessionID":"<live-session>","connected":true,"transactional":true}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
Dynamic SAP session ID sessionID	string Not marked required by	Map the sessionID returned by Dynamic Connection. It identifies a live, process-local JCo context; do not hard-code or reuse it after

Field / key	Type / required	Details
	editor	termination or restart.
Connection status connected	boolean Not marked required by editor	True when Dynamic Connection successfully opens and pings its JCo worker; it does not guarantee later calls will succeed.
Transaction mode transactional	boolean Not marked required by editor	Whether a stateful transactional context is requested. Only operations sharing the returned live sessionID participate in that context.

Advanced

Delivery, transactions and performance

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

Implemented behavior and limitations

Close returns terminated/committed rather than connected. Sessions are process-local and expire after idle timeout; default is 600000 ms. A failed transactional worker call attempts rollback and removes its session. Ensure every branch closes its session. Do not confuse this with a JDBC transaction group or distributed transaction.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Troubleshooting procedure

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

Acceptance test

Acceptance: perform two reversible test changes in one session, first rollback and then commit; verify both outcomes directly in SAP.

Fault	Description
SAP_CONNECTION	The SAP JCo/RFC connection could not be established.
SAP_RFC	SAP returned an RFC or BAPI exception.
SAP_IDOC	The IDoc could not be parsed, posted, confirmed, or acknowledged.
SAP_TRANSACTION	The tRFC, qRFC, bgRFC, or transactional context failed.
SAP_SCHEMA	The downloaded RFC, table, or IDoc metadata is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs

IDoc Acknowledgment

Activities / SAP ECC

Overview

Call an SAP acknowledgement function

Use only when the SAP team has defined an acknowledgement RFC and its required parameters.

Prerequisites

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: idoc_acknowledgment.

Configuration

Configuration procedure

Select the SAP resource and status intent. External mode requires functionName, but this activity editor does not currently expose that field. It must be present in resolved configuration/shared settings; otherwise execution fails.

Status/successMessage/errorMessage alone do not implement an SAP acknowledgement protocol.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
SAP ECC shared connection resourceId	resource Not marked required by editor	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: sap
SAP IDoc status status	text Not marked required by editor	Desired acknowledgement status metadata. Actual SAP status changes depend on the configured acknowledgement RFC and its parameters.
Success message successMessage	text Not marked required by editor	Editor text for acknowledgement intent; not automatically translated into an SAP RFC parameter by the native adapter.
Error message errorMessage	text Not marked required by editor	Editor error text; explicitly map the required acknowledgement RFC arguments rather than assuming this field changes SAP status.

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
Resolved functionName = <approved acknowledgement RFC>
payload = ${input.acknowledgementArguments}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
SAP activity payload payload	object Required	Map the whole input document expected by this operation. Use the Data tree to select an earlier output; the examples below describe its required shape.
Dynamic connection session ID sessionID	string Not marked required by editor	Map the sessionID returned by Dynamic Connection. It identifies a live, process-local JCo context; do not hard-code or reuse it after termination or restart.
SAP IDoc status status	string Not marked required by editor	Desired acknowledgement status metadata. Actual SAP status changes depend on the configured acknowledgement RFC and its parameters.
Success message successMessage	string Not marked required by editor	Editor text for acknowledgement intent; not automatically translated into an SAP RFC parameter by the native adapter.
Error message errorMessage	string Not marked required by editor	Editor error text; explicitly map the required acknowledgement RFC arguments rather than assuming this field changes SAP status.

Output

Illustrative runtime output

```
{"acknowledged":true,"status":"<configured-status>","idocNumber":"<supplied-number>"}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
SAP response result	object Not marked required by editor	Generic editor result placeholder. Native SAP calls return exports, changing parameters, tables and operation-specific metadata; inspect actual Job Data before mapping result.
Completion status successful	boolean Not marked required by editor	Generic editor completion indicator. See the runtime output example: several native SAP operations return operation-specific fields instead of this generic field.

Advanced

Delivery, transactions and performance

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

Implemented behavior and limitations

The runtime invokes the configured RFC with mapped arguments and adds acknowledgement metadata. Check the RFC response before treating business acknowledgement as accepted. This is distinct from broker Confirm Message and JCo listener transport acknowledgement.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Troubleshooting procedure

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

Acceptance test

Acceptance: verify the expected SAP status transition for a test IDoc and a deliberately rejected acknowledgement.

Fault	Description
SAP_CONNECTION	The SAP JCo/RFC connection could not be established.
SAP_RFC	SAP returned an RFC or BAPI exception.
SAP_IDOC	The IDoc could not be parsed, posted, confirmed, or acknowledged.
SAP_TRANSACTION	The tRFC, qRFC, bgRFC, or transactional context failed.
SAP_SCHEMA	The downloaded RFC, table, or IDoc metadata is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs

IDoc Confirmation

Activities / SAP ECC

Overview

Confirm through a configured SAP RFC

Use for the confirmation contract explicitly provided by the target SAP system.

Prerequisites

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: idoc_confirmation.

Configuration

Configuration procedure

Set functionName to the approved confirmation RFC and map its actual parameters. Select the SAP connection. confirmationMode and destination are descriptive/editor settings, not a generic implementation of every confirmation protocol.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
SAP ECC shared connection resourceId	resource Not marked required by editor	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: sap
Confirmation mode confirmationMode	select Not marked required by editor	Editor setting; the current native adapter does not implement a separate generic confirmation-mode switch. Read the operation-specific limitations below.
Confirmation destination destination	text Not marked required by editor	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
Confirmation RFC functionName	text Not marked required by editor	Exact remote-enabled function/BAPI name available in the target SAP repository. Its case and parameter names must match fetched metadata.

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
functionName = <approved confirmation RFC>
payload = ${input.confirmationArguments}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
SAP activity payload payload	object Required	Map the whole input document expected by this operation. Use the Data tree to select an earlier output; the examples below describe its required shape.
Dynamic connection session ID sessionID	string Not marked required by editor	Map the sessionID returned by Dynamic Connection. It identifies a live, process-local JCo context; do not hard-code or reuse it after termination or restart.
Confirmation mode confirmationMode	string Not marked required by editor	Editor setting; the current native adapter does not implement a separate generic confirmation-mode switch. Read the operation-specific limitations below.
Confirmation destination destination	string Not marked required by editor	Resolved queue/topic name. A dynamic destination takes precedence over the configured queue or topic. Always set it explicitly; an omitted destination can fall back to "default".
Confirmation RFC functionName	string Not marked required by editor	Exact remote-enabled function/BAPI name available in the target SAP repository. Its case and parameter names must match fetched metadata.

Output

Illustrative runtime output

```
{"acknowledged":true,"status":"<configured-status>","idocNumber":"<supplied-number>"}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
SAP response result	object Not marked required by editor	Generic editor result placeholder. Native SAP calls return exports, changing parameters, tables and operation-specific metadata; inspect actual Job Data before mapping result.
Completion status successful	boolean Not marked required by editor	Generic editor completion indicator. See the runtime output example: several native SAP operations return operation-specific fields instead of this generic field.

Advanced

Delivery, transactions and performance

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

Implemented behavior and limitations

No universal confirmation RFC name or parameter shape is assumed. The runtime requires functionName in external mode; a mock acknowledged result is not proof of a real SAP confirmation.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Troubleshooting procedure

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

Acceptance test

Acceptance: inspect the configured RFC return and SAP document/transaction state, including duplicate confirmation behavior.

Fault	Description
SAP_CONNECTION	The SAP JCo/RFC connection could not be established.
SAP_RFC	SAP returned an RFC or BAPI exception.
SAP_IDOC	The IDoc could not be parsed, posted, confirmed, or acknowledged.
SAP_TRANSACTION	The tRFC, qRFC, bgRFC, or transactional context failed.
SAP_SCHEMA	The downloaded RFC, table, or IDoc metadata is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs

IDoc Converter

Activities / SAP ECC

Overview

Convert an IDoc representation

Use inside a task to transform supported IDoc source content into the selected representation without posting it.

Prerequisites

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: idoc_converter.

Configuration

Configuration procedure

Select the saved IDoc basic type/extension metadata. Map the full IDoc source. XML output returns serialized XML; Raw returns the original source. Use IDoc Renderer when transport-ready control/data records are required.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
SAP ECC shared connection resourceId	resource Not marked required by editor	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: sap
IDoc type fetched from SAP idocType	idoc Not marked required by editor	Select and save the basic IDoc type fetched from the chosen SAP connection. The saved catalog/schema drives segment layout, field offsets and mapping suggestions.
Extension / CIM type extensionType	text Not marked required by editor	Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata.
SAP release release	text Not marked required by editor	SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string.
IDoc output mode idocOutputMode	select Not marked required by editor	Parser: JSON or XML. Converter also exposes Raw; RAW returns the original source rather than manufacturing transport-ready EDI records. Choices: XML, Raw

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
IDoc = ${SAP-Listener.payload}
SAPIDoc = ${SAP-Listener.controlRecord}
idocOutputMode = XML
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
SAP IDoc control record SAPIDoc	object Required	For parser input, control metadata or a structured IDoc. For parser output, the parsed document; selected basic-type and legacy direct branches may both be present.
Raw IDoc IDoc	string object Required	Raw IDoc input: XML text, supported structured control/data records or flat IDoc bytes/text. Do not map only one segment when the operation needs the complete document.
Extension / CIM type extensionType	string Not marked required by editor	Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata.
SAP release release	string Not marked required by editor	SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string.
IDoc output mode idocOutputMode	string Not marked required by editor	Parser: JSON or XML. Converter also exposes Raw; RAW returns the original source rather than manufacturing transport-ready EDI records.

Output

Illustrative runtime output

```
{"format": "XML", "IDocXML": "<IDoc>...</IDoc>", "payload": "<IDoc>...</IDoc>", "SAPIDoc": {}}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
Parsed IDoc JSON SAPIDoc	object Not marked required by editor	For parser input, control metadata or a structured IDoc. For parser output, the parsed document; selected basic-type and legacy direct branches may both be present.
Output format format	string Not marked required by editor	Output representation selected by the parser/converter: JSON, XML or RAW. Map the accompanying content field appropriate for that format.

Advanced

Delivery, transactions and performance

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound

workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

Implemented behavior and limitations

The converter shares the parser implementation. Its generic editor output can differ from the selected mode; XML also includes structured views, RAW preserves the supplied value. This activity is not an outbound SAP call.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Troubleshooting procedure

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

Acceptance test

Acceptance: convert a small fetched-type fixture, then parse it back and compare control fields, segment values and array cardinality.

Fault	Description
SAP_CONNECTION	The SAP JCo/RFC connection could not be established.
SAP_RFC	SAP returned an RFC or BAPI exception.
SAP_IDOC	The IDoc could not be parsed, posted, confirmed, or acknowledged.
SAP_TRANSACTION	The tRFC, qRFC, bgRFC, or transactional context failed.
SAP_SCHEMA	The downloaded RFC, table, or IDoc metadata is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs

IDoc Listener

Activities / SAP ECC

Overview

Start a flow when an IDoc arrives

Use as an event starter. Direct mode registers a JCo inbound server; broker mode consumes an already-published IDoc.

Prerequisites

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: idoc_listener.

Configuration

Configuration procedure

Configure SAP program ID/gateway and partner/RFC routing, fetch and save IDoc metadata, then run the task. NoMessaging uses direct JCo. For EMS/JMS/Kafka select the additional broker resource and destination; MINA does not automatically create the upstream SAP-to-broker bridge.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
SAP ECC shared connection resourceId	resource Not marked required by editor	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: sap
Messaging source messagingSource	select Not marked required by editor	NoMessaging receives directly through JCo. EMS/JMS/Kafka modes consume an IDoc carried by a broker; an upstream SAP-to-broker producer is still required. Choices: NoMessaging, EMS, JMS, Kafka
Messaging shared connection messagingResourceId	resource Not marked required by editor	Shared broker connection matching the chosen messaging source, in addition to the SAP metadata connection. Conditional field: (config) => !["", "NoMessaging"].includes(config.messagingSource "NoMessaging") Connection types:
Queue or topic messagingDestination	text Not marked required by editor	Queue or topic carrying the complete IDoc document. Conditional field: (config) => !["", "NoMessaging"].includes(config.messagingSource "NoMessaging")
SAP TID Manager resource tidManagerId	text Not marked required by editor	Shared SAP transaction-ID store selection. Durable TID state supports duplicate detection; it does not make downstream external writes atomic. Read-only

Field / key	Type / required	Details
IDoc type fetched from SAP idocType	idoc Not marked required by editor	Select and save the basic IDoc type fetched from the chosen SAP connection. The saved catalog/schema drives segment layout, field offsets and mapping suggestions.
Extension / CIM type extensionType	text Not marked required by editor	Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata. Read-only
SAP release release	text Not marked required by editor	SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string. Read-only
Program ID programId	text Not marked required by editor	Registered-server program ID. Must match the SAP RFC destination and gateway registration permissions. Read-only
Invocation protocol invocationProtocol	text Not marked required by editor	Editor protocol selection. Native bridge implements synchronous RFC and tRFC/qRFC dispatch; bgRFC is not a separately implemented native execution mode. Read-only

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
messagingSource = NoMessaging
IDoc Parser IDoc = ${SAP-Listener.payload}
IDoc Parser SAPIDoc = ${SAP-Listener.controlRecord}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
Messaging source messagingSource	string Not marked required by editor	NoMessaging receives directly through JCo. EMS/JMS/Kafka modes consume an IDoc carried by a broker; an upstream SAP-to-broker producer is still required.
Queue or topic messagingDestination	string Not marked required by editor	Queue or topic carrying the complete IDoc document.
SAP TID Manager resource tidManagerId	string Not marked required by editor	Shared SAP transaction-ID store selection. Durable TID state supports duplicate detection; it does not make downstream external writes atomic.
Extension / CIM type extensionType	string Not marked required by editor	Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata.
SAP release release	string Not marked required by editor	SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string.
Program ID programId	string Not marked required by editor	Registered-server program ID. Must match the SAP RFC destination and gateway registration permissions.

Field / key	Type / required	Details
Invocation protocol invocationProtocol	string Not marked required by editor	Editor protocol selection. Native bridge implements synchronous RFC and tRFC/qRFC dispatch; bgRFC is not a separately implemented native execution mode.

Output

Illustrative runtime output

```
{ "SAPIDoc": {}, "controlRecord": { "IDOC TYP": "<basic-type>", "payload": "<IDoc>...</IDoc>", "TID": "<transaction>" }
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
IDoc/RFC metadata SAPIDoc	object Not marked required by editor	For parser input, control metadata or a structured IDoc. For parser output, the parsed document; selected basic-type and legacy direct branches may both be present.
SAP IDoc control record controlRecord	object Not marked required by editor	EDI_DC40 control metadata describing sender, receiver, message/basic type and extension. Keep it with the associated data records.
Inbound SAP payload (XML) payload	string Not marked required by editor	Map the whole input document expected by this operation. Use the Data tree to select an earlier output; the examples below describe its required shape.
Transaction ID TID	string Not marked required by editor	SAP transaction identifier when provided by the transport. It is not the IDoc business document number and is not a JMS acknowledgement token.

Advanced

Delivery, transactions and performance

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

Implemented behavior and limitations

Direct delivery uses durable TID handling and listener completion; broker modes inherit the broker path acknowledgement limitations. Metadata arrays follow the saved schema. Do not acknowledge transport success before irreversible downstream work unless your recovery design tolerates it. Debug breakpoints can hold requests until the listener timeout.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Troubleshooting procedure

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

Acceptance test

Acceptance: send a test IDoc while Run is active, confirm automatic execution without Step Into, then test duplicate TID, downstream error, stop/restart and redelivery.

Fault	Description
SAP_CONNECTION	The SAP JCo/RFC connection could not be established.
SAP_RFC	SAP returned an RFC or BAPI exception.
SAP_IDOC	The IDoc could not be parsed, posted, confirmed, or acknowledged.
SAP_TRANSACTION	The tRFC, qRFC, bgRFC, or transactional context failed.
SAP_SCHEMA	The downloaded RFC, table, or IDoc metadata is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs

IDoc Parser

Activities / SAP ECC

Overview

Decode an IDoc into mapped fields

Use after a listener, file read or broker receive to expose control and segment fields in JSON/XML.

Prerequisites

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: idoc_parser.

Configuration

Configuration procedure

Fetch/save the matching basic type, extension and release. Choose JSON for field mappings or XML for serialized output. For flat data set the actual encoding/framing; keep strict invalid-byte handling unless replacement is intentional.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
SAP ECC shared connection resourceId	resource Not marked required by editor	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: sap
IDoc source destination sourceDestination	text Not marked required by editor	Editor metadata describing the IDoc source. The native parser/reader does not use this value as a separate network destination; select the actual shared resource.
IDoc type fetched from SAP idocType	idoc Not marked required by editor	Select and save the basic IDoc type fetched from the chosen SAP connection. The saved catalog/schema drives segment layout, field offsets and mapping suggestions.
Extension / CIM type extensionType	text Not marked required by editor	Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata.
SAP release release	text Not marked required by editor	SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string.
Flat IDoc encoding idocEncoding	text Not marked required by editor	Character set used to decode flat records and SDATA byte offsets, for example UTF-8 or windows-1252. Must match the actual sender encoding.
Flat record delimiter idocRecordDelimiter	text Not marked required by editor	Explicit escaped record separator for flat input, such as \r\n. Use fixed-width/custom framing when segment data itself contains line breaks.

Field / key	Type / required	Details
Invalid byte policy invalidCharacterPolicy	select Not marked required by editor	strict rejects invalid byte sequences; replace substitutes invalid characters and can lose data. Use strict when data fidelity is required. Choices: strict, replace
Parser engine parserEngine	select Not marked required by editor	Built-in parses XML, structured and supported flat records. The optional licensed IDoc library path validates XML; it does not implement flat-byte parsing. Choices: Built-in, SAP JIDocLib (when installed)
Reject mismatched IDoc type or extension validateIdocType	boolean Not marked required by editor	Checks available control-record basic type and configured extension against saved metadata. Missing metadata is not proof the payload was validated.
IDoc output format idocOutputMode	select Not marked required by editor	Parser: JSON or XML. Converter also exposes Raw; RAW returns the original source rather than manufacturing transport-ready EDI records. Choices: JSON, XML

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
IDoc = ${SAP-Listener.payload}
SAPIDoc = ${SAP-Listener.controlRecord}
idocOutputMode = JSON
Next message = ${SAP-Parser.SAPIDoc}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
SAP IDoc control record SAPIDoc	object Required	For parser input, control metadata or a structured IDoc. For parser output, the parsed document; selected basic-type and legacy direct branches may both be present.
Raw IDoc IDoc	string/object Required	Raw IDoc input: XML text, supported structured control/data records or flat IDoc bytes/text. Do not map only one segment when the operation needs the complete document.
IDoc source destination sourceDestination	string Not marked required by editor	Editor metadata describing the IDoc source. The native parser/reader does not use this value as a separate network destination; select the actual shared resource.
Extension / CIM type extensionType	string Not marked required by editor	Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata.
SAP release release	string Not marked required by editor	SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string.
Flat IDoc encoding idocEncoding	string Not marked required by editor	Character set used to decode flat records and SDATA byte offsets, for example UTF-8 or windows-1252. Must match the actual sender encoding.

Field / key	Type / required	Details
Flat record delimiter idocRecordDelimiter	string Not marked required by editor	Explicit escaped record separator for flat input, such as <code>\r\n</code> . Use fixed-width/custom framing when segment data itself contains line breaks.
Invalid byte policy invalidCharacterPolicy	string Not marked required by editor	strict rejects invalid byte sequences; replace substitutes invalid characters and can lose data. Use strict when data fidelity is required.
Parser engine parserEngine	string Not marked required by editor	Built-in parses XML, structured and supported flat records. The optional licensed IDoc library path validates XML; it does not implement flat-byte parsing.
Reject mismatched IDoc type or extension validateIdocType	boolean Not marked required by editor	Checks available control-record basic type and configured extension against saved metadata. Missing metadata is not proof the payload was validated.
IDoc output format idocOutputMode	string Not marked required by editor	Parser: JSON or XML. Converter also exposes Raw; RAW returns the original source rather than manufacturing transport-ready EDI records.

Output

Illustrative runtime output

```
{
  "format": "JSON",
  "idocType": "<basic-type>",
  "SAPIDoc": {
    "IDOC": {
      "EDI_DC40": {},
      "<repeating-segment>": [
        {
          "<field>": "value"
        }
      ]
    }
  }
}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
Parsed IDoc JSON SAPIDoc	object Not marked required by editor	For parser input, control metadata or a structured IDoc. For parser output, the parsed document; selected basic-type and legacy direct branches may both be present.
Output format format	string Not marked required by editor	Output representation selected by the parser/converter: JSON, XML or RAW. Map the accompanying content field appropriate for that format.

Advanced

Delivery, transactions and performance

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

Implemented behavior and limitations

Illustrative shape only: actual paths come from your fetched schema. Repeating segments remain arrays even for one occurrence; optional absent fields are not miniated. The runtime also exposes the basic-type-named branch. The licensed IDoc-library option validates XML only; flat records require Built-in. Missing metadata can prevent SDATA expansion, and parser memory grows with document size.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Troubleshooting procedure

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

Acceptance test

Acceptance: test one, multiple and absent repeating segments; malformed XML; mismatched type/extension; non-UTF-8 bytes; and segment data containing line breaks.

Fault	Description
SAP_CONNECTION	The SAP JCo/RFC connection could not be established.
SAP_RFC	SAP returned an RFC or BAPI exception.
SAP_IDOC	The IDoc could not be parsed, posted, confirmed, or acknowledged.
SAP_TRANSACTION	The tRFC, qRFC, bgRFC, or transactional context failed.
SAP_SCHEMA	The downloaded RFC, table, or IDoc metadata is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs

IDoc Reader

Activities / SAP ECC

Overview

Dispatch an IDoc using queued RFC

Despite its label, the current runtime converts input to control/data tables and calls the inbound-IDoc RFC. It does not fetch an existing IDoc by document number.

Prerequisites

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: idoc_reader.

Configuration

Configuration procedure

Select metadata and SAP connection, provide a complete input IDoc, and configure queueName in resolved settings for qRFC ordering. Reader defaults to qRFC; the editor does not expose every dispatch setting.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
SAP ECC shared connection resourceId	resource Not marked required by editor	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: sap
IDoc source destination sourceDestination	text Not marked required by editor	Editor metadata describing the IDoc source. The native parser/reader does not use this value as a separate network destination; select the actual shared resource.
IDoc type fetched from SAP idocType	idoc Not marked required by editor	Select and save the basic IDoc type fetched from the chosen SAP connection. The saved catalog/schema drives segment layout, field offsets and mapping suggestions.
Extension / CIM type extensionType	text Not marked required by editor	Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata.
SAP release release	text Not marked required by editor	SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string.
Confirmation mode confirmationMode	text Not marked required by editor	Editor setting; the current native adapter does not implement a separate generic confirmation-mode switch. Read the operation-specific limitations below.

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
payload = ${SAP-Renderer.rawIDoc}  
Resolved queueName = ${properties.sap.outboundQueue}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
SAP activity payload payload	object Required	Map the whole input document expected by this operation. Use the Data tree to select an earlier output; the examples below describe its required shape.
Dynamic connection session ID sessionID	string Not marked required by editor	Map the sessionID returned by Dynamic Connection. It identifies a live, process-local JCo context; do not hard-code or reuse it after termination or restart.
IDoc source destination sourceDestination	string Not marked required by editor	Editor metadata describing the IDoc source. The native parser/reader does not use this value as a separate network destination; select the actual shared resource.
Extension / CIM type extensionType	string Not marked required by editor	Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata.
SAP release release	string Not marked required by editor	SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string.
Confirmation mode confirmationMode	string Not marked required by editor	Editor setting; the current native adapter does not implement a separate generic confirmation-mode switch. Read the operation-specific limitations below.

Output

Illustrative runtime output

```
{"TID":"<transaction>","transactional":true,"transactionProtocol":"qrfc","confirmed":true}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
SAP response result	object Not marked required by editor	Generic editor result placeholder. Native SAP calls return exports, changing parameters, tables and operation-specific metadata; inspect actual Job Data before mapping result.
Completion status successful	boolean Not marked required by editor	Generic editor completion indicator. See the runtime output example: several native SAP operations return operation-specific fields instead of this generic field.

Advanced

Delivery, transactions and performance

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

Implemented behavior and limitations

Without queueName the bridge falls back to a transactional execution overload without the intended queue name. Treat this as a dispatch activity, not historical retrieval. Both EDI_DC40 control data and at least one EDI_DD40 segment are required.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Troubleshooting procedure

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

Acceptance test

Acceptance: verify the queued outbound dispatch and ordering in SAP; do not test it against production expecting a read-only operation.

Fault	Description
SAP_CONNECTION	The SAP JCo/RFC connection could not be established.
SAP_RFC	SAP returned an RFC or BAPI exception.
SAP_IDOC	The IDoc could not be parsed, posted, confirmed, or acknowledged.
SAP_TRANSACTION	The tRFC, qRFC, bgRFC, or transactional context failed.
SAP_SCHEMA	The downloaded RFC, table, or IDoc metadata is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs

Post IDoc to SAP

Activities / SAP ECC

Overview

Post a business IDoc to SAP

Use inside a flow to dispatch a complete IDoc using the selected SAP metadata and transport protocol.

Prerequisites

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: post_idoc.

Configuration

Configuration procedure

Select SAP resource and saved basic type/extension. Map XML, structured IDoc or rendered control/data envelope. Select tRFC (default) or qRFC and supply queueName for ordered dispatch.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
SAP ECC shared connection resourceId	resource Not marked required by editor	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: sap
IDoc type fetched from SAP idocType	idoc Not marked required by editor	Select and save the basic IDoc type fetched from the chosen SAP connection. The saved catalog/schema drives segment layout, field offsets and mapping suggestions.
Extension / CIM type extensionType	text Not marked required by editor	Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata.
SAP release release	text Not marked required by editor	SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string.
IDoc delivery protocol idocInputMode	select Not marked required by editor	IDoc delivery protocol. Post defaults to tRFC; Reader defaults to qRFC. qRFC requires a real queue name for ordered dispatch. Choices: tRFC, qRFC
IDoc payload format inputFormat	select Not marked required by editor	Describes the intended XML or raw input for posting; the native adapter converts the actual input using saved metadata. Choices: XML, Raw
Confirmation mode confirmationMode	text Not marked required by editor	Editor setting; the current native adapter does not implement a separate generic confirmation-mode switch. Read the operation-specific limitations below.

Field / key	Type / required	Details
SAP queue name queueName	text Not marked required by editor	SAP outbound qRFC queue name. Without a queue name the bridge cannot establish the intended ordered queue execution.

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
payload = ${SAP-Renderer.rawIDoc}
idocInputMode = tRFC
For qRFC: queueName = ${properties.sap.outboundQueue}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
SAP activity payload payload	object Required	Map the whole input document expected by this operation. Use the Data tree to select an earlier output; the examples below describe its required shape.
Dynamic connection session ID sessionID	string Not marked required by editor	Map the sessionID returned by Dynamic Connection. It identifies a live, process-local JCo context; do not hard-code or reuse it after termination or restart.
Extension / CIM type extensionType	string Not marked required by editor	Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata.
SAP release release	string Not marked required by editor	SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string.
IDoc delivery protocol idocInputMode	string Not marked required by editor	IDoc delivery protocol. Post defaults to tRFC; Reader defaults to qRFC. qRFC requires a real queue name for ordered dispatch.
IDoc payload format inputFormat	string Not marked required by editor	Describes the intended XML or raw input for posting; the native adapter converts the actual input using saved metadata.
Confirmation mode confirmationMode	string Not marked required by editor	Editor setting; the current native adapter does not implement a separate generic confirmation-mode switch. Read the operation-specific limitations below.
SAP queue name queueName	string Not marked required by editor	SAP outbound qRFC queue name. Without a queue name the bridge cannot establish the intended ordered queue execution.

Output

Illustrative runtime output

```
{"TID": "<transaction>", "transactional": true, "transactionProtocol": "trfc", "confirmed": true}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
SAP response result	object Not marked required by editor	Generic editor result placeholder. Native SAP calls return exports, changing parameters, tables and operation-specific metadata; inspect actual Job Data before mapping result.
Completion status successful	boolean Not marked required by editor	Generic editor completion indicator. See the runtime output example: several native SAP operations return operation-specific fields instead of this generic field.

Advanced

Delivery, transactions and performance

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

Implemented behavior and limitations

The adapter builds IDOC_CONTROL_REC_40 and IDOC_DATA_REC_40 table parameters and calls IDOC_INBOUND_ASYNCHRONOUS unless functionName overrides it. confirmed indicates transport confirmation, not final SAP business posting status. Retrying can create another transport transaction; use a business-level duplicate strategy.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Troubleshooting procedure

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

Acceptance test

Acceptance: inspect SAP inbound document status as well as transport success; test a rejected business document and ambiguous network failure.

Fault	Description
SAP_CONNECTION	The SAP JCo/RFC connection could not be established.
SAP_RFC	SAP returned an RFC or BAPI exception.
SAP_IDOC	The IDoc could not be parsed, posted, confirmed, or acknowledged.
SAP_TRANSACTION	The tRFC, qRFC, bgRFC, or transactional context failed.
SAP_SCHEMA	The downloaded RFC, table, or IDoc metadata is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs

IDoc Renderer

Activities / SAP ECC

Overview

Build transport-ready control/data records

Use to turn structured/XML IDoc data into the control/data envelope used by Post IDoc or Reader.

Prerequisites

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: idoc_renderer.

Configuration

Configuration procedure

Select the exact metadata and map the complete document. The saved field offsets, lengths and encoding govern SDATA construction. This operation renders locally; it does not post to SAP.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
SAP ECC shared connection resourceId	resource Not marked required by editor	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: sap
IDoc type fetched from SAP idocType	idoc Not marked required by editor	Select and save the basic IDoc type fetched from the chosen SAP connection. The saved catalog/schema drives segment layout, field offsets and mapping suggestions.
Extension / CIM type extensionType	text Not marked required by editor	Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata.
SAP release release	text Not marked required by editor	SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string.
IDoc delivery protocol idocInputMode	select Not marked required by editor	IDoc delivery protocol. Post defaults to tRFC; Reader defaults to qRFC. qRFC requires a real queue name for ordered dispatch. Choices: tRFC, qRFC

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
payload = ${SAP-Parser.SAPIDoc}
Post IDoc payload = ${SAP-Renderer.rawIDoc}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
SAP activity payload payload	object Required	Map the whole input document expected by this operation. Use the Data tree to select an earlier output; the examples below describe its required shape.
Dynamic connection session ID sessionID	string Not marked required by editor	Map the sessionID returned by Dynamic Connection. It identifies a live, process-local JCo context; do not hard-code or reuse it after termination or restart.
Extension / CIM type extensionType	string Not marked required by editor	Optional IDoc extension/CIM type. It must match the sender/receiver contract; changing it requires refreshing metadata.
SAP release release	string Not marked required by editor	SAP release used to resolve IDoc metadata. Use the release supported by the connected system, not an arbitrary version string.
IDoc delivery protocol idocInputMode	string Not marked required by editor	IDoc delivery protocol. Post defaults to tRFC; Reader defaults to qRFC. qRFC requires a real queue name for ordered dispatch.

Output

Illustrative runtime output

```
{"rawIDoc":{"control":{"IDOCTYP":"<basic-type>"},"data":[{"SEGNAM":"<segment>","SDATA":"<encoded-fields>"}]},"format":"RAW"}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
SAP response result	object Not marked required by editor	Generic editor result placeholder. Native SAP calls return exports, changing parameters, tables and operation-specific metadata; inspect actual Job Data before mapping result.
Completion status successful	boolean Not marked required by editor	Generic editor completion indicator. See the runtime output example: several native SAP operations return operation-specific fields instead of this generic field.

Advanced

Delivery, transactions and performance

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound

workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

Implemented behavior and limitations

rawIDoc is a structured control/data envelope, not necessarily a flat text file. The base editor advertises generic result/successful fields; use actual rawIDoc output. Ensure input fits the fetched field lengths and test encoding before dispatch.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Troubleshooting procedure

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

Acceptance test

Acceptance: render and parse back a fixture with repeating segments, blank fields and non-ASCII text; compare every business field.

Fault	Description
SAP_CONNECTION	The SAP JCo/RFC connection could not be established.
SAP_RFC	SAP returned an RFC or BAPI exception.
SAP_IDOC	The IDoc could not be parsed, posted, confirmed, or acknowledged.
SAP_TRANSACTION	The tRFC, qRFC, bgRFC, or transactional context failed.
SAP_SCHEMA	The downloaded RFC, table, or IDoc metadata is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs

RFC BAPI Listener

Activities / SAP ECC

Overview

Start a flow for an inbound RFC/BAPI call

Use as an event starter registered with the SAP gateway for a selected function.

Prerequisites

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: rfc_bapi_listener.

Configuration

Configuration procedure

Set functionName, programId and gateway settings. Provision the matching SAP destination and registration rules. Bound maximumConnections/maxPendingEvents and choose a reply timeout long enough for the flow. Fetch the function metadata.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
SAP ECC shared connection resourceId	resource Not marked required by editor	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: sap
RFC / BAPI name functionName	text Not marked required by editor	Exact remote-enabled function/BAPI name available in the target SAP repository. Its case and parameter names must match fetched metadata.
Invocation protocol invocationProtocol	select Not marked required by editor	Editor protocol selection. Native bridge implements synchronous RFC and tRFC/qRFC dispatch; bgRFC is not a separately implemented native execution mode. Choices: Request/Reply, tRFC, qRFC, bgRFC
Program ID programId	text Not marked required by editor	Registered-server program ID. Must match the SAP RFC destination and gateway registration permissions.
Gateway host override gatewayHost	text Not marked required by editor	SAP gateway hostname override for inbound RFC registration.
Gateway service override gatewayService	text Not marked required by editor	Gateway service/port override for inbound RFC registration.

Field / key	Type / required	Details
Maximum concurrent requests maximumConnections	number Not marked required by editor	Listener connection/concurrency limit. Size it with SAP gateway capacity and downstream processing, not just CPU count.
Maximum buffered requests maxPendingEvents	number Not marked required by editor	Bound on buffered listener events; prevents unbounded in-memory accumulation while downstream work is slow.
Reply / acknowledgement timeout (seconds) ackTimeoutSeconds	number Not marked required by editor	Maximum listener reply/acknowledgement wait in seconds. Set it above expected flow processing time and test timeout/redelivery behavior.
SAP TIDManager resource tidManagerId	resource Not marked required by editor	Shared SAP transaction-ID store selection. Durable TID state supports duplicate detection; it does not make downstream external writes atomic. Connection type: sap_tid

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
Downstream input = ${SAP-RFC-Listener.imports}
Downstream tables = ${SAP-RFC-Listener.tables}
End response with Reply from RFC BAPI in SAP
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
RFC / BAPI name functionName	string Not marked required by editor	Exact remote-enabled function/BAPI name available in the target SAP repository. Its case and parameter names must match fetched metadata.
Invocation protocol invocationProtocol	string Not marked required by editor	Editor protocol selection. Native bridge implements synchronous RFC and tRFC/qRFC dispatch; bgRFC is not a separately implemented native execution mode.
Program ID programId	string Not marked required by editor	Registered-server program ID. Must match the SAP RFC destination and gateway registration permissions.
Gateway host override gatewayHost	string Not marked required by editor	SAP gateway hostname override for inbound RFC registration.
Gateway service override gatewayService	string Not marked required by editor	Gateway service/port override for inbound RFC registration.
Maximum concurrent requests maximumConnections	number Not marked required by editor	Listener connection/concurrency limit. Size it with SAP gateway capacity and downstream processing, not just CPU count.
Maximum buffered requests maxPendingEvents	number Not marked required by editor	Bound on buffered listener events; prevents unbounded in-memory accumulation while downstream work is slow.

Field / key	Type / required	Details
Reply / acknowledgement timeout (seconds) ackTimeoutSeconds	number Not marked required by editor	Maximum listener reply/acknowledgement wait in seconds. Set it above expected flow processing time and test timeout/redelivery behavior.

Output

Illustrative runtime output

```
{ "RfcRequest": {}, "imports": { "<parameter>": "value" }, "tables": {}, "functionName": "<RFC>", "TID": "<when-present>" }
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
RFC/BAPI request RfcRequest	object Not marked required by editor	Inbound RFC request envelope. Parameter shapes follow the selected SAP function metadata.
RFC import parameters imports	object Not marked required by editor	Inbound RFC import parameter object.
RFC table parameters tables	object Not marked required by editor	Inbound RFC table parameter arrays.
Invoked RFC/BAPI functionName	string Not marked required by editor	Exact remote-enabled function/BAPI name available in the target SAP repository. Its case and parameter names must match fetched metadata.
Invocation protocol invocationProtocol	string Not marked required by editor	Editor protocol selection. Native bridge implements synchronous RFC and tRFC/qRFC dispatch; bgRFC is not a separately implemented native execution mode.
Transaction ID TID	string Not marked required by editor	SAP transaction identifier when provided by the transport. It is not the IDoc business document number and is not a JMS acknowledgement token.
SAP connection ID CPIC_ID	string Not marked required by editor	Connection identifier supplied by SAP transport when available.

Advanced

Delivery, transactions and performance

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

Implemented behavior and limitations

A request/reply caller waits for the flow response. Reply must execute within the originating transport context. bgRFC appears in the editor but there is no dedicated bgRFC native dispatch implementation to claim as supported.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Troubleshooting procedure

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

Acceptance test

Acceptance: invoke from a test SAP caller, inspect imports/tables, return a known value and test timeout/error propagation and concurrent callers.

Fault	Description
SAP_CONNECTION	The SAP JCo/RFC connection could not be established.
SAP_RFC	SAP returned an RFC or BAPI exception.
SAP_IDOC	The IDoc could not be parsed, posted, confirmed, or acknowledged.
SAP_TRANSACTION	The tRFC, qRFC, bgRFC, or transactional context failed.
SAP_SCHEMA	The downloaded RFC, table, or IDoc metadata is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs

Invoke RFC BAPI in SAP

Activities / SAP ECC

Overview

Call a remote-enabled SAP function

Use inside a flow to pass import/changing/table parameters to a remote-enabled function or BAPI.

Prerequisites

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: invoke_rfc_bapi.

Configuration

Configuration procedure

Select SAP resource and exact functionName. Match the fetched parameter types. For a multi-step logical unit of work map sessionID from Dynamic Connection and close the context explicitly.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
SAP ECC shared connection resourceId	resource Not marked required by editor	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: sap
RFC / BAPI name functionName	text Not marked required by editor	Exact remote-enabled function/BAPI name available in the target SAP repository. Its case and parameter names must match fetched metadata.
Invocation protocol invocationProtocol	select Not marked required by editor	Editor protocol selection. Native bridge implements synchronous RFC and tRFC/qRFC dispatch; bgRFC is not a separately implemented native execution mode. Choices: Request/Reply, tRFC, qRFC, bgRFC
SAP queue name queueName	text Not marked required by editor	SAP outbound qRFC queue name. Without a queue name the bridge cannot establish the intended ordered queue execution.
Transactional transactional	boolean Not marked required by editor	Whether a stateful transactional context is requested. Only operations sharing the returned live sessionID participate in that context.
Auto commit autoCommit	boolean Not marked required by editor	Requests BAPI_TRANSACTION_COMMIT for an eligible call. Calls inside an explicit dynamic context suppress per-activity commits until the context boundary.

Field / key	Type / required	Details
Context end contextEnd	boolean Not marked required by editor	Ends the dynamic context after this call; autoCommit controls whether that explicit boundary commits. Do not reuse the session afterwards.
Timeout (ms) timeout	number Not marked required by editor	Unit depends on the activity: Kafka Get uses seconds; SAP editor fields use milliseconds. Consult the runtime notes because SAP worker calls use timeoutSeconds separately.

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
functionName = <authorized remote function>
importParameters = ${input.rfcImports}
changingParameters = ${input.rfcChanging}
tableParameters = ${input.rfcTables}
sessionID = ${SAP-Open.sessionID}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
RFC import parameters importParameters	object Not marked required by editor	Object keyed by exact RFC import parameter names; nested structures are flattened into named JCo parameters.
RFC changing parameters changingParameters	object Not marked required by editor	Object keyed by RFC changing parameter names. Values are sent through the JCo changing-parameter list.
RFC table parameters tableParameters	object Not marked required by editor	Object whose keys are RFC table parameter names and whose values are arrays of row objects, including single-row tables.
Dynamic connection session ID sessionID	string Not marked required by editor	Map the sessionID returned by Dynamic Connection. It identifies a live, process-local JCo context; do not hard-code or reuse it after termination or restart.
RFC / BAPI name functionName	string Not marked required by editor	Exact remote-enabled function/BAPI name available in the target SAP repository. Its case and parameter names must match fetched metadata.
Invocation protocol invocationProtocol	string Not marked required by editor	Editor protocol selection. Native bridge implements synchronous RFC and tRFC/qRFC dispatch; bgRFC is not a separately implemented native execution mode.
SAP queue name queueName	string Not marked required by editor	SAP outbound qRFC queue name. Without a queue name the bridge cannot establish the intended ordered queue execution.
Transactional transactional	boolean Not marked required by editor	Whether a stateful transactional context is requested. Only operations sharing the returned live sessionID participate in that context.

Field / key	Type / required	Details
Auto commit autoCommit	boolean Not marked required by editor	Requests BAPI_TRANSACTION_COMMIT for an eligible call. Calls inside an explicit dynamic context suppress per-activity commits until the context boundary.
Context end contextEnd	boolean Not marked required by editor	Ends the dynamic context after this call; autoCommit controls whether that explicit boundary commits. Do not reuse the session afterwards.
Timeout (ms) timeout	number Not marked required by editor	Unit depends on the activity: Kafka Get uses seconds; SAP editor fields use milliseconds. Consult the runtime notes because SAP worker calls use timeoutSeconds separately.

Output

Illustrative runtime output

```
{"exports":{"<parameter>":"value"},"changing":{},"tables":{"RETURN":[{"TYPE":"S","MESSAGE":"<message>"}]}}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
SAP response result	object Not marked required by editor	Generic editor result placeholder. Native SAP calls return exports, changing parameters, tables and operation-specific metadata; inspect actual Job Data before mapping result.
Completion status successful	boolean Not marked required by editor	Generic editor completion indicator. See the runtime output example: several native SAP operations return operation-specific fields instead of this generic field.

Advanced

Delivery, transactions and performance

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

Implemented behavior and limitations

RETURN is illustrative: not every function has that table. The runtime returns native exports/changing/tables rather than wrapping every result in result. Check business error fields explicitly. bgRFC is not a dedicated supported native path; worker timeout uses timeoutSeconds, so the editor timeout field alone should not be treated as a verified per-call deadline.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"

Field / key	Type / required	Details
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Troubleshooting procedure

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

Acceptance test

Acceptance: test scalar, nested and table parameters, business error returns, rollback/commit boundaries and a disconnected server.

Fault	Description
SAP_CONNECTION	The SAP JCo/RFC connection could not be established.
SAP_RFC	SAP returned an RFC or BAPI exception.
SAP_IDOC	The IDoc could not be parsed, posted, confirmed, or acknowledged.
SAP_TRANSACTION	The tRFC, qRFC, bgRFC, or transactional context failed.
SAP_SCHEMA	The downloaded RFC, table, or IDoc metadata is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs

Reply from RFC BAPI in SAP

Activities / SAP ECC

Overview

Complete the originating RFC request

Use in a flow started by RFC BAPI Listener to send its output parameters back to the waiting SAP caller.

Prerequisites

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: reply_rfc_bapi.

Configuration

Configuration procedure

Map the response to the function output contract. The engine requires the original deliveryId/listenerKey transport context; functionName/listenerActivity alone cannot reply to a request from another job.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
SAP ECC shared connection resourceId	resource Not marked required by editor	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: sap
RFC / BAPI name functionName	text Not marked required by editor	Exact remote-enabled function/BAPI name available in the target SAP repository. Its case and parameter names must match fetched metadata.
RFC BAPI Listener activity listenerActivity	text Not marked required by editor	Design-time reference to the listener. Runtime reply routing uses the current listener transport context, not an arbitrary activity name.

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
payload = ${input.rfcResponse}  
Illustrative response: {"exports":{"RESULT":"OK"},"tables":{}}
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
SAP activity payload payload	object Required	Map the whole input document expected by this operation. Use the Data tree to select an earlier output; the examples below describe its required shape.
Dynamic connection session ID sessionID	string Not marked required by editor	Map the sessionID returned by Dynamic Connection. It identifies a live, process-local JCo context; do not hard-code or reuse it after termination or restart.
RFC / BAPI name functionName	string Not marked required by editor	Exact remote-enabled function/BAPI name available in the target SAP repository. Its case and parameter names must match fetched metadata.
RFC BAPI Listener activity listenerActivity	string Not marked required by editor	Design-time reference to the listener. Runtime reply routing uses the current listener transport context, not an arbitrary activity name.

Output

Illustrative runtime output

```
{"replied":true,"functionName":"<RFC>","response":{"exports":{"RESULT":"OK"}}}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
SAP response result	object Not marked required by editor	Generic editor result placeholder. Native SAP calls return exports, changing parameters, tables and operation-specific metadata; inspect actual Job Data before mapping result.
Completion status successful	boolean Not marked required by editor	Generic editor completion indicator. See the runtime output example: several native SAP operations return operation-specific fields instead of this generic field.

Advanced

Delivery, transactions and performance

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

Implemented behavior and limitations

Running this activity alone in external mode fails without a live listener context. Mock replied=true does not establish an SAP response. Reply within the configured timeout and avoid sending a second response on another branch.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

Field / key	Type / required	Details
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Troubleshooting procedure

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

Acceptance test

Acceptance: verify the exact output in the waiting SAP caller, then test a late reply and a reply outside a listener flow.

Fault	Description
SAP_CONNECTION	The SAP JCo/RFC connection could not be established.
SAP_RFC	SAP returned an RFC or BAPI exception.
SAP_IDOC	The IDoc could not be parsed, posted, confirmed, or acknowledged.
SAP_TRANSACTION	The tRFC, qRFC, bgRFC, or transactional context failed.
SAP_SCHEMA	The downloaded RFC, table, or IDoc metadata is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs

Read Table

Activities / SAP ECC

Overview

Read bounded SAP table rows

Use for an authorized RFC_READ_TABLE query, not unrestricted SQL or a bulk replication engine.

Prerequisites

Install licensed SAP JCo JAR and the OS/architecture-matched native library on the execution host; optional IDoc-library validation also needs its licensed JAR. Configure the SAP shared resource, test connectivity, fetch and save the exact IDoc/function metadata required by the activity. Network routing, gateway registration, SAP authorizations and partner profiles must be provisioned by the SAP team. MINA is not SAP-certified by virtue of using JCo.

Executes the selected SAP ECC operation.

The shared SAP connection supplies system and authentication settings. IDoc metadata selected from SAP defines listener, parser, renderer, and sender structures.

Activity type: sap; operation: read_table.

Configuration

Configuration procedure

Select tableName, explicit fields, ABAP WHERE clauses, delimiter and a bounded rowCount. Use rowSkip for paging and avoid changing source data during paging when consistency matters.

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
SAP ECC shared connection resourceId	resource Not marked required by editor	Select a saved shared connection of this activity family. Its credentials and environment properties are resolved at runtime; this is a resource ID, not a server URL. Connection type: sap
SAP table / view tableName	text Not marked required by editor	Authorized SAP table or view read through RFC_READ_TABLE. This is not a general-purpose database connection.
Fields (comma separated) fields	text Not marked required by editor	For Read Table input, a field-name array or configured comma-separated list. Output fields contains SAP field metadata, not row data.
ABAP WHERE clauses where	text Not marked required by editor	ABAP filter clauses, not SQL. Dynamic input is an array of clause strings. Treat user-supplied values as untrusted and validate them before constructing filters.
Rows to skip rowSkip	number Not marked required by editor	Rows skipped by RFC_READ_TABLE, used with a bounded rowCount for paging.
Maximum rows rowCount	number Not marked required by editor	Maximum rows requested; zero delegates an unbounded read to SAP. Prefer an explicit limit for memory and operational safety.
Delimiter delimiter	text Not marked required by editor	Separator used in RFC_READ_TABLE text rows; choose one absent from column values because the adapter splits by this separator.

Input

Mapping example

Examples are mapping/configuration assignments, not a project import file. Rename activity references using your Data tree, and replace angle-bracket placeholders with your own approved metadata. Keep credentials in properties/secrets.

```
tableName = <authorized-table>
fields = ["<key-field>", "<value-field>"]
where = ["<validated ABAP filter>"]
rowCount = 100
rowSkip = 0
delimiter = |
```

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
Dynamic filter clauses where	array Not marked required by editor	ABAP filter clauses, not SQL. Dynamic input is an array of clause strings. Treat user-supplied values as untrusted and validate them before constructing filters.
Dynamic field list fields	array Not marked required by editor	For Read Table input, a field-name array or configured comma-separated list. Output fields contains SAP field metadata, not row data.
SAP table / view tableName	string Not marked required by editor	Authorized SAP table or view read through RFC_READ_TABLE. This is not a general-purpose database connection.
Rows to skip rowSkip	number Not marked required by editor	Rows skipped by RFC_READ_TABLE, used with a bounded rowCount for paging.
Maximum rows rowCount	number Not marked required by editor	Maximum rows requested; zero delegates an unbounded read to SAP. Prefer an explicit limit for memory and operational safety.
Delimiter delimiter	string Not marked required by editor	Separator used in RFC_READ_TABLE text rows; choose one absent from column values because the adapter splits by this separator.

Output

Illustrative runtime output

```
{"rows":[{"<key-field>":"value", "<value-field>":"text"}], "fields": [], "rowCount":1, "tableName":"<table>"
}
```

IDs and values are illustrative. Optional fields vary by provider response, execution mode and whether a message was received. Use the runtime shape described here when it differs from the generic editor contract.

SAP IDoc listener/parser schemas change with selected IDoc metadata and JSON/XML output mode. Repeating segment fields follow the fetched schema. No live ECC schema is embedded in this generic guide.

Field / key	Type / required	Details
SAP table rows rows	array Not marked required by editor	Array of row objects. SAP Read Table returns decoded text values; convert numeric/date columns explicitly before arithmetic or comparison.

Field / key	Type / required	Details
Field metadata fields	array Not marked required by editor	For Read Table input, a field-name array or configured comma-separated list. Output fields contains SAP field metadata, not row data.

Advanced

Delivery, transactions and performance

SAP transport success and business success are different. Inspect BAPI RETURN/status data even when the RFC call completes. Use bounded payloads and batches; parsing/rendering materializes documents in memory. Persistent outbound workers reduce JVM startup overhead but do not eliminate provider/network latency. Use a live Dynamic Connection session only within its owning runtime; never persist its ID for later reuse.

Implemented behavior and limitations

The adapter splits DATA.WA by the configured delimiter and returns strings. Embedded delimiters can split values incorrectly. SAP row-width/function restrictions still apply; rowCount=0 is potentially unbounded. A successful connection does not imply table authorization.

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Troubleshooting procedure

SAP_CONNECTION: verify native-library loading, routing and logon credentials. SAP_RFC: inspect the function name, parameter metadata and SAP authorization/ABAP error. SAP_IDOC or SAP_SCHEMA: compare basic type, extension, release and saved field offsets with the actual payload. SAP_TRANSACTION: inspect TID/queue/context state before retrying. Some underlying paths raise generic runtime errors instead of the advertised editor fault code; retain the full message and trace.

Acceptance test

Acceptance: test a small known result, no matches, paging, authorization failure and values containing the delimiter.

Fault	Description
SAP_CONNECTION	The SAP JCo/RFC connection could not be established.
SAP_RFC	SAP returned an RFC or BAPI exception.
SAP_IDOC	The IDoc could not be parsed, posted, confirmed, or acknowledged.

Fault	Description
SAP_TRANSACTION	The tRFC, qRFC, bgRFC, or transactional context failed.
SAP_SCHEMA	The downloaded RFC, table, or IDoc metadata is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx; backend/app/runtime.py; backend/app/sap.py; backend/app/java_bridge.py; frontend/scripts/connector-documentation.mjs

JDBC Create

Activities / JDBC

Overview

Executes a JDBC Query, Update, Call Procedure, or SQL Direct contract.

The JDBC designer runs and fetches SQL metadata, derives prepared parameters and result fields, supports dynamic SQL overrides, batching, subsets, NULL handling, maximum rows, transaction overrides, and database-specific drivers.

Activity type: jdbc; operation: insert.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
JDBC shared connection resourceId	resource Required	Connection type: jdbc
Timeout (seconds) timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override transaction behavior overrideTransactionBehavior	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override JDBC connection overrideJdbcConnection	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Insert if record does not exist insertIfRecordDoesNotExist	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Batch update batchUpdate	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Allow mapped SQL override overrideSqlStatement	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

Field / key	Type / required	Details
Database server time zone ServerTimeZone	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Dynamic timeout timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Prepared statement parameters parameters	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Batch parameter records records	array Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic update SQL SqlUpdateStatement	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Insert-if-missing SQL InsertStatement	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override transaction behavior overrideTransactionBehavior	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override JDBC connection overrideJdbcConnection	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Insert if record does not exist insertIfRecordDoesNotExist	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Batch update batchUpdate	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Allow mapped SQL override overrideSqlStatement	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

Field / key	Type / required	Details
Number of updates noOfUpdates	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Affected rows rowCount	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Generated key lastInsertId	integer string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
JDBCConnectionNotFoundException	The selected JDBC shared connection is unavailable.
InvalidTimeZoneException	The database server time zone is invalid.
JDBCSQLException	The database rejected the SQL or procedure call.
LoginTimedOutException	The database login timed out.
InvalidSQLTypeException	A prepared parameter type does not match its database column.
DuplicatedFieldNameException	The query result contains an unsupported duplicate field name.
ActivityTimedOutException	The configured activity timeout was reached.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

JDBC Update

Activities / JDBC

Overview

Executes a JDBC Query, Update, Call Procedure, or SQL Direct contract.

The JDBC designer runs and fetches SQL metadata, derives prepared parameters and result fields, supports dynamic SQL overrides, batching, subsets, NULL handling, maximum rows, transaction overrides, and database-specific drivers.

Activity type: jdbc; operation: update.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
JDBC shared connection resourceId	resource Required	Connection type: jdbc
Timeout (seconds) timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override transaction behavior overrideTransactionBehavior	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override JDBC connection overrideJdbcConnection	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Insert if record does not exist insertIfRecordDoesNotExist	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Batch update batchUpdate	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Allow mapped SQL override overrideSqlStatement	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

Field / key	Type / required	Details
Database server time zone ServerTimeZone	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Dynamic timeout timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Prepared statement parameters parameters	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Batch parameter records records	array Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic update SQL SqlUpdateStatement	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Insert-if-missing SQL InsertStatement	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override transaction behavior overrideTransactionBehavior	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override JDBC connection overrideJdbcConnection	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Insert if record does not exist insertIfRecordDoesNotExist	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Batch update batchUpdate	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Allow mapped SQL override overrideSqlStatement	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

Field / key	Type / required	Details
Number of updates noOfUpdates	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Affected rows rowCount	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Generated key lastInsertId	integer string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
JDBCConnectionNotFoundException	The selected JDBC shared connection is unavailable.
InvalidTimeZoneException	The database server time zone is invalid.
JDBCSQLException	The database rejected the SQL or procedure call.
LoginTimedOutException	The database login timed out.
InvalidSQLTypeException	A prepared parameter type does not match its database column.
DuplicatedFieldNameException	The query result contains an unsupported duplicate field name.
ActivityTimedOutException	The configured activity timeout was reached.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

JDBC Query

Activities / JDBC

Overview

Executes a JDBC Query, Update, Call Procedure, or SQL Direct contract.

The JDBC designer runs and fetches SQL metadata, derives prepared parameters and result fields, supports dynamic SQL overrides, batching, subsets, NULL handling, maximum rows, transaction overrides, and database-specific drivers.

Activity type: jdbc; operation: query.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
JDBC shared connection resourceId	resource Required	Connection type: jdbc
Timeout (seconds) timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Maximum rows (0 retrieves all) maxRows	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override transaction behavior overrideTransactionBehavior	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override JDBC connection overrideJdbcConnection	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Use nil for SQL NULL useNil	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Interpret invalid XML characters interpretInvalidXmlChars	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Process result in subsets processInSubsets	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Subset size subsetSize	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Allow mapped SQL override overrideSqlStatement	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

Field / key	Type / required	Details
Database server time zone ServerTimeZone	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic timeout timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic maximum rows maxRows	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic SQL override SqlStatement	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Prepared statement parameters parameters	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override transaction behavior overrideTransactionBehavior	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override JDBC connection overrideJdbcConnection	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Use nil for SQL NULL useNil	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Interpret invalid XML characters interpretInvalidXmlChars	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Process result in subsets processInSubsets	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Subset size subsetSize	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Allow mapped SQL override overrideSqlStatement	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

Field / key	Type / required	Details
JDBC result set resultSet	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Result records resultSet.Record	array Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Result records rows	array Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Rows returned rowCount	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Last result subset lastSubset	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
JDBCCConnectionNotFoundException	The selected JDBC shared connection is unavailable.
InvalidTimeZoneException	The database server time zone is invalid.
JDBCSQLException	The database rejected the SQL or procedure call.
LoginTimedOutException	The database login timed out.
InvalidSQLTypeException	A prepared parameter type does not match its database column.
DuplicatedFieldNameException	The query result contains an unsupported duplicate field name.
ActivityTimedOutException	The configured activity timeout was reached.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose

sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: `frontend/src/ActivityEditor.tsx`

JDBC Truncate

Activities / JDBC

Overview

Executes a JDBC Query, Update, Call Procedure, or SQL Direct contract.

The JDBC designer runs and fetches SQL metadata, derives prepared parameters and result fields, supports dynamic SQL overrides, batching, subsets, NULL handling, maximum rows, transaction overrides, and database-specific drivers.

Activity type: jdbc; operation: truncate.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
JDBC shared connection resourceId	resource Required	Connection type: jdbc
Timeout (seconds) timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override transaction behavior overrideTransactionBehavior	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override JDBC connection overrideJdbcConnection	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

Field / key	Type / required	Details
Database server time zone ServerTimeZone	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic timeout timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Prepared statement parameters parameters	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Batch parameter records records	array Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic update SQL SqlUpdateStatement	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Insert-if-missing SQL InsertStatement	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override transaction behavior overrideTransactionBehavior	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override JDBC connection overrideJdbcConnection	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

Field / key	Type / required	Details
Number of updates noOfUpdates	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Affected rows rowCount	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Generated key lastInsertId	integer string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
JDBCConnectionNotFoundException	The selected JDBC shared connection is unavailable.
InvalidTimeZoneException	The database server time zone is invalid.

Fault	Description
JDBCSQLException	The database rejected the SQL or procedure call.
LoginTimedOutException	The database login timed out.
InvalidSQLTypeException	A prepared parameter type does not match its database column.
DuplicatedFieldNameException	The query result contains an unsupported duplicate field name.
ActivityTimedOutException	The configured activity timeout was reached.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

JDBC Delete

Activities / JDBC

Overview

Executes a JDBC Query, Update, Call Procedure, or SQL Direct contract.

The JDBC designer runs and fetches SQL metadata, derives prepared parameters and result fields, supports dynamic SQL overrides, batching, subsets, NULL handling, maximum rows, transaction overrides, and database-specific drivers.

Activity type: jdbc; operation: delete.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
JDBC shared connection resourceId	resource Required	Connection type: jdbc
Timeout (seconds) timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override transaction behavior overrideTransactionBehavior	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override JDBC connection overrideJdbcConnection	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Insert if record does not exist insertIfRecordDoesNotExist	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Batch update batchUpdate	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Allow mapped SQL override overrideSqlStatement	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

Field / key	Type / required	Details
Database server time zone ServerTimeZone	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Dynamic timeout timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Prepared statement parameters parameters	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Batch parameter records records	array Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic update SQL SqlUpdateStatement	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Insert-if-missing SQL InsertStatement	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override transaction behavior overrideTransactionBehavior	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override JDBC connection overrideJdbcConnection	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Insert if record does not exist insertIfRecordDoesNotExist	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Batch update batchUpdate	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Allow mapped SQL override overrideSqlStatement	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

Field / key	Type / required	Details
Number of updates noOfUpdates	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Affected rows rowCount	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Generated key lastInsertId	integer string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
JDBCConnectionNotFoundException	The selected JDBC shared connection is unavailable.
InvalidTimeZoneException	The database server time zone is invalid.
JDBCSQLException	The database rejected the SQL or procedure call.
LoginTimedOutException	The database login timed out.
InvalidSQLTypeException	A prepared parameter type does not match its database column.
DuplicatedFieldNameException	The query result contains an unsupported duplicate field name.
ActivityTimedOutException	The configured activity timeout was reached.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

Stored Procedure

Activities / JDBC

Overview

Executes a JDBC Query, Update, Call Procedure, or SQL Direct contract.

The JDBC designer runs and fetches SQL metadata, derives prepared parameters and result fields, supports dynamic SQL overrides, batching, subsets, NULL handling, maximum rows, transaction overrides, and database-specific drivers.

Activity type: jdbc; operation: call.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
JDBC shared connection resourceId	resource Required	Connection type: jdbc
Procedure schema schema	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Catalog or package catalog	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Procedure / function procedure	text Not marked required by editor	Browse and refresh procedure metadata from the JDBC designer below.
Timeout (seconds) timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Maximum rows (0 retrieves all) maxRows	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override transaction behavior overrideTransactionBehavior	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override JDBC connection overrideJdbcConnection	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

Field / key	Type / required	Details
Database server time zone ServerTimeZone	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Dynamic timeout timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
IN and IN/OUT procedure parameters parameters	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Procedure schema schema	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Catalog or package catalog	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Procedure / function procedure	string Not marked required by editor	Browse and refresh procedure metadata from the JDBC designer below.
Maximum rows (0 retrieves all) maxRows	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override transaction behavior overrideTransactionBehavior	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override JDBC connection overrideJdbcConnection	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

Field / key	Type / required	Details
Procedure result sets resultSets	array Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
OUT and IN/OUT parameters outParameters	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Unresolved result sets UnresolvedResultSets	array Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"

Field / key	Type / required	Details
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
JDBCConnectionNotFoundException	The selected JDBC shared connection is unavailable.
InvalidTimeZoneException	The database server time zone is invalid.
JDBCSQLException	The database rejected the SQL or procedure call.
LoginTimedOutException	The database login timed out.
InvalidSQLTypeException	A prepared parameter type does not match its database column.
DuplicatedFieldNameException	The query result contains an unsupported duplicate field name.
ActivityTimedOutException	The configured activity timeout was reached.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

Dynamic SQL

Activities / JDBC

Overview

Executes a JDBC Query, Update, Call Procedure, or SQL Direct contract.

The JDBC designer runs and fetches SQL metadata, derives prepared parameters and result fields, supports dynamic SQL overrides, batching, subsets, NULL handling, maximum rows, transaction overrides, and database-specific drivers.

Activity type: jdbc; operation: dynamic.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
JDBC shared connection resourceId	resource Required	Connection type: jdbc
Timeout (seconds) timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Maximum rows (0 retrieves all) maxRows	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override transaction behavior overrideTransactionBehavior	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override JDBC connection overrideJdbcConnection	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Use nil for SQL NULL useNil	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Interpret invalid XML characters interpretInvalidXmlChars	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Process result in subsets processInSubsets	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Subset size subsetSize	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Allow mapped SQL override overrideSqlStatement	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

Field / key	Type / required	Details
Database server time zone ServerTimeZone	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic timeout timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic maximum rows maxRows	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic SQL override SqlStatement	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Prepared statement parameters parameters	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override transaction behavior overrideTransactionBehavior	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override JDBC connection overrideJdbcConnection	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Use nil for SQL NULL useNil	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Interpret invalid XML characters interpretInvalidXmlChars	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Process result in subsets processInSubsets	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Subset size subsetSize	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Allow mapped SQL override overrideSqlStatement	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

Prepared SQL parameters and fetched output column metadata add typed fields to these base contracts. Dynamic SQL needs matching parameter names and values at runtime.

Field / key	Type / required	Details
JDBC result set resultSet	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Result records resultSet.Record	array Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Result records rows	array Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Rows returned rowCount	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Last result subset lastSubset	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
JDBCCConnectionNotFoundException	The selected JDBC shared connection is unavailable.
InvalidTimeZoneException	The database server time zone is invalid.
JDBCSQLException	The database rejected the SQL or procedure call.
LoginTimedOutException	The database login timed out.
InvalidSQLTypeException	A prepared parameter type does not match its database column.
DuplicatedFieldNameException	The query result contains an unsupported duplicate field name.
ActivityTimedOutException	The configured activity timeout was reached.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose

sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: `frontend/src/ActivityEditor.tsx`

Snowflake Insert

Activities / Snowflake

Overview

Runs Snowflake Insert, Query, Update, Delete, or staged Bulk Load operations.

The Snowflake JDBC shared connection supplies authentication, warehouse, database, schema, pooling, and downloaded table/view metadata. Activity inputs and outputs use MINA's schema-aware Snowflake contracts.

Activity type: snowflake; operation: insert.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Snowflake JDBC connection resourceId	resource Required	Connection type: snowflake
Entity entity	snowflake_entity Not marked required by editor	Tables and views retrieved in the Snowflake connection Schema section.
Activity timeout (seconds) timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override database name overrideDatabaseName	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override schema name overrideSchemaName	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Create table from XSD createTableFromXsd	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Table XSD/schema schemaId	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Table name tableName	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Values columns valueColumns	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Batch size batchSize	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Fault on batch failure faultOnBatchFailure	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Merge existing records merge	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Merge on columns mergeOnColumns	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Entity/table metadata adds actual input record and result row columns.

Field / key	Type / required	Details
Rows to insert records	array Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Activity timeout (seconds) timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override database name overrideDatabaseName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override schema name overrideSchemaName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Create table from XSD createTableFromXsd	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Table XSD/schema schemaId	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Table name tableName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Values columns valueColumns	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Batch size batchSize	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Fault on batch failure faultOnBatchFailure	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Merge existing records merge	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Merge on columns mergeOnColumns	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

Entity/table metadata adds actual input record and result row columns.

Field / key	Type / required	Details
Rows attempted rowsAttempted	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Rows affected rowsAffected	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Batch failure details batchFailures	array Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
SNOWFLAKE_CONNECTION	The Snowflake shared resource or connection pool is unavailable.
SNOWFLAKE_DATABASE_JDBC-500005	Snowflake failed to create a prepared statement.
SNOWFLAKE_DATABASE_JDBC-500007	Snowflake failed to bind prepared-statement parameters.
SNOWFLAKE_DATABASE_JDBC-500009	Snowflake failed to execute the generated query.
SNOWFLAKE_DATABASE_JDBC-500013	The activity could not generate its structured output.
SNOWFLAKE_DATABASE_JDBC-500016	One or more insert batches failed.
SNOWFLAKE_DATABASE_JDBC-500022	Schema or table creation failed.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: `frontend/src/ActivityEditor.tsx`

Snowflake Query

Activities / Snowflake

Overview

Runs Snowflake Insert, Query, Update, Delete, or staged Bulk Load operations.

The Snowflake JDBC shared connection supplies authentication, warehouse, database, schema, pooling, and downloaded table/view metadata. Activity inputs and outputs use MINA's schema-aware Snowflake contracts.

Activity type: snowflake; operation: query.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Snowflake JDBC connection resourceId	resource Required	Connection type: snowflake
Entity entity	snowflake_entity Not marked required by editor	Tables and views retrieved in the Snowflake connection Schema section.
Activity timeout (seconds) timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override database name overrideDatabaseName	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override schema name overrideSchemaName	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Snowflake SELECT statement statement	textarea Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Prepared parameters [name:type] preparedParameters	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Maximum rows (0 retrieves all) maximumRows	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Entity/table metadata adds actual input record and result row columns.

Field / key	Type / required	Details
Prepared statement parameters parameters	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Activity timeout (seconds) timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override database name overrideDatabaseName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override schema name overrideSchemaName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Snowflake SELECT statement statement	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Prepared parameters [name:type] preparedParameters	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Maximum rows (0 retrieves all) maximumRows	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

Entity/table metadata adds actual input record and result row columns.

Field / key	Type / required	Details
Query result records rows	array Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Rows returned rowCount	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"

Field / key	Type / required	Details
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
SNOWFLAKE_CONNECTION	The Snowflake shared resource or connection pool is unavailable.
SNOWFLAKE_DATABASE_JDBC-500005	Snowflake failed to create a prepared statement.
SNOWFLAKE_DATABASE_JDBC-500007	Snowflake failed to bind prepared-statement parameters.
SNOWFLAKE_DATABASE_JDBC-500009	Snowflake failed to execute the generated query.
SNOWFLAKE_DATABASE_JDBC-500013	The activity could not generate its structured output.
SNOWFLAKE_DATABASE_JDBC-500014	Maximum Rows is negative.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

Snowflake Update

Activities / Snowflake

Overview

Runs Snowflake Insert, Query, Update, Delete, or staged Bulk Load operations.

The Snowflake JDBC shared connection supplies authentication, warehouse, database, schema, pooling, and downloaded table/view metadata. Activity inputs and outputs use MINA's schema-aware Snowflake contracts.

Activity type: snowflake; operation: update.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Snowflake JDBC connection resourceId	resource Required	Connection type: snowflake
Entity entity	snowflake_entity Not marked required by editor	Tables and views retrieved in the Snowflake connection Schema section.
Activity timeout (seconds) timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override database name overrideDatabaseName	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override schema name overrideSchemaName	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
SET / Values columns valueColumns	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
WHERE / Parameter columns parameterColumns	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Insert when record does not exist merge	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Merge on columns mergeOnColumns	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Create table if none exists createTableIfNoneExists	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Entity/table metadata adds actual input record and result row columns.

Field / key	Type / required	Details
Rows containing values and parameters records	array Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Activity timeout (seconds) timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override database name overrideDatabaseName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override schema name overrideSchemaName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
SET / Values columns valueColumns	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
WHERE / Parameter columns parameterColumns	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Insert when record does not exist merge	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Merge on columns mergeOnColumns	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Create table if none exists createTableIfNoneExists	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

Entity/table metadata adds actual input record and result row columns.

Field / key	Type / required	Details
Rows affected rowsAffected	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

Field / key	Type / required	Details
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
SNOWFLAKE_CONNECTION	The Snowflake shared resource or connection pool is unavailable.
SNOWFLAKE_DATABASE_JDBC-500005	Snowflake failed to create a prepared statement.
SNOWFLAKE_DATABASE_JDBC-500007	Snowflake failed to bind prepared-statement parameters.
SNOWFLAKE_DATABASE_JDBC-500009	Snowflake failed to execute the generated query.
SNOWFLAKE_DATABASE_JDBC-500013	The activity could not generate its structured output.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

Snowflake Delete

Activities / Snowflake

Overview

Runs Snowflake Insert, Query, Update, Delete, or staged Bulk Load operations.

The Snowflake JDBC shared connection supplies authentication, warehouse, database, schema, pooling, and downloaded table/view metadata. Activity inputs and outputs use MINA's schema-aware Snowflake contracts.

Activity type: snowflake; operation: delete.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Snowflake JDBC connection resourceId	resource Required	Connection type: snowflake
Entity entity	snowflake_entity Not marked required by editor	Tables and views retrieved in the Snowflake connection Schema section.
Activity timeout (seconds) timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override database name overrideDatabaseName	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override schema name overrideSchemaName	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
WHERE / Parameter columns parameterColumns	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Merge when record does not exist merge	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Merge on columns mergeOnColumns	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Create table if none exists createTableIfNoneExists	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Entity/table metadata adds actual input record and result row columns.

Field / key	Type / required	Details
Rows containing delete parameters records	array Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Activity timeout (seconds) timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override database name overrideDatabaseName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override schema name overrideSchemaName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
WHERE / Parameter columns parameterColumns	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Merge when record does not exist merge	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Merge on columns mergeOnColumns	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Create table if none exists createTableIfNoneExists	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

Entity/table metadata adds actual input record and result row columns.

Field / key	Type / required	Details
Rows affected rowsAffected	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"

Field / key	Type / required	Details
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
SNOWFLAKE_CONNECTION	The Snowflake shared resource or connection pool is unavailable.
SNOWFLAKE_DATABASE_JDBC-500005	Snowflake failed to create a prepared statement.
SNOWFLAKE_DATABASE_JDBC-500007	Snowflake failed to bind prepared-statement parameters.
SNOWFLAKE_DATABASE_JDBC-500009	Snowflake failed to execute the generated query.
SNOWFLAKE_DATABASE_JDBC-500013	The activity could not generate its structured output.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

Snowflake Bulk Load

Activities / Snowflake

Overview

Runs Snowflake Insert, Query, Update, Delete, or staged Bulk Load operations.

The Snowflake JDBC shared connection supplies authentication, warehouse, database, schema, pooling, and downloaded table/view metadata. Activity inputs and outputs use MINA's schema-aware Snowflake contracts.

Activity type: snowflake; operation: bulk_load.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Snowflake JDBC connection resourceId	resource Required	Connection type: snowflake
Entity entity	snowflake_entity Not marked required by editor	Tables and views retrieved in the Snowflake connection Schema section.
Activity timeout (seconds) timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override database name overrideDatabaseName	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override schema name overrideSchemaName	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Create table from XSD createTableFromXsd	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Table XSD/schema schemaId	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Table name tableName	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Stage type stageType	select Not marked required by editor	Choices: UserStage, TableStage, NamedStage, AmazonS3
Named stage / storage integration stage namedStage	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Amazon S3 file format fileFormat	select Not marked required by editor	Choices: DelimitedFiles, JSON, AVRO, ORC, PARQUET, XML

Field / key	Type / required	Details
Validation mode validationMode	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Purge stage files purgeStageFiles	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Compress internal-stage data compressData	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
On error onError	select Not marked required by editor	Choices: CONTINUE, SKIP_FILE, SKIP_FILE_<num>, SKIP_FILE_<num>%, ABORT_STATEMENT
Skip file at error count skipFileErrorCount	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Skip file at error percentage skipFileErrorPercentage	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Merge loaded records merge	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Merge on columns mergeOnColumns	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

Entity/table metadata adds actual input record and result row columns.

Field / key	Type / required	Details
Rows for an internal stage records	array Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Local file for an internal stage filePath	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Stage load options loadOptions	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Activity timeout (seconds) timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override database name overrideDatabaseName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Override schema name overrideSchemaName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Interpret empty string as NULL interpretEmptyStringAsNull	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Create table from XSD createTableFromXsd	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Table XSD/schema schemaId	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Table name tableName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Stage type stageType	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Named stage / storage integration stage namedStage	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Amazon S3 file format fileFormat	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Validation mode validationMode	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Purge stage files purgeStageFiles	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Compress internal-stage data compressData	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
On error onError	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Skip file at error count skipFileErrorCount	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Skip file at error percentage skipFileErrorPercentage	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Merge loaded records merge	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Merge on columns mergeOnColumns	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

Entity/table metadata adds actual input record and result row columns.

Field / key	Type / required	Details
Per-file load results loadResults	array Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Source file loadResults.FILE	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Load status loadResults.STATUS	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Rows parsed loadResults.ROWS_PARSED	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Rows loaded loadResults.ROWS_LOADED	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Errors seen loadResults.ERRORS_SEEN	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
First error loadResults.FIRST_ERROR	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
First error line loadResults.FIRST_ERROR_LINE	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
First error character loadResults.FIRST_ERROR_CHARACTER	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
First error column loadResults.FIRST_ERROR_COLUMN_NAME	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
SNOWFLAKE_CONNECTION	The Snowflake shared resource or connection pool is unavailable.
SNOWFLAKE_DATABASE_JDBC-500005	Snowflake failed to create a prepared statement.
SNOWFLAKE_DATABASE_JDBC-500007	Snowflake failed to bind prepared-statement parameters.
SNOWFLAKE_DATABASE_JDBC-500009	Snowflake failed to execute the generated query.
SNOWFLAKE_DATABASE_JDBC-500013	The activity could not generate its structured output.
SNOWFLAKE_DATABASE_JDBC-500018	Uploading data to the Snowflake stage failed.
SNOWFLAKE_DATABASE_JDBC-500020	Bulk-load validation found invalid data.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

AMQP Receive Message

Activities / AMQP

Overview

Sends, receives, gets, or dead-letters AMQP messages.

The shared connection supports RabbitMQ AMQP 0.9.1/1.0 and AMQP 1.0 broker profiles. Destinations, subscriptions, typed messages, properties, settlement handles, recovery, SSL, sessions, and Azure dead-letter behavior are configured in MINA.

Activity type: amqp; operation: receive.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
AMQP connection resourceId	resource Required	Connection type: amqp
Queue name queueName	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Entity type entityType	select Not marked required by editor	Choices: Queue, Topic
Entity name entityName	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Subscription name subscriptionName	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Durable subscription durableSubscription	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Shared subscription sharedSubscription	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Message type messageType	select Not marked required by editor	Choices: TextMessage, BytesMessage, Simple, Any
Maximum messages maxMessages	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Acknowledge mode acknowledgeMode	select Not marked required by editor	Choices: Auto, Client, Client or Dead Letter Queue
Azure receiver mode receiverMode	select Not marked required by editor	Choices: PeekLock, ReceiveAndDelete

Field / key	Type / required	Details
Get correlation ID getCorrelationID	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Azure session enabled sessionEnabled	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Session receive type receiveType	select Not marked required by editor	Choices: FirstAvailableSession, SpecificSession, AllAvailableSessions
Maximum concurrent sessions maxConcurrentSessions	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Specific session ID sessionId	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Prefetch count / credits prefetchCount	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Dynamic queue queueName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic topic topicName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic subscription subscriptionName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Entity type entityType	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Entity name entityName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Durable subscription durableSubscription	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Shared subscription sharedSubscription	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Message type messageType	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Maximum messages maxMessages	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Acknowledge mode acknowledgeMode	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Azure receiver mode receiverMode	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Get correlation ID getCorrelationID	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Azure session enabled sessionEnabled	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Session receive type receiveType	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Maximum concurrent sessions maxConcurrentSessions	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Specific session ID sessionId	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Prefetch count / credits prefetchCount	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Message received received	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
User properties UserProperties	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
AMQP message properties MessageProperties	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Message body body	string binary Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Client/dead-letter settlement token settlementToken	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Confirm Message handle ackId	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
AMQPPluginException	The AMQP operation failed or its message configuration is invalid.
AMQPConnectionException	The broker connection failed or could not be recovered.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

AMQP Get Message

Activities / AMQP

Overview

Sends, receives, gets, or dead-letters AMQP messages.

The shared connection supports RabbitMQ AMQP 0.9.1/1.0 and AMQP 1.0 broker profiles. Destinations, subscriptions, typed messages, properties, settlement handles, recovery, SSL, sessions, and Azure dead-letter behavior are configured in MINA.

Activity type: amqp; operation: get.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
AMQP connection resourceId	resource Required	Connection type: amqp
Queue name queueName	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Entity type entityType	select Not marked required by editor	Choices: Queue, Topic
Entity name entityName	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Subscription name subscriptionName	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Durable subscription durableSubscription	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Shared subscription sharedSubscription	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Message type messageType	select Not marked required by editor	Choices: TextMessage, BytesMessage, Simple
Maximum messages maxMessages	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Acknowledge mode acknowledgeMode	select Not marked required by editor	Choices: Auto, Client
Azure receiver mode receiverMode	select Not marked required by editor	Choices: PeekLock, ReceiveAndDelete

Field / key	Type / required	Details
Get correlation ID getCorrelationID	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Azure session enabled sessionEnabled	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Session receive type receiveType	select Not marked required by editor	Choices: FirstAvailableSession, SpecificSession, AllAvailableSessions
Maximum concurrent sessions maxConcurrentSessions	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Specific session ID sessionId	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Prefetch count / credits prefetchCount	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Dynamic queue queueName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic topic topicName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic subscription subscriptionName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Entity type entityType	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Entity name entityName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Durable subscription durableSubscription	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Shared subscription sharedSubscription	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Message type messageType	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Maximum messages maxMessages	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Acknowledge mode acknowledgeMode	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Azure receiver mode receiverMode	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Get correlation ID getCorrelationID	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Azure session enabled sessionEnabled	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Session receive type receiveType	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Maximum concurrent sessions maxConcurrentSessions	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Specific session ID sessionId	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Prefetch count / credits prefetchCount	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Message received received	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
User properties UserProperties	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
AMQP message properties MessageProperties	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Message body body	string binary Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Client/dead-letter settlement token settlementToken	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Confirm Message handle ackId	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
AMQPPluginException	The AMQP operation failed or its message configuration is invalid.
AMQPConnectionException	The broker connection failed or could not be recovered.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

AMQP Send Message

Activities / AMQP

Overview

Sends, receives, gets, or dead-letters AMQP messages.

The shared connection supports RabbitMQ AMQP 0.9.1/1.0 and AMQP 1.0 broker profiles. Destinations, subscriptions, typed messages, properties, settlement handles, recovery, SSL, sessions, and Azure dead-letter behavior are configured in MINA.

Activity type: amqp; operation: send.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
AMQP connection resourceId	resource Required	Connection type: amqp
RabbitMQ destination type destinationType	select Not marked required by editor	Choices: Queue, Exchange
Queue name queueName	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Entity type entityType	select Not marked required by editor	Choices: Queue, Topic
Entity name entityName	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Exchange type exchangeType	select Not marked required by editor	Choices: direct, topic, fanout, headers
Exchange name exchangeName	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Routing key routingKey	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Message type messageType	select Not marked required by editor	Choices: TextMessage, BytesMessage, Simple
Get message ID getMessageID	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Delivery mode deliveryMode	select Not marked required by editor	Choices: Persistent, Non-Persistent

Field / key	Type / required	Details
Expiration (msec) expiration	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Priority (0-9) priority	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Message type/property type	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
User properties userProperties	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic delivery mode deliveryMode	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Message ID messageID	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic expiration expiration	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic priority priority	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic type type	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Content type contentType	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Azure session ID sessionID	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Correlation ID correlationID	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic exchange exchangeName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic routing key routingKey	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Dynamic topic topicName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic queue queueName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Message body body	string binary Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
RabbitMQ destination type destinationType	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Entity type entityType	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Entity name entityName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Exchange type exchangeType	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Message type messageType	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Get message ID getMessageID	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Send succeeded sendResult	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Broker message ID MessageId	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"

Field / key	Type / required	Details
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
AMQPPluginException	The AMQP operation failed or its message configuration is invalid.
AMQPConnectionException	The broker connection failed or could not be recovered.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

AMQP Dead Letter Message

Activities / AMQP

Overview

Sends, receives, gets, or dead-letters AMQP messages.

The shared connection supports RabbitMQ AMQP 0.9.1/1.0 and AMQP 1.0 broker profiles. Destinations, subscriptions, typed messages, properties, settlement handles, recovery, SSL, sessions, and Azure dead-letter behavior are configured in MINA.

Activity type: amqp; operation: dead_letter.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
AMQP connection resourceId	resource Required	Connection type: amqp
Use retry useRetry	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Total timeout (seconds) totalTimeoutSeconds	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Maximum attempts maxAttempts	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Backoff time (msec) backoffTimeMsec	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Settlement token settlementToken	string Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dead-letter reason deadLetterReason	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dead-letter error description deadLetterErrorDescription	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic retry useRetry	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Dynamic total timeout totalTimeoutSeconds	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic attempts maxAttempts	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic backoff backoffTimeMsec	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Dead-letter status status	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Settlement token settlementToken	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Processed message ID messageId	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"
retryEnabled retryEnabled	boolean expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryEnabled}"
retryCount retryCount	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	number expression Not marked required by editor	Outbound retry policy resolved from the active environment. Retries may repeat external side effects; design for idempotency. Default: "\${properties.advanced.retryIntervalSeconds}"

Outbound retry is available for this operation. Coordinate it with any Repeat-on-Error group to avoid multiplying attempts.

Errors

Fault	Description
AMQPPluginException	The AMQP operation failed or its message configuration is invalid.
AMQPConnectionException	The broker connection failed or could not be recovered.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

Parse XML

Activities / Data

Overview

Parses XML text into an XSD-defined tree or renders an XSD-defined tree as XML.

Parse XML accepts serialized XML in Input and defines its published tree in Output Editor. Render XML defines and maps its source tree in Input Editor, then emits xmlString (or binary output when selected).

Activity type: xml; operation: parse.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Input style inputStyle	select Not marked required by editor	Choices: Text, Binary, Dynamic
Validate parsed output validateOutput	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Encoding encoding	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The selected XSD or inline schema determines actual parse output or render input.

Field / key	Type / required	Details
XML text xmlString	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
XML binary/base64 xmlBinary	binary Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Forced encoding forceEncoding	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The selected XSD or inline schema determines actual parse output or render input.

Field / key	Type / required	Details
Parsed XML xml	xml Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
XML_PARSE	The XML input cannot be parsed.
XML_RENDER	The value cannot be rendered as XML.
SCHEMA_VALIDATION	The data does not conform to the selected schema.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

Render XML

Activities / Data

Overview

Parses XML text into an XSD-defined tree or renders an XSD-defined tree as XML.

Parse XML accepts serialized XML in Input and defines its published tree in Output Editor. Render XML defines and maps its source tree in Input Editor, then emits xmlString (or binary output when selected).

Activity type: xml; operation: render.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Output style outputStyle	select Not marked required by editor	Choices: Text, Binary
Validate input tree validateInput	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Encoding encoding	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Suppress XML declaration suppressXmlDeclaration	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Pretty print prettyPrint	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The selected XSD or inline schema determines actual parse output or render input.

Field / key	Type / required	Details
XML schema tree value	object Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The selected XSD or inline schema determines actual parse output or render input.

Field / key	Type / required	Details
Rendered XML text xmlString	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
XML_PARSE	The XML input cannot be parsed.
XML_RENDER	The value cannot be rendered as XML.
SCHEMA_VALIDATION	The data does not conform to the selected schema.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

Parse JSON

Activities / Data

Overview

Converts between serialized JSON and a schema-defined activity tree.

Parse JSON accepts jsonString and publishes the structure selected in Output Editor. Render JSON maps the structure selected in Input Editor and emits jsonString.

Activity type: json; operation: parse.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Validate parsed output validateOutput	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Duplicate keys duplicateKeyPolicy	select Not marked required by editor	Choices: Last wins, First wins, Error

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The selected JSON schema determines actual parse output or render input.

Field / key	Type / required	Details
JSON text jsonString	string Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The selected JSON schema determines actual parse output or render input.

Field / key	Type / required	Details
Parsed JSON value value	json Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
JSON_PARSE	The JSON input cannot be parsed.
JSON_RENDER	The value cannot be rendered as JSON.
SCHEMA_VALIDATION	The data does not conform to the selected schema.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

Render JSON

Activities / Data

Overview

Converts between serialized JSON and a schema-defined activity tree.

Parse JSON accepts jsonString and publishes the structure selected in Output Editor. Render JSON maps the structure selected in Input Editor and emits jsonString.

Activity type: json; operation: render.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Validate input tree validateInput	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
JSON root rootStyle	select Not marked required by editor	Choices: With root, Anonymous
Pretty print prettyPrint	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Indent spaces indent	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Escape non-ASCII characters asciiOnly	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Omit null fields omitNulls	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The selected JSON schema determines actual parse output or render input.

Field / key	Type / required	Details
JSON schema tree value	object Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The selected JSON schema determines actual parse output or render input.

Field / key	Type / required	Details
Rendered JSON text jsonString	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
JSON_PARSE	The JSON input cannot be parsed.
JSON_RENDER	The value cannot be rendered as JSON.
SCHEMA_VALIDATION	The data does not conform to the selected schema.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

Parse Data

Activities / Data

Overview

Parses delimited/fixed-width text into records or renders records as formatted text.

The schema contract defines record fields; Configuration controls separators, widths, headers, trimming, and line endings. Parse Data publishes records while Render Data accepts mapped records and emits text.

Activity type: flat; operation: parse.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Input source inputSource	select Not marked required by editor	Choices: String, File path
Input file path filePath	text Not marked required by editor	Read the formatted data directly from this local runtime path when Input source is File path.
File encoding fileEncoding	text Not marked required by editor	Defaults to UTF-8 with optional BOM detection.
Output XML root element rootElement	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Output XML record element recordElement	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Format format	select Not marked required by editor	Choices: delimited, fixed
Column separator delimiter	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Multi-character separator rule separatorRule	select Not marked required by editor	Choices: single, any-character
Line separator lineSeparator	select Not marked required by editor	Choices: Auto, LF, CRLF, CR
First row is header header	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Field names fields	text Not marked required by editor	Comma-separated names used when the input has no header. An attached schema supplies these automatically.

Field / key	Type / required	Details
Field data types fieldTypes	text Not marked required by editor	Optional comma-separated XML schema types such as string, integer, decimal, and boolean.
Fixed field widths widths	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Fixed-width fill character fillCharacter	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Require exact column count strictColumns	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Trim parsed values trimValues	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Skip blank lines skipBlankLines	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Append final line separator includeFinalLineSeparator	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The record schema determines child fields under records.

Field / key	Type / required	Details
Formatted data string text	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Direct file path filePath	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The record schema determines child fields under records.

Field / key	Type / required	Details
Parsed data XML xml	xml Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
FLAT_PARSE	The FLAT input cannot be parsed.
FLAT_RENDER	The value cannot be rendered as FLAT.
SCHEMA_VALIDATION	The data does not conform to the selected schema.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

Render Data

Activities / Data

Overview

Parses delimited/fixed-width text into records or renders records as formatted text.

The schema contract defines record fields; Configuration controls separators, widths, headers, trimming, and line endings. Parse Data publishes records while Render Data accepts mapped records and emits text.

Activity type: flat; operation: render.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Format format	select Not marked required by editor	Choices: delimited, fixed
Column separator delimiter	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Multi-character separator rule separatorRule	select Not marked required by editor	Choices: single, any-character
Line separator lineSeparator	select Not marked required by editor	Choices: Auto, LF, CRLF, CR
First row is header header	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Field names fields	text Not marked required by editor	Comma-separated names used when the input has no header. An attached schema supplies these automatically.
Field data types fieldTypes	text Not marked required by editor	Optional comma-separated XML schema types such as string, integer, decimal, and boolean.
Fixed field widths widths	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Fixed-width fill character fillCharacter	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Require exact column count strictColumns	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Trim parsed values trimValues	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Skip blank lines skipBlankLines	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Append final line separator includeFinalLineSeparator	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The record schema determines child fields under records.

Field / key	Type / required	Details
Data records records	array Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The record schema determines child fields under records.

Field / key	Type / required	Details
Rendered FLAT text text	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
FLAT_PARSE	The FLAT input cannot be parsed.
FLAT_RENDER	The value cannot be rendered as FLAT.
SCHEMA_VALIDATION	The data does not conform to the selected schema.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

Read Excel Workbook

Activities / Data

Overview

Reads an Excel workbook and publishes every selected worksheet as nested JSON.

Leave Worksheet blank to read every tab. The configured header row becomes field names; dotted headers such as customer.address.city create nested objects. Formula cells can return their cached calculated values.

Activity type: excel; operation: read.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Workbook file path filePath	text Not marked required by editor	Reads .xlsx, .xlsm, or legacy .xls workbooks at runtime.
Worksheet name sheetName	text Not marked required by editor	Leave blank to return every worksheet/tab.
Header row headerRow	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
First data row startRow	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Maximum rows per sheet (0 = all) maximumRows	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Return calculated cell values dataOnly	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Convert dotted headers into nested JSON fields nestedHeaders	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Skip blank rows skipBlankRows	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Dynamic workbook path filePath	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Dynamic worksheet/tab name sheetName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic header row headerRow	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic first row startRow	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic maximum rows maximumRows	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Return calculated cell values dataOnly	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Convert dotted headers into nested JSON fields nestedHeaders	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Skip blank rows skipBlankRows	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Workbook result workbook	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Workbook file name workbook.fileName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Worksheet count workbook.sheetCount	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Worksheets/tabs workbook.sheets	array Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Worksheet name workbook.sheets.name	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Column headers workbook.sheets.headers	array Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Nested JSON row data workbook.sheets.rows	array Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
EXCEL_READ	The workbook, requested worksheet, or cell data could not be read.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

Mapper

Activities / General

Overview

Maps execution-path data into a selected target schema.

The consolidated Mapper includes schema trees, XPath-style functions, repeating For-Each/For-Each-Group rules, AI-assisted recommendations, validation, and an executable test surface.

Activity type: mapper; operation: map.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

No additional fields declared for this section.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The selected target schema and mapping rules determine actual output fields.

Field / key	Type / required	Details
Source document source	object Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The selected target schema and mapping rules determine actual output fields.

Field / key	Type / required	Details
Mapped document result	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
MAPPING	A mapping expression failed.
SCHEMA_VALIDATION	The mapped output violates its target schema.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

Mediation

Activities / General

Overview

Converts partner messages into a consistent business structure using readable mediation recipes.

Configuration provides JSON/XML/CSV formats, source collection, field rules, required fields, defaults, constants, conversions and value lookup. Input payload defaults to the previous activity output; map payload to override it. Output is the transformed document directly, not a result wrapper. Preview uses the same engine as Run, Debug and Python export and explains each rule without contacting connectors. Advanced retry/logging policies are inherited. MEDIATION errors identify failed records/rules without exposing field values. Load Normalize orders, CSV customer onboarding or XML to business JSON for working examples. Limits: 2 MB text, 10,000 records, 200 rules; CSV output is flat. XML attributes become fields, repeated children become lists, mixed text uses `_text`; DTDs/entities and namespace-prefixed output names are not supported. This is deterministic rule-based mediation, not AI inference or full XSD validation.

Activity type: mediation; operation: mediate.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

No additional fields declared for this section.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Source message payload	any Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Mediated document (direct activity output) result	any Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: <code>"\${properties.advanced.logPayload}"</code>

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
MEDIATION	Missing required fields, invalid rules, conversion errors or unsupported document shape. Preview identifies the record and rule.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

Transform

Activities / General

Overview

Transforms payloads with MINA's executable DataWeave-compatible language subset.

Scripts support payload/attributes/vars selectors, objects, arrays, defaults, conditionals, concatenation, common coercion/string/collection functions, map/filter/groupBy/orderBy/distinctBy, JSON, XML, and text output. The editor validates and runs the same engine used at runtime.

Activity type: dataweave; operation: transform.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

No additional fields declared for this section.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
DataWeave payload payload	object Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Input attributes attributes	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Transform variables variables	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Transformed payload result	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
DATAWEAVE_SYNTAX	The transform script could not be parsed.
DATAWEAVE_EXECUTION	A selector, function, coercion, or collection operation failed.
DATAWEAVE_OUTPUT	The transformed value cannot be rendered in the requested MIME type.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

Log

Activities / General

Overview

Writes a structured application log event.

Message, level, and payload accept constants or dynamic mappings. Advanced automatic payload logging can be used when a dedicated Log activity is unnecessary.

Activity type: log; operation: write.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Log level level	select Not marked required by editor	Choices: DEBUG, INFO, WARN, ERROR
Message template message	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Include payload includePayload	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Message expression message	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Payload text payload	string Not marked required by editor	Map a value or enter a text/expression; objects are rendered into the log as structured JSON.
Log level level	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Include payload includePayload	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Unchanged payload payload	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
LOGGING	The runtime logger rejected the event.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

Catch Exception

Activities / General

Overview

Runs Catch Exception on the task execution path.

Activity type: catch; operation: catch.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Catch all unhandled exceptions catchAll	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Exception type errorType	text Not marked required by editor	Used when Catch All is false; for example XMLParseException.
Error code filter errorCode	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Catch all unhandled exceptions catchAll	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Exception type errorType	string Not marked required by editor	Used when Catch All is false; for example XMLParseException.
Error code filter errorCode	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Exception type type	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Error code code	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Error message message	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Stack trace stackTrace	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Failed activity ID activityId	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Fault details details	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Original cause cause	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

No additional fields declared for this section.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

Throw Exception

Activities / General

Overview

Runs Throw Exception on the task execution path.

Activity type: throw; operation: throw.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Exception type errorType	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Error code code	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Error message message	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Fault details (JSON) details	textarea Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Error message message	string Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Error code code	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Exception type type	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Stack trace stackTrace	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Fault details details	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Exception type errorType	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

No additional fields declared for this section.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
UserDefinedException	The configured business fault is raised to the nearest matching handler.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

Rethrow Exception

Activities / General

Overview

Runs Rethrow Exception on the task execution path.

Activity type: rethrow; operation: rethrow.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

No additional fields declared for this section.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Current caught exception error	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

No additional fields declared for this section.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
RethrowException	The current caught fault is propagated to the next available handler or calling task.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

Confirm Message

Activities / General

Overview

Acknowledges one or more client-managed messages.

Map the acknowledgement handle emitted by an EMS, Kafka, Pub/Sub, or compatible future receiver. The runtime dispatches confirmation to the originating connector.

Activity type: confirm; operation: acknowledge.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Acknowledgement handle ackId	text Not marked required by editor	Map a broker acknowledgement handle, or place this activity after an SAP IDoc Parser to confirm the SAP tRFC early.
Fail when handle is missing failIfMissing	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Acknowledgement handle ackId	string Not marked required by editor	Optional for SAP IDoc early confirmation; otherwise map the client acknowledgement handle.
Multiple acknowledgement handles ackIds	array Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Fail when handle is missing failIfMissing	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Confirmation status confirmed	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Confirmed message count count	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Confirmed transports technologies	array Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
ACKNOWLEDGEMENT_NOT_FOUND	The acknowledgement handle is missing, expired, or was already confirmed.
SAP_EARLY_CONFIRM_FAILED	SAP rejected the early IDoc confirmation.
ACKNOWLEDGEMENT_FAILED	The transport rejected the message confirmation.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

Java Activity

Activities / General

Overview

Runs Java Activity on the task execution path.

Activity type: java; operation: invoke.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Source mode mode	select Not marked required by editor	Choices: JAR / class, Inline source
JAR, .class, or .java artifactPath	artifact Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Fully qualified Java class className	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Method method	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Instantiate class instantiate	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Inline Java source sourceCode	textarea Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Timeout seconds timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Method parameters parameters	array Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
JSON input payload	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Source mode mode	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
JAR, .class, or .java artifactPath	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Fully qualified Java class className	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Method method	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Instantiate class instantiate	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Inline Java source sourceCode	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Timeout seconds timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Method return value methodReturnValue	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Invoked class className	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Invoked method method	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
JAVA_CLASS_NOT_FOUND	The configured class or JAR cannot be loaded.

Fault	Description
JAVA_INVOCATION	The Java method threw an exception.
JAVA_TIMEOUT	The worker exceeded its timeout.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

Python Invoke

Activities / General

Overview

Runs Python Invoke on the task execution path.

Activity type: python; operation: invoke.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Source mode mode	select Not marked required by editor	Choices: Python file / package, Inline source
.py, .zip, or .whl artifactPath	artifact Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Module name moduleName	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Function function	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Inline Python source sourceCode	textarea Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Timeout seconds timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Function parameters parameters	array Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
JSON input payload	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Source mode mode	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
.py, .zip, or .whl artifactPath	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Module name moduleName	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Function function	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Inline Python source sourceCode	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Timeout seconds timeout	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Python result result	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Loaded module module	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Invoked function function	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
PYTHON_IMPORT	The module or package could not be imported.
PYTHON_INVOCATION	The selected Python function raised an exception.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

External Command

Activities / General

Overview

Executes the selected general-purpose activity.

External Command runs an executable without a shell, accepts stdin/environment/working-directory overrides, can capture or persist stdout and stderr, applies configured line splitting, and publishes the process return code.

Activity type: basic; operation: `external_command`.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Command to execute command	textarea Not marked required by editor	Executable and arguments. Shell pipes are intentionally not evaluated.
Provide command output provideCommandOutput	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Remove parameter quotes removeParameterQuotes	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Output filename outputFile	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Output line splitting outputLineSplitting	select Not marked required by editor	Choices: None, AtOperatingSystemLineEnd, AtSpecifiedToken
Specified split token splitToken	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Working directory workingDirectory	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Environment variables [NAME=value,...] environment	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Replace inherited environment replaceEnvironment	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Timeout seconds timeoutSeconds	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Output encoding encoding	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Dynamic command command	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Standard input input	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic output file outputFile	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic environment environment	object string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic working directory workingDirectory	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Dynamic split token splitToken	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Provide command output provideCommandOutput	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Remove parameter quotes removeParameterQuotes	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Output line splitting outputLineSplitting	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Replace inherited environment replaceEnvironment	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Timeout seconds timeoutSeconds	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Output encoding encoding	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Process return code returnCode	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Field / key	Type / required	Details
Standard output output	string array Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Standard error error	string array Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Written output file outputFile	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
CommandExecutionError	The operating system command could not start or timed out.
FileIOError	Command output could not be written to the configured file.
InvalidInputException	The command or line-splitting configuration is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

Assign

Activities / General

Overview

Executes the selected general-purpose activity.

External Command runs an executable without a shell, accepts stdin/environment/working-directory overrides, can capture or persist stdout and stderr, applies configured line splitting, and publishes the process return code.

Activity type: basic; operation: assign.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Process variable variable	text Not marked required by editor	Name stored in process variables.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Assigned value value	object Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Process variable variable	string Not marked required by editor	Name stored in process variables.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Variable name name	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Assigned value value	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.

Field / key	Type / required	Details
	editor	Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
VALIDATION	Required input or mapped data is invalid.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

Checkpoint

Activities / General

Overview

Executes the selected general-purpose activity.

External Command runs an executable without a shell, accepts stdin/environment/working-directory overrides, can capture or persist stdout and stderr, applies configured line splitting, and publishes the process return code.

Activity type: basic; operation: checkpoint.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Checkpoint name checkpointName	text Not marked required by editor	Optional logical name stored in runtime execution metadata.
Include process state includeProcessState	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Checkpoint name checkpointName	string Not marked required by editor	Optional logical name stored in runtime execution metadata.
Include process state includeProcessState	boolean Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Checkpoint ID checkpointId	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Checkpoint timestamp timestamp	dateTime Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by editor	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data. Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
CheckpointException	The process state could not be persisted.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

Sleep

Activities / General

Overview

Executes the selected general-purpose activity.

External Command runs an executable without a shell, accepts stdin/environment/working-directory overrides, can capture or persist stdout and stderr, applies configured line splitting, and publishes the process return code.

Activity type: basic; operation: sleep.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Duration duration	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Unit unit	select Not marked required by editor	Choices: milliseconds, seconds, minutes

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Dynamic duration duration	number Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Unit unit	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Elapsed delay sleptMilliseconds	integer Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.

Field / key	Type / required	Details
	editor	Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

Fault	Description
INTERRUPTED	The wait was interrupted or cancelled.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: frontend/src/ActivityEditor.tsx

Get Shared Variable

Activities / General

Overview

Executes the selected general-purpose activity.

External Command runs an executable without a shell, accepts stdin/environment/working-directory overrides, can capture or persist stdout and stderr, applies configured line splitting, and publishes the process return code.

Activity type: basic; operation: get_shared_variable.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Shared variable name name	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Default value default	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Shared variable name name	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Variable name name	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Shared value value	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.

Field / key	Type / required	Details
	editor	Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

No additional fields declared for this section.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

Set Shared Variable

Activities / General

Overview

Executes the selected general-purpose activity.

External Command runs an executable without a shell, accepts stdin/environment/working-directory overrides, can capture or persist stdout and stderr, applies configured line splitting, and publishes the process return code.

Activity type: basic; operation: set_shared_variable.

Configuration

Display name identifies this activity and its data references. Configuration supplies operation settings and shared-resource selection. The table lists the editor contract; additional connection requirements are documented under Shared connections.

Field / key	Type / required	Details
Shared variable name name	text Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Input

Map fields using the Data tree, functions, constants or environment properties. The table includes configuration values exposed as runtime mapping targets by Studio.

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Shared value value	object Required	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Shared variable name name	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Output

The table is the base editor contract. Runtime values depend on mapped inputs, selected operation and provider response.

Field / key	Type / required	Details
Variable name name	string Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.
Shared value value	object Not marked required by editor	No additional field description is supplied by the editor. Requiredness can also depend on operation, selected schema, provider and runtime validation.

Advanced

Field / key	Type / required	Details
logPayload logPayload	boolean expression Not marked required by	Controls automatic payload logging; browse to choose another environment property. Avoid logging sensitive data.

Field / key	Type / required	Details
	editor	Default: "\${properties.advanced.logPayload}"

This operation does not expose outbound retry in the common Advanced settings.

Errors

No additional fields declared for this section.

On error: propagate (default), continue with an error document, or ignore and continue. Error output variable selects where the fault is exposed; Include activity input in fault is enabled by default. Including payloads in faults or logs may expose sensitive data. An empty declared-fault list does not guarantee that execution cannot fail.

Reference source: [frontend/src/ActivityEditor.tsx](#)

If

Groups

Configuration

Evaluates condition before entry; false skips the group.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use `parent_group_id` for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

Field / key	Type / required	Details
condition condition	expression / setting Not marked required by editor	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.

Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${input.enabled} == true
```

Output

Member activities publish their normal outputs. Groups do not have a separate universal output schema.

Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: `backend/app/runtime.py`; `frontend/src/main.tsx`

While True

Groups

Configuration

Evaluates condition before each iteration, including the first.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use `parent_group_id` for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

Field / key	Type / required	Details
condition condition	expression / setting Not marked required by editor	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.
maxIterations maxIterations	expression / setting Not marked required by editor	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. Default: "\${properties.advanced.groupMaxIterations}"
indexVariable indexVariable	expression / setting Not marked required by editor	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. Default: "index"

Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${input.enabled} == true
```

Output

Member activities publish their normal outputs. Groups do not have a separate universal output schema.

Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: `backend/app/runtime.py`; `frontend/src/main.tsx`

For Each

Groups

Configuration

Resolves the collection at entry and runs once per item. Range mode uses inclusive start/end with a nonzero increment.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use `parent_group_id` for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

Field / key	Type / required	Details
iterationMode iterationMode	expression / setting Not marked required by editor	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. Default: "collection"
source source	expression / setting Not marked required by editor	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.
start start	expression / setting Not marked required by editor	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.
end end	expression / setting Not marked required by editor	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.
increment increment	expression / setting Not marked required by editor	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.
currentElementName currentElementName	expression / setting Not marked required by editor	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. Default: "currentElement"
itemVariable itemVariable	expression / setting Not marked required by editor	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. Default: "currentElement"
indexVariable indexVariable	expression / setting Not marked required by editor	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. Default: "index"
maxIterations maxIterations	expression / setting Not marked required by editor	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. Default: "\${properties.advanced.groupMaxIterations}"
accumulateOutput accumulateOutput	expression / setting Not marked required by editor	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.
accumulatorVariable accumulatorVariable	expression / setting Not marked required by editor	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.

Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${input.records}  
${vars.currentElement}
```



```
${vars.currentIndex}
```

Output

The configured current element and one-based index are visible inside the group. Optional accumulation records each successful group-exit result under `accumulatorVariable`.

Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: `backend/app/runtime.py`; `frontend/src/main.tsx`

Iterate

Groups

Configuration

Resolves source once at entry and iterates its elements.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use `parent_group_id` for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

Field / key	Type / required	Details
source source	expression / setting Not marked required by editor	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.
currentElementName currentElementName	expression / setting Not marked required by editor	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. Default: "currentElement"
itemVariable itemVariable	expression / setting Not marked required by editor	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. Default: "currentElement"
indexVariable indexVariable	expression / setting Not marked required by editor	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. Default: "index"
maxIterations maxIterations	expression / setting Not marked required by editor	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. Default: "\${properties.advanced.groupMaxIterations}"
accumulateOutput accumulateOutput	expression / setting Not marked required by editor	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.
accumulatorVariable accumulatorVariable	expression / setting Not marked required by editor	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.

Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${input.records}  
${vars.currentElement}  
${vars.currentIndex}
```

Output

The configured current element and one-based index are visible inside the group. Optional accumulation records each successful group-exit result under `accumulatorVariable`.

Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: `backend/app/runtime.py`; `frontend/src/main.tsx`

Repeat Until True

Groups

Configuration

Runs once, checks condition at exit and repeats until true.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use `parent_group_id` for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

Field / key	Type / required	Details
condition condition	expression / setting Not marked required by editor	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.
maxIterations maxIterations	expression / setting Not marked required by editor	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. Default: "\${properties.advanced.groupMaxIterations}"
indexVariable indexVariable	expression / setting Not marked required by editor	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. Default: "index"

Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${input.enabled} == true
```

Output

Member activities publish their normal outputs. Groups do not have a separate universal output schema.

Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: `backend/app/runtime.py`; `frontend/src/main.tsx`

Repeat on Error Until True

Groups

Configuration

Retries the whole group after an unhandled fault while stopCondition is false. A true stop condition or exhausted retry count propagates the fault.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use parent_group_id for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

Field / key	Type / required	Details
stopCondition stopCondition	expression / setting Not marked required by editor	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.
retryCount retryCount	expression / setting Not marked required by editor	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. Default: "\${properties.advanced.retryCount}"
retryIntervalSeconds retryIntervalSeconds	expression / setting Not marked required by editor	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. Default: "\${properties.advanced.retryIntervalSeconds}"
indexVariable indexVariable	expression / setting Not marked required by editor	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed. Default: "index"

Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${context.error}  
${properties.advanced.retryCount}  
${properties.advanced.retryIntervalSeconds}
```

Output

Member activities publish their normal outputs. Groups do not have a separate universal output schema.

Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: backend/app/runtime.py; frontend/src/main.tsx

Scope

Groups

Configuration

Runs the enclosed graph once. Nested groups retain their own semantics.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use `parent_group_id` for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

No additional fields declared for this section.

Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${input.enabled} == true
```

Output

Member activities publish their normal outputs. Groups do not have a separate universal output schema.

Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: `backend/app/runtime.py`; `frontend/src/main.tsx`

None

Groups

Configuration

Runs the enclosed graph once; this is an execution boundary, not a separate transaction.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use `parent_group_id` for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

No additional fields declared for this section.

Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${input.enabled} == true
```

Output

Member activities publish their normal outputs. Groups do not have a separate universal output schema.

Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: `backend/app/runtime.py`; `frontend/src/main.tsx`

Critical Section

Groups

Configuration

Holds a runtime-local named lock until completion, failure or cancellation. This is not a distributed lock across machines or separate runtime processes.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use `parent_group_id` for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

Field / key	Type / required	Details
lockName lockName	expression / setting Not marked required by editor	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.

Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${input.enabled} == true
```

Output

Member activities publish their normal outputs. Groups do not have a separate universal output schema.

Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: `backend/app/runtime.py`; `frontend/src/main.tsx`

JDBC Transaction

Groups

Configuration

Uses one database connection for matching JDBC operations, commits on successful exit and rolls back on an escaping fault or cancellation. External non-JDBC side effects are not rolled back.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use `parent_group_id` for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

Field / key	Type / required	Details
<code>resourceId</code> <code>resourceId</code>	expression / setting Not marked required by editor	See the group behavior and Data-driven values below. Only fields applicable to the selected mode are displayed.

Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${input.enabled} == true
```

Output

Member activities publish their normal outputs. Groups do not have a separate universal output schema.

Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: `backend/app/runtime.py`; `frontend/src/main.tsx`

Pick First

Groups

Configuration

Chooses the first eligible transition in graph order. This is first-match routing, not concurrent event racing or fastest-response selection.

Set Group name, Parent group and direct activity membership. A member has one direct owner; use `parent_group_id` for nesting. Normal groups require a connected single-entry, single-exit graph. Pick First has specialized branch selection.

No additional fields declared for this section.

Input

Conditions, collections, limits and retry intervals accept mapped earlier output, initial task input, properties, functions or constants. While/Repeat can inspect prior-iteration output. An initial condition cannot depend on output that has never executed.

```
${input.enabled} == true
```

Output

Member activities publish their normal outputs. Groups do not have a separate universal output schema.

Advanced

Iteration limits and retry counts provide runaway protection. Enclosing retry policies can repeat external writes. Debug uses the group scheduler and exposes group iteration context; task archives preserve membership and nesting.

Errors

Missing members, cyclic parents, invalid expressions and invalid boundaries fail validation. Starter/event, End and Catch are task boundaries, not ordinary direct members. Parallel fan-out in non-Pick-First groups remains explicitly restricted. Escaping exceptions unwind group resources before propagating to task or caller handlers.

Reference source: `backend/app/runtime.py`; `frontend/src/main.tsx`

concat

Functions / String

Overview

Concatenate two or more values.

Signature

```
concat(value, ...values)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
concat($value, "")
```

Output

Concatenate two or more values.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

trim

Functions / String

Overview

Remove leading and trailing whitespace.

Signature

```
trim(value)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
trim($value)
```

Output

Remove leading and trailing whitespace.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

normalizeSpace

Functions / String

Overview

Trim and collapse repeated whitespace.

Signature

```
normalizeSpace(value)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
normalizeSpace($value)
```

Output

Trim and collapse repeated whitespace.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

upperCase

Functions / String

Overview

Convert text to uppercase.

Signature

```
upperCase(value)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
upperCase($value)
```

Output

Convert text to uppercase.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

lowerCase

Functions / String

Overview

Convert text to lowercase.

Signature

```
lowerCase(value)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
lowerCase($value)
```

Output

Convert text to lowercase.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

capitalize

Functions / String

Overview

Uppercase the first character.

Signature

```
capitalize(value)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
capitalize($value)
```

Output

Uppercase the first character.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

stringLength

Functions / String

Overview

Return the number of characters.

Signature

```
stringLength(value)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
stringLength($value)
```

Output

Return the number of characters.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

substring

Functions / String

Overview

Extract text using one-based positions.

Signature

```
substring(value, start, length?)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
substring($value, 1, 10)
```

Output

Extract text using one-based positions.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

substringBefore

Functions / String

Overview

Return text before the first token.

Signature

```
substringBefore(value, token)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
substringBefore($value, "-")
```

Output

Return text before the first token.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

substringAfter

Functions / String

Overview

Return text after the first token.

Signature

```
substringAfter(value, token)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
substringAfter($value, "-")
```

Output

Return text after the first token.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

replace

Functions / String

Overview

Replace text using a regular expression.

Signature

```
replace(value, pattern, replacement)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
replace($value, "old", "new")
```

Output

Replace text using a regular expression.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

matches

Functions / String

Overview

Test text against a regular expression.

Signature

```
matches(value, pattern)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
matches($value, "[A-Z]+$")
```

Output

Test text against a regular expression.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

startsWith

Functions / String

Overview

Test whether text starts with a prefix.

Signature

```
startsWith(value, prefix)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
startsWith($value, "PRE")
```

Output

Test whether text starts with a prefix.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

endsWith

Functions / String

Overview

Test whether text ends with a suffix.

Signature

```
endsWith(value, suffix)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
endsWith($value, "END")
```

Output

Test whether text ends with a suffix.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

contains

Functions / String

Overview

Test whether text contains a token.

Signature

```
contains(value, token)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
contains($value, "token")
```

Output

Test whether text contains a token.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

compare

Functions / String

Overview

Compare two strings and return -1, 0, or 1.

Signature

```
compare(value, other)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
compare($value, "other")
```

Output

Compare two strings and return -1, 0, or 1.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

translate

Functions / String

Overview

Translate characters by position.

Signature

```
translate(value, from, to)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
translate($value, "abc", "ABC")
```

Output

Translate characters by position.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

padLeft

Functions / String

Overview

Left-pad text to a fixed width.

Signature

```
padLeft(value, width, fill?)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
padLeft($value, 10, "0")
```

Output

Left-pad text to a fixed width.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

padRight

Functions / String

Overview

Right-pad text to a fixed width.

Signature

```
padRight(value, width, fill?)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
padRight($value, 10, " ")
```

Output

Right-pad text to a fixed width.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

tokenize

Functions / String

Overview

Split text using a regular expression.

Signature

```
tokenize(value, pattern)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
tokenize($value, ",")
```

Output

Split text using a regular expression.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

currentDate

Functions / Date & Time

Overview

Current UTC date in ISO format.

Signature

```
currentDate()
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
currentDate()
```

Output

Current UTC date in ISO format.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

currentTime

Functions / Date & Time

Overview

Current UTC time in ISO format.

Signature

```
currentTime()
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
currentTime()
```

Output

Current UTC time in ISO format.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

currentDateTime

Functions / Date & Time

Overview

Current UTC timestamp in ISO format.

Signature

```
currentDateTime()
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
currentDateTime()
```

Output

Current UTC timestamp in ISO format.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

parseDate

Functions / Date & Time

Overview

Parse an ISO date or timestamp.

Signature

```
parseDate(value)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
parseDate($value)
```

Output

Parse an ISO date or timestamp.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

formatDate

Functions / Date & Time

Overview

Format a date with yyyy/MM/dd-style patterns.

Signature

```
formatDate(value, pattern)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
formatDate($value, "yyyy-MM-dd")
```

Output

Format a date with yyyy/MM/dd-style patterns.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

formatDateTime

Functions / Date & Time

Overview

Format a timestamp.

Signature

```
formatDateTime(value, pattern)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
formatDateTime($value, "yyyy-MM-dd HH:mm:ss")
```

Output

Format a timestamp.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

addDays

Functions / Date & Time

Overview

Add days to a date or timestamp.

Signature

```
addDays(value, days)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
addDays($value, 1)
```

Output

Add days to a date or timestamp.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

addMonths

Functions / Date & Time

Overview

Add calendar months safely.

Signature

```
addMonths(value, months)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
addMonths($value, 1)
```

Output

Add calendar months safely.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

addHours

Functions / Date & Time

Overview

Add hours to a timestamp.

Signature

```
addHours(value, hours)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
addHours($value, 1)
```

Output

Add hours to a timestamp.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

addMinutes

Functions / Date & Time

Overview

Add minutes to a timestamp.

Signature

```
addMinutes(value, minutes)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
addMinutes($value, 30)
```

Output

Add minutes to a timestamp.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

addSeconds

Functions / Date & Time

Overview

Add seconds to a timestamp.

Signature

```
addSeconds(value, seconds)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
addSeconds($value, 30)
```

Output

Add seconds to a timestamp.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

dateDifference

Functions / Date & Time

Overview

Return the difference in days.

Signature

```
dateDifference(from, to)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
dateDifference($value, currentDateTime())
```

Output

Return the difference in days.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

year

Functions / Date & Time

Overview

Extract the year component.

Signature

```
year(value)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
year($value)
```

Output

Extract the year component.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

month

Functions / Date & Time

Overview

Extract the month component.

Signature

```
month(value)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
month($value)
```

Output

Extract the month component.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

day

Functions / Date & Time

Overview

Extract the day component.

Signature

```
day(value)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
day($value)
```

Output

Extract the day component.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

hour

Functions / Date & Time

Overview

Extract the hour component.

Signature

```
hour(value)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
hour($value)
```

Output

Extract the hour component.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

minute

Functions / Date & Time

Overview

Extract the minute component.

Signature

```
minute(value)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
minute($value)
```

Output

Extract the minute component.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

second

Functions / Date & Time

Overview

Extract the second component.

Signature

```
second(value)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
second($value)
```

Output

Extract the second component.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

timezoneFromDateTime

Functions / Date & Time

Overview

Return the timestamp timezone offset.

Signature

```
timezoneFromDateTime(value)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
timezoneFromDateTime($value)
```

Output

Return the timestamp timezone offset.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

number

Functions / Number

Overview

Convert a value to a number.

Signature

```
number(value)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
number($value)
```

Output

Convert a value to a number.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

integer

Functions / Number

Overview

Convert a value to an integer.

Signature

```
integer(value)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
integer($value)
```

Output

Convert a value to an integer.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

round

Functions / Number

Overview

Round to an optional decimal precision.

Signature

```
round(value, precision?)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
round($value, 2)
```

Output

Round to an optional decimal precision.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

roundHalfToEven

Functions / Number

Overview

Banker's rounding for financial values.

Signature

```
roundHalfToEven(value, precision?)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
roundHalfToEven($value, 2)
```

Output

Banker's rounding for financial values.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

floor

Functions / Number

Overview

Round down to an integer.

Signature

```
floor(value)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
floor($value)
```

Output

Round down to an integer.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

ceiling

Functions / Number

Overview

Round up to an integer.

Signature

```
ceiling(value)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
ceiling($value)
```

Output

Round up to an integer.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

abs

Functions / Number

Overview

Return the absolute value.

Signature

```
abs(value)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
abs($value)
```

Output

Return the absolute value.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

sqrt

Functions / Number

Overview

Return the square root.

Signature

```
sqrt(value)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
sqrt($value)
```

Output

Return the square root.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

power

Functions / Number

Overview

Raise a value to a power.

Signature

```
power(value, exponent)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
power($value, 2)
```

Output

Raise a value to a power.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

modulo

Functions / Number

Overview

Return the division remainder.

Signature

```
modulo(value, divisor)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
modulo($value, 2)
```

Output

Return the division remainder.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

clamp

Functions / Number

Overview

Constrain a number to a range.

Signature

```
clamp(value, minimum, maximum)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
clamp($value, 0, 100)
```

Output

Constrain a number to a range.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

min

Functions / Number

Overview

Min collection values.

Signature

```
min(values)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
min($value)
```

Output

Min collection values.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

max

Functions / Number

Overview

Max collection values.

Signature

```
max(values)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
max($value)
```

Output

Max collection values.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

sum

Functions / Number

Overview

Sum collection values.

Signature

```
sum(values)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
sum($value)
```

Output

Sum collection values.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

average

Functions / Number

Overview

Average collection values.

Signature

```
average(values)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
average($value)
```

Output

Average collection values.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

count

Functions / Number

Overview

Count collection values.

Signature

```
count(values)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
count($value)
```

Output

Count collection values.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

distinctValues

Functions / Collection

Overview

Remove duplicate values while preserving order.

Signature

```
distinctValues(values)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
distinctValues($value)
```

Output

Remove duplicate values while preserving order.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

sort

Functions / Collection

Overview

Sort collection values.

Signature

```
sort(values, descending?)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
sort($value)
```

Output

Sort collection values.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

reverse

Functions / Collection

Overview

Reverse a collection or string.

Signature

```
reverse(values)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
reverse($value)
```

Output

Reverse a collection or string.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

first

Functions / Collection

Overview

Return the first value.

Signature

```
first(values)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
first($value)
```

Output

Return the first value.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

last

Functions / Collection

Overview

Return the last value.

Signature

```
last(values)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
last($value)
```

Output

Return the last value.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

indexOf

Functions / Collection

Overview

Return the zero-based position of an item.

Signature

```
indexOf(values, item)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
indexOf($value, "item")
```

Output

Return the zero-based position of an item.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

subsequence

Functions / Collection

Overview

Return part of a collection.

Signature

```
subsequence(values, start, end?)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
subsequence($value, 0, 10)
```

Output

Return part of a collection.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

flatten

Functions / Collection

Overview

Flatten one level of nested arrays.

Signature

```
flatten(values)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
flatten($value)
```

Output

Flatten one level of nested arrays.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

join

Functions / Collection

Overview

Join collection values into text.

Signature

```
join(values, separator)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
join($value, ",")
```

Output

Join collection values into text.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

split

Functions / Collection

Overview

Split text into a collection.

Signature

```
split(value, pattern)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
split($value, ",")
```

Output

Split text into a collection.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

deepEqual

Functions / Collection

Overview

Compare nested values structurally.

Signature

```
deepEqual(value, other)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
deepEqual($value, $input)
```

Output

Compare nested values structurally.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

string

Functions / Conversion

Overview

Convert a value to text.

Signature

```
string(value)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
string($value)
```

Output

Convert a value to text.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

boolean

Functions / Conversion

Overview

Convert common true/false representations.

Signature

```
boolean(value)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
boolean($value)
```

Output

Convert common true/false representations.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

jsonParse

Functions / Conversion

Overview

Parse JSON text.

Signature

```
jsonParse(value)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
jsonParse($value)
```

Output

Parse JSON text.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

jsonRender

Functions / Conversion

Overview

Serialize a value as JSON.

Signature

```
jsonRender(value)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
jsonRender($value)
```

Output

Serialize a value as JSON.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

render-xml

Functions / Conversion

Overview

Convert an XML object (including a parsed XML value) into an XML string. Use `@name` for attributes and `#text` for element text.

Signature

```
render-xml(value, rootName?, pretty?)
```

Input / example

Insertion template: replace `$value` and other placeholders with data expressions or typed constants for your mapping context.

```
render-xml($value)
```

Output

Convert an XML object (including a parsed XML value) into an XML string. Use `@name` for attributes and `#text` for element text.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: `frontend/src/mapper-functions.ts`

coalesce

Functions / General

Overview

Return the first non-null/non-empty value.

Signature

```
coalesce(value, ...fallbacks)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
coalesce($value, "fallback")
```

Output

Return the first non-null/non-empty value.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

default

Functions / General

Overview

Apply a fallback to null or empty values.

Signature

```
default(value, fallback)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
default($value, "fallback")
```

Output

Apply a fallback to null or empty values.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

exists

Functions / General

Overview

Test whether a value exists.

Signature

```
exists(value)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
exists($value)
```

Output

Test whether a value exists.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

empty

Functions / General

Overview

Test whether a value is empty.

Signature

```
empty(value)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
empty($value)
```

Output

Test whether a value is empty.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

not

Functions / General

Overview

Negate a boolean value.

Signature

```
not(value)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
not($value)
```

Output

Negate a boolean value.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

ifThenElse

Functions / General

Overview

Choose between two values.

Signature

```
ifThenElse(condition, trueValue, falseValue)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
ifThenElse($value, "yes", "no")
```

Output

Choose between two values.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

uuid

Functions / General

Overview

Generate a UUID.

Signature

```
uuid()
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
uuid()
```

Output

Generate a UUID.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

base64Encode

Functions / Encoding

Overview

Encode UTF-8 text as Base64.

Signature

```
base64Encode(value)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
base64Encode($value)
```

Output

Encode UTF-8 text as Base64.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

base64Decode

Functions / Encoding

Overview

Decode Base64 into UTF-8 text.

Signature

```
base64Decode(value)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
base64Decode($value)
```

Output

Decode Base64 into UTF-8 text.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

hexEncode

Functions / Encoding

Overview

Encode UTF-8 text as hexadecimal.

Signature

```
hexEncode(value)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
hexEncode($value)
```

Output

Encode UTF-8 text as hexadecimal.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

hexDecode

Functions / Encoding

Overview

Decode hexadecimal into UTF-8 text.

Signature

```
hexDecode(value)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
hexDecode($value)
```

Output

Decode hexadecimal into UTF-8 text.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

urlEncode

Functions / Encoding

Overview

Percent-encode text for a URL.

Signature

```
ur lEncode( va lue )
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
ur lEncode( $va lue )
```

Output

Percent-encode text for a URL.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

urlDecode

Functions / Encoding

Overview

Decode URL-encoded text.

Signature

```
ur lDecode( va lue )
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
ur lDecode( $va lue )
```

Output

Decode URL-encoded text.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

hash

Functions / Encoding

Overview

Hash text with SHA-256 or another supported algorithm.

Signature

```
hash(value, algorithm?)
```

Input / example

Insertion template: replace \$value and other placeholders with data expressions or typed constants for your mapping context.

```
hash($value, "sha256")
```

Output

Hash text with SHA-256 or another supported algorithm.

Errors and usage

Use Map & Test with representative values. Invalid types, missing arguments or malformed patterns can fail evaluation. Optional arguments use the function implementation defaults; the signature is the authoritative editor insertion guide.

Reference source: [frontend/src/mapper-functions.ts](#)

JDBC shared connection

Shared connections

Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

Field / key	Type / required	Details
Database driver	select Not marked required by editor	Choices: sqlite, postgresql, mysql, mariadb, sqlserver, oracle, db2, snowflake, databricks Default: "postgresql"
Connection runtime connectionMode	select Not marked required by editor	Choices: jdbc, python Default: "python" Conditional field: (config) => ["sqlserver", "mssql", "oracle"].includes(config.driver)
JDBC driver JAR directory driverDirectory	text Not marked required by editor	Default: "\${properties.connections.jdbc.driverDirectory}" Conditional field: (config) => ["sqlserver", "mssql", "oracle"].includes(config.driver) && config.connectionMode !== "python" Editor hint: Blank uses the MINA driver directory
JDBC driver class (blank = vendor default) driverClass	text Not marked required by editor	Default: "\${properties.connections.jdbc.driverClass}" Conditional field: (config) => ["sqlserver", "mssql", "oracle"].includes(config.driver) && config.connectionMode !== "python"
JDBC URL url	text Not marked required by editor	Default: "\${properties.connections.jdbc.url}"
Host host	text Not marked required by editor	Default: "\${properties.connections.jdbc.host}"
Port port	text Not marked required by editor	Default: "\${properties.connections.jdbc.port}"
Database name database	text Not marked required by editor	Default: "\${properties.connections.jdbc.database}"
Schema schema	text Not marked required by editor	Default: "\${properties.connections.jdbc.schema}"
Username username	text Not marked required by editor	Default: "\${properties.connections.jdbc.username}"
Password password	password Not marked required by editor	Default: "\${properties.connections.jdbc.password}"
SQL Server ODBC driver (legacy Python mode) odbcDriver	text Not marked required by editor	Default: "\${properties.connections.jdbc.odbcDriver}" Conditional field: (config) => ["sqlserver", "mssql"].includes(config.driver) && config.connectionMode === "python"

Field / key	Type / required	Details
SQL Server authentication authentication	select Not marked required by editor	Choices: SQL Server Authentication, Windows Integrated Default: "SQL Server Authentication" Conditional field: (config) => ["sqlserver", "mssql"].includes(config.driver)
Encrypt SQL Server connection encrypt	select Not marked required by editor	Choices: true, false Default: "true" Conditional field: (config) => ["sqlserver", "mssql"].includes(config.driver)
Trust SQL Server certificate trustServerCertificate	select Not marked required by editor	Choices: false, true Default: "false" Conditional field: (config) => ["sqlserver", "mssql"].includes(config.driver)
Oracle service name serviceName	text Not marked required by editor	Default: "\${properties.connections.jdbc.serviceName}" Conditional field: (config) => config.driver === "oracle"
Oracle SID (when service name is not used) sid	text Not marked required by editor	Default: "\${properties.connections.jdbc.sid}" Conditional field: (config) => config.driver === "oracle"
MySQL/MariaDB CA certificate file sslCaFile	text Not marked required by editor	Default: "\${properties.connections.jdbc.sslCaFile}" Conditional field: (config) => ["mysql", "mariadb"].includes(config.driver)
Databricks server hostname serverHostname	text Not marked required by editor	Default: "\${properties.connections.jdbc.serverHostname}" Conditional field: (config) => config.driver === "databricks"
Databricks HTTP path httpPath	text Not marked required by editor	Default: "\${properties.connections.jdbc.httpPath}" Conditional field: (config) => config.driver === "databricks"
Databricks authentication authentication	select Not marked required by editor	Choices: Personal Access Token, OAuth M2M, OAuth U2M Default: "SQL Server Authentication" Conditional field: (config) => config.driver === "databricks"
Databricks access token accessToken	password Not marked required by editor	Default: "\${properties.connections.jdbc.accessToken}" Conditional field: (config) => config.driver === "databricks" && config.authentication === "Personal Access Token"
Databricks OAuth client ID clientId	text Not marked required by editor	Default: "\${properties.connections.jdbc.clientId}" Conditional field: (config) => config.driver === "databricks" && config.authentication === "OAuth M2M"
Databricks OAuth client secret clientSecret	password Not marked required by editor	Default: "\${properties.connections.jdbc.clientSecret}" Conditional field: (config) => config.driver === "databricks" && config.authentication === "OAuth M2M"
Databricks catalog catalog	text Not marked required by editor	Default: "\${properties.connections.jdbc.catalog}" Conditional field: (config) => config.driver === "databricks"
Databricks CloudFetch useCloudFetch	select Not marked required by editor	Choices: true, false Default: "\${properties.connections.jdbc.useCloudFetch}" Conditional field: (config) => config.driver === "databricks"
Connection timeout (seconds) timeoutSeconds	text Not marked required by editor	Default: "\${properties.connections.jdbc.timeoutSeconds}"
Minimum pool size minimumPoolSize	text Not marked required by editor	Default: "\${properties.connections.jdbc.minimumPoolSize}"

Field / key	Type / required	Details
Maximum pool size maximumPoolSize	text Not marked required by editor	Default: "\${properties.connections.jdbc.maximumPoolSize}"

Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Authentication and mandatory fields depend on the selected provider/mode; validate in Studio and on the target runtime.

Reference source: `frontend/src/main.tsx`

SNOWFLAKE shared connection

Shared connections

Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

Field / key	Type / required	Details
Runtime mode mode	select Not marked required by editor	Choices: external, mock Default: "external"
Authentication type authenticationType	select Not marked required by editor	Choices: Username/Password, Federated Authentication and SSO, OAuth, Key Pair Authentication Default: "Username/Password"
Authentication provider provider	select Not marked required by editor	Choices: Snowflake, Okta Default: "Snowflake"
Account [account.region.platform] account	text Not marked required by editor	Default: "\${properties.connections.snowflake.account}"
Snowflake username username	text Not marked required by editor	Default: "\${properties.connections.snowflake.username}"
Snowflake password password	password Not marked required by editor	Default: "\${properties.connections.snowflake.password}" Conditional field: (config) => config.authenticationType === "Username/Password"
Okta token endpoint oktaTokenEndpoint	text Not marked required by editor	Default: "\${properties.connections.snowflake.oktaTokenEndpoint}" Conditional field: (config) => config.authenticationType === "OAuth" && config.provider === "Okta"
Okta username oktaUsername	text Not marked required by editor	Default: "\${properties.connections.snowflake.oktaUsername}" Conditional field: (config) => config.authenticationType === "Federated Authentication and SSO" && config.provider === "Okta"
Okta password oktaPassword	password Not marked required by editor	Default: "\${properties.connections.snowflake.oktaPassword}" Conditional field: (config) => config.authenticationType === "Federated Authentication and SSO" && config.provider === "Okta"
Okta endpoint URL oktaEndpointUrl	text Not marked required by editor	Default: "\${properties.connections.snowflake.oktaEndpointUrl}" Conditional field: (config) => config.authenticationType === "Federated Authentication and SSO" && config.provider === "Okta"
OAuth client ID clientId	text Not marked required by editor	Default: "\${properties.connections.snowflake.clientId}" Conditional field: (config) => config.authenticationType === "OAuth"
OAuth client secret clientSecret	password Not marked required by editor	Default: "\${properties.connections.snowflake.clientSecret}" Conditional field: (config) => config.authenticationType === "OAuth"
OAuth scope [session:role:<role>] scope	text Not marked required by editor	Default: "\${properties.connections.snowflake.scope}" Conditional field: (config) => config.authenticationType === "OAuth"

Field / key	Type / required	Details
Authorization code authorizationCode	password Not marked required by editor	Default: "\${properties.connections.snowflake.authorizationCode}" Conditional field: (config) => config.authenticationType === "OAuth" && config.provider === "Snowflake"
OAuth redirect URI redirectUri	text Not marked required by editor	Default: "\${properties.connections.snowflake.redirectUri}" Conditional field: (config) => config.authenticationType === "OAuth" && config.provider === "Snowflake"
OAuth access token accessToken	password Not marked required by editor	Default: "\${properties.connections.snowflake.accessToken}" Conditional field: (config) => config.authenticationType === "OAuth"
Private key file privateKeyFile	text Not marked required by editor	Default: "\${properties.connections.snowflake.privateKeyFile}" Conditional field: (config) => config.authenticationType === "Key Pair Authentication"
Private key passphrase privateKeyPassphrase	password Not marked required by editor	Default: "\${properties.connections.snowflake.privateKeyPassphrase}" Conditional field: (config) => config.authenticationType === "Key Pair Authentication"
Login timeout (seconds) loginTimeoutSeconds	text Not marked required by editor	Default: "\${properties.connections.snowflake.loginTimeoutSeconds}"
Warehouse warehouse	text Not marked required by editor	Default: "\${properties.connections.snowflake.warehouse}"
Default database database	text Not marked required by editor	Default: "\${properties.connections.snowflake.database}"
Default schema schema	text Not marked required by editor	Default: "\${properties.connections.snowflake.schema}"
Default role role	text Not marked required by editor	Default: "\${properties.connections.snowflake.role}"
Other properties [Name=Value;] otherProperties	text Not marked required by editor	Default: "\${properties.connections.snowflake.otherProperties}"
Minimum connections minimumConnections	text Not marked required by editor	Default: "\${properties.connections.snowflake.minimumConnections}"
Maximum connections maximumConnections	text Not marked required by editor	Default: "\${properties.connections.snowflake.maximumConnections}"
Maximum connection wait (seconds) maximumConnectionWaitSeconds	text Not marked required by editor	Default: "\${properties.connections.snowflake.maximumConnectionWaitSeconds}"
Service number of threads serviceThreads	text Not marked required by editor	Default: "\${properties.connections.snowflake.serviceThreads}"

Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Authentication and mandatory fields depend on the selected provider/mode; validate in Studio and on the target runtime.

Reference source: `frontend/src/main.tsx`

AMQP shared connection

Shared connections

Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

Field / key	Type / required	Details
Broker type brokerType	select Not marked required by editor	Choices: Qpid-1-0, RabbitMQ, ActiveMQ-1-0, AzureSB-1-0, AMQ-1-0, ActiveMQ-Artemis-1-0 Default: "RabbitMQ"
AMQP version amqpVersion	select Not marked required by editor	Choices: AMQP-0-9-1, AMQP-1-0 Default: "AMQP-0-9-1" Conditional field: (config) => config.brokerType === "RabbitMQ"
Host:Port (comma-separated for failover) hostPort	text Required	Default: "\${properties.connections.amqp.hostPort}" Conditional field: (config) => config.brokerType !== "AzureSB-1-0"
Virtual host virtualHost	text Not marked required by editor	Default: "\${properties.connections.amqp.virtualHost}" Conditional field: (config) => ["RabbitMQ", "Qpid-1-0"].includes(config.brokerType)
Username username	text Not marked required by editor	Default: "\${properties.connections.amqp.username}" Conditional field: (config) => config.brokerType !== "AzureSB-1-0"
Password password	password Not marked required by editor	Default: "\${properties.connections.amqp.password}" Conditional field: (config) => config.brokerType !== "AzureSB-1-0"
Client ID clientId	text Not marked required by editor	Default: "\${properties.connections.amqp.clientId}" Conditional field: (config) => !["RabbitMQ", "AzureSB-1-0"].includes(config.brokerType)
Azure authentication authenticationType	select Not marked required by editor	Choices: SAS, OAuth, ManagedIdentity Default: "SAS" Conditional field: (config) => config.brokerType === "AzureSB-1-0"
Azure Service Bus endpoint / connection string connectionString	text Required	Default: "\${properties.connections.amqp.connectionString}" Conditional field: (config) => config.brokerType === "AzureSB-1-0"
Azure tenant ID tenantId	text Required	Default: "\${properties.connections.amqp.tenantId}" Conditional field: (config) => config.brokerType === "AzureSB-1-0" && config.authenticationType === "OAuth"
Azure application/client ID azureClientId	text Required	Default: "\${properties.connections.amqp.azureClientId}" Conditional field: (config) => config.brokerType === "AzureSB-1-0" && ["OAuth", "ManagedIdentity"].includes(config.authenticationType)
Azure client secret clientSecret	password Required	Default: "\${properties.connections.amqp.clientSecret}" Conditional field: (config) => config.brokerType === "AzureSB-1-0" && config.authenticationType === "OAuth"
Shared access key name sharedAccessKeyName	text Not marked required by editor	Default: "\${properties.connections.amqp.sharedAccessKeyName}" Conditional field: (config) => config.brokerType === "AzureSB-1-0" && config.authenticationType === "SAS"

Field / key	Type / required	Details
Shared access key sharedAccessKey	password Not marked required by editor	Default: "\${properties.connections.amqp.sharedAccessKey}" Conditional field: (config) => config.brokerType === "AzureSB-1-0" && config.authenticationType === "SAS"
Default entity type entityType	select Not marked required by editor	Choices: Queue, Topic Default: "\${properties.connections.amqp.entityType}" Conditional field: (config) => config.brokerType === "AzureSB-1-0"
Default entity name entityName	text Not marked required by editor	Default: "\${properties.connections.amqp.entityName}" Conditional field: (config) => config.brokerType === "AzureSB-1-0"
Default subscription entitySubscriberName	text Not marked required by editor	Default: "\${properties.connections.amqp.entitySubscriberName}" Conditional field: (config) => config.brokerType === "AzureSB-1-0" && config.entityType === "Topic"
Connection timeout (msec) connectionTimeoutMsec	text Not marked required by editor	Default: "\${properties.connections.amqp.connectionTimeoutMsec}"
RabbitMQ session count (1-20) sessionCount	text Not marked required by editor	Default: "\${properties.connections.amqp.sessionCount}" Conditional field: (config) => config.brokerType === "RabbitMQ"
Idle timeout (msec) idleTimeoutMsec	text Not marked required by editor	Default: "\${properties.connections.amqp.idleTimeoutMsec}" Conditional field: (config) => config.brokerType === "RabbitMQ" && config.amqpVersion === "AMQP-1-0"
Connection recovery connectionRecovery	select Not marked required by editor	Choices: true, false Default: "true"
Retry interval (msec) retryIntervalMsec	text Not marked required by editor	Default: "\${properties.connections.amqp.retryIntervalMsec}" Conditional field: (config) => config.brokerType !== "RabbitMQ"
Retry attempts retryAttempts	text Not marked required by editor	Default: "\${properties.connections.amqp.retryAttempts}" Conditional field: (config) => config.brokerType !== "RabbitMQ"
RabbitMQ recovery interval (msec) networkRecoveryIntervalMsec	text Not marked required by editor	Default: "\${properties.connections.amqp.networkRecoveryIntervalMsec}" Conditional field: (config) => config.brokerType === "RabbitMQ"
SSL confidentiality sslEnabled	select Not marked required by editor	Choices: false, true Default: "false"
Trusted CA file caFile	text Not marked required by editor	Default: "\${properties.connections.amqp.caFile}" Conditional field: (config) => String(config.sslEnabled) === "true"
Client certificate clientCertificateFile	text Not marked required by editor	Default: "\${properties.connections.amqp.clientCertificateFile}" Conditional field: (config) => String(config.sslEnabled) === "true"
Client private key clientKeyFile	text Not marked required by editor	Default: "\${properties.connections.amqp.clientKeyFile}" Conditional field: (config) => String(config.sslEnabled) === "true"
Private key password clientKeyPassword	password Not marked required by editor	Default: "\${properties.connections.amqp.clientKeyPassword}" Conditional field: (config) => String(config.sslEnabled) === "true"

Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Authentication and mandatory fields depend on the selected provider/mode; validate in Studio and on the target runtime.

Reference source: `frontend/src/main.tsx`

HTTP shared connection

Shared connections

Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

Field / key	Type / required	Details
Connector mode connectorMode	select Not marked required by editor	Choices: server, client, both Default: "both"
Protocol scheme	select Not marked required by editor	Choices: http, https Default: "http"
Listener host host	text Not marked required by editor	Default: "\${properties.connections.http.host}"
Listener port port	text Not marked required by editor	Default: "\${properties.connections.http.port}"
Listener base path basePath	text Not marked required by editor	Default: "\${properties.connections.http.basePath}"
Outbound base URL baseUrl	text Not marked required by editor	Default: "\${properties.connections.http.baseUrl}"
Authentication authentication	select Not marked required by editor	Choices: None, Basic, Bearer, Certificate Default: "None"
Basic-auth username username	text Not marked required by editor	Default: "\${properties.connections.http.username}"
Basic-auth password password	password Not marked required by editor	Default: "\${properties.connections.http.password}"
Bearer token bearerToken	password Not marked required by editor	Default: "\${properties.connections.http.bearerToken}"
Enable HTTPS / SSL tlsEnabled	select Not marked required by editor	Choices: false, true Default: "false"
Server certificate file certificateFile	text Not marked required by editor	Default: "\${properties.connections.http.certificateFile}"
Server private key file privateKeyFile	text Not marked required by editor	Default: "\${properties.connections.http.privateKeyFile}"

Field / key	Type / required	Details
Private key password privateKeyPassword	password Not marked required by editor	Default: "\${properties.connections.http.privateKeyPassword}"
Trusted CA file certificateAuthorityFile	text Not marked required by editor	Default: "\${properties.connections.http.certificateAuthorityFile}"
Client certificate authentication clientAuthentication	select Not marked required by editor	Choices: none, optional, required Default: "none"
Minimum TLS version tlsVersion	select Not marked required by editor	Choices: TLSv1.2, TLSv1.3 Default: "TLSv1.2"
Connect timeout (seconds) connectTimeoutSeconds	text Not marked required by editor	Default: "\${properties.connections.http.connectTimeoutSeconds}"
Read timeout (seconds) timeoutSeconds	text Not marked required by editor	Default: "\${properties.connections.http.timeoutSeconds}"
Proxy host proxyHost	text Not marked required by editor	Default: "\${properties.connections.http.proxyHost}"
Proxy port proxyPort	text Not marked required by editor	Default: "\${properties.connections.http.proxyPort}"
Verify outbound TLS verifyTls	select Not marked required by editor	Choices: true, false Default: "true"

Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Authentication and mandatory fields depend on the selected provider/mode; validate in Studio and on the target runtime.

Reference source: `frontend/src/main.tsx`

FTP shared connection

Shared connections

Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

Field / key	Type / required	Details
Host host	text Not marked required by editor	Default: "\${properties.connections.ftp.host}"
Port port	text Not marked required by editor	Default: "\${properties.connections.ftp.port}"
Username username	text Not marked required by editor	Default: "\${properties.connections.ftp.username}"
Password password	password Not marked required by editor	Default: "\${properties.connections.ftp.password}"
Working directory workingDirectory	text Not marked required by editor	Default: "\${properties.connections.ftp.workingDirectory}"
Passive mode passiveMode	select Not marked required by editor	Choices: true, false Default: "\${properties.connections.ftp.passiveMode}"
Timeout (seconds) timeoutSeconds	text Not marked required by editor	Default: "\${properties.connections.ftp.timeoutSeconds}"

Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Authentication and mandatory fields depend on the selected provider/mode; validate in Studio and on the target runtime.

Reference source: frontend/src/main.tsx

SFTP shared connection

Shared connections

Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

Field / key	Type / required	Details
Host host	text Not marked required by editor	Default: "\${properties.connections.sftp.host}"
Port port	text Not marked required by editor	Default: "\${properties.connections.sftp.port}"
Username username	text Not marked required by editor	Default: "\${properties.connections.sftp.username}"
Password password	password Not marked required by editor	Default: "\${properties.connections.sftp.password}"
Working directory workingDirectory	text Not marked required by editor	Default: "\${properties.connections.sftp.workingDirectory}"
Private key file privateKeyFile	text Not marked required by editor	Default: "\${properties.connections.sftp.privateKeyFile}"
Private key passphrase privateKeyPassphrase	password Not marked required by editor	Default: "\${properties.connections.sftp.privateKeyPassphrase}"
Known hosts file knownHostsFile	text Not marked required by editor	Default: "\${properties.connections.sftp.knownHostsFile}"
Strict host-key checking strictHostKeyChecking	select Not marked required by editor	Choices: true, false Default: "\${properties.connections.sftp.strictHostKeyChecking}"
Timeout (seconds) timeoutSeconds	text Not marked required by editor	Default: "\${properties.connections.sftp.timeoutSeconds}"

Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Authentication and mandatory fields depend on the selected provider/mode; validate in Studio and on the target runtime.

Reference source: `frontend/src/main.tsx`

EMS shared connection

Shared connections

Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

Field / key	Type / required	Details
JMS connection URL serverUrl	text Required	Default: "\${properties.connections.ems.serverUrl}" Editor hint: tcp://ems-host:7222
EMS/JMS driver JAR directory driverDirectory	text Not marked required by editor	Default: "\${properties.connections.ems.driverDirectory}" Editor hint: Blank uses C:\ProgramData\MINA Studio\drivers\jms
Native connection factory class connectionFactoryClass	text Not marked required by editor	Default: [Provider-specific class supplied by the runtime; use the exact class from your driver documentation when overriding.] Editor hint: Fully qualified class name from your driver documentation
Connection factory type connectionFactoryType	select Not marked required by editor	Choices: Direct, JNDI Default: "Direct"
Messaging style messagingStyle	select Not marked required by editor	Choices: Generic, Queue/Topic Default: "\${properties.connections.ems.messagingStyle}"
Username username	text Required	Default: "\${properties.connections.ems.username}"
Password password	password Required	Default: "\${properties.connections.ems.password}"
Client ID clientId	text Not marked required by editor	Default: "\${properties.connections.ems.clientId}" Editor hint: Generated automatically when blank
JNDI connection factory connectionFactory	select Required	Choices: ConnectionFactory, QueueConnectionFactory, TopicConnectionFactory Default: "\${properties.connections.ems.connectionFactory}" Conditional field: (config) => config.connectionFactoryType === "JNDI"
Queue connection factory queueConnectionFactory	text Not marked required by editor	Default: "\${properties.connections.ems.queueConnectionFactory}" Conditional field: (config) => config.connectionFactoryType === "JNDI" && config.messagingStyle === "Queue/Topic"
Topic connection factory topicConnectionFactory	text Not marked required by editor	Default: "\${properties.connections.ems.topicConnectionFactory}" Conditional field: (config) => config.connectionFactoryType === "JNDI" && config.messagingStyle === "Queue/Topic"
JNDI context factory jndiContextFactory	text Required	Default: [Provider-specific class supplied by the runtime; use the exact class from your driver documentation when overriding.] Conditional field: (config) => config.connectionFactoryType === "JNDI"
JNDI provider URL jndiProviderUrl	text Required	Default: "\${properties.connections.ems.jndiProviderUrl}" Conditional field: (config) => config.connectionFactoryType === "JNDI"

Field / key	Type / required	Details
JNDI username jndiUsername	text Required	Default: "\${properties.connections.ems.jndiUsername}" Conditional field: (config) => config.connectionFactoryType === "JNDI"
JNDI password jndiPassword	password Required	Default: "\${properties.connections.ems.jndiPassword}" Conditional field: (config) => config.connectionFactoryType === "JNDI"
Use XA connection factory useXa	select Not marked required by editor	Choices: false, true Default: "\${properties.connections.ems.useXa}"
Use EMS unshared failover useUfo	select Not marked required by editor	Choices: false, true Default: "\${properties.connections.ems.useUfo}"
Enable SSL sslEnabled	select Not marked required by editor	Choices: false, true Default: "\${properties.connections.ems.sslEnabled}"
SSL trusted certificates sslTrustedCertificates	text Not marked required by editor	Default: "\${properties.connections.ems.sslTrustedCertificates}"
Reconnect attempts reconnectAttempts	text Not marked required by editor	Default: "\${properties.connections.ems.reconnectAttempts}"
Reconnect delay (ms) reconnectDelayMs	text Not marked required by editor	Default: "\${properties.connections.ems.reconnectDelayMs}"
Outgoing heartbeat (ms) heartbeatOutgoingMs	text Not marked required by editor	Default: "\${properties.connections.ems.heartbeatOutgoingMs}"
Incoming heartbeat (ms) heartbeatIncomingMs	text Not marked required by editor	Default: "\${properties.connections.ems.heartbeatIncomingMs}"
Connection timeout (seconds) connectionTimeoutSeconds	text Not marked required by editor	Default: 30

Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Direct connections use JMS URL, username and password. JNDI settings become required only when JNDI is enabled. Optional connection IDs are generated by the implementation.

Reference source: frontend/src/main.tsx

JMS shared connection

Shared connections

Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

Field / key	Type / required	Details
JMS provider provider	text Not marked required by editor	Default: "\${properties.connections.jms.provider}"
JMS connection URL serverUrl	text Required	Default: "\${properties.connections.jms.serverUrl}" Editor hint: tcp://jms-host:61613
JMS provider JAR directory driverDirectory	text Not marked required by editor	Default: "\${properties.connections.jms.driverDirectory}" Editor hint: Blank uses C:\ProgramData\MINA Studio\drivers\jms
Connection factory class connectionFactoryClass	text Required	Default: [Provider-specific class supplied by the runtime; use the exact class from your driver documentation when overriding.]
Connection factory type connectionFactoryType	select Not marked required by editor	Choices: Direct, JNDI Default: "Direct"
Username username	text Required	Default: "\${properties.connections.jms.username}"
Password password	password Required	Default: "\${properties.connections.jms.password}"
Client ID clientId	text Not marked required by editor	Default: "\${properties.connections.jms.clientId}" Editor hint: Generated automatically when blank
JNDI connection factory connectionFactory	text Required	Default: "\${properties.connections.jms.connectionFactory}" Conditional field: (config) => config.connectionFactoryType === "JNDI"
JNDI initial context factory jndiContextFactory	text Required	Default: [Provider-specific class supplied by the runtime; use the exact class from your driver documentation when overriding.] Conditional field: (config) => config.connectionFactoryType === "JNDI"
JNDI provider URL jndiProviderUrl	text Required	Default: "\${properties.connections.jms.jndiProviderUrl}" Conditional field: (config) => config.connectionFactoryType === "JNDI"
JNDI username jndiUsername	text Required	Default: "\${properties.connections.jms.jndiUsername}" Conditional field: (config) => config.connectionFactoryType === "JNDI"
JNDI password jndiPassword	password Required	Default: "\${properties.connections.jms.jndiPassword}" Conditional field: (config) => config.connectionFactoryType === "JNDI"
Enable SSL sslEnabled	select Not marked required by editor	Choices: false, true Default: "\${properties.connections.jms.sslEnabled}"

Field / key	Type / required	Details
Trusted certificates sslTrustedCertificates	text Not marked required by editor	Default: "\${properties.connections.jms.sslTrustedCertificates}"
Reconnect attempts reconnectAttempts	text Not marked required by editor	Default: "\${properties.connections.jms.reconnectAttempts}"
Reconnect delay (ms) reconnectDelayMs	text Not marked required by editor	Default: "\${properties.connections.jms.reconnectDelayMs}"
Connection timeout (seconds) connectionTimeoutSeconds	text Not marked required by editor	Default: 30

Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Direct connections use JMS URL, username and password. JNDI settings become required only when JNDI is enabled. Optional connection IDs are generated by the implementation.

Reference source: frontend/src/main.tsx

KAFKA shared connection

Shared connections

Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

Field / key	Type / required	Details
Bootstrap servers bootstrapServers	text Required	Default: "\${properties.connections.kafka.bootstrapServers}"
Client ID clientId	text Not marked required by editor	Default: "\${properties.connections.kafka.clientId}"
Default consumer group groupId	text Not marked required by editor	Default: "\${properties.connections.kafka.groupId}"
Security protocol securityProtocol	select Not marked required by editor	Choices: PLAINTEXT, SASL_PLAINTEXT, SASL_SSL, SSL Default: "\${properties.connections.kafka.securityProtocol}"
Authentication type authenticationType	select Not marked required by editor	Choices: None, API Key / Secret, PLAIN, SCRAM-SHA-256, SCRAM-SHA-512, GSSAPI, OAUTHBEARER Default: "\${properties.connections.kafka.authenticationType}"
SASL mechanism saslMechanism	select Not marked required by editor	Choices: PLAIN, SCRAM-SHA-256, SCRAM-SHA-512, GSSAPI, OAUTHBEARER Default: "\${properties.connections.kafka.saslMechanism}" Conditional field: (config) => String(config.securityProtocol "").includes("SASL") && config.authenticationType !== "None"
Username / API key username	text Required	Default: "\${properties.connections.kafka.username}" Conditional field: (config) => ["API Key / Secret", "PLAIN", "SCRAM-SHA-256", "SCRAM-SHA-512"].includes(config.authenticationType) && String(config.securityProtocol "").includes("SASL")
Password / API secret password	password Required	Default: "\${properties.connections.kafka.password}" Conditional field: (config) => ["API Key / Secret", "PLAIN", "SCRAM-SHA-256", "SCRAM-SHA-512"].includes(config.authenticationType) && String(config.securityProtocol "").includes("SASL")
SSL CA location sslCaLocation	text Not marked required by editor	Default: "\${properties.connections.kafka.sslCaLocation}" Conditional field: (config) => ["SSL", "SASL_SSL"].includes(config.securityProtocol)
SSL certificate location sslCertificateLocation	text Not marked required by editor	Default: "\${properties.connections.kafka.sslCertificateLocation}" Conditional field: (config) => ["SSL", "SASL_SSL"].includes(config.securityProtocol)
SSL key location sslKeyLocation	text Not marked required by editor	Default: "\${properties.connections.kafka.sslKeyLocation}" Conditional field: (config) => ["SSL", "SASL_SSL"].includes(config.securityProtocol)
SSL key password sslKeyPassword	password Not marked required by editor	Default: "\${properties.connections.kafka.sslKeyPassword}" Conditional field: (config) => ["SSL", "SASL_SSL"].includes(config.securityProtocol)
Schema Registry URL schemaRegistryUrl	text Not marked required by editor	Default: "\${properties.connections.kafka.schemaRegistryUrl}"

Field / key	Type / required	Details
Schema Registry username schemaRegistryUsername	text Not marked required by editor	Default: "\${properties.connections.kafka.schemaRegistryUsername}"
Schema Registry password schemaRegistryPassword	password Not marked required by editor	Default: "\${properties.connections.kafka.schemaRegistryPassword}"
Schema Registry vendor schemaRegistryVendor	select Not marked required by editor	Choices: Confluent, Other, Apicurio Default: "\${properties.connections.kafka.schemaRegistryVendor}"
Use Kerberos ticket cache useTicketCache	select Not marked required by editor	Choices: false, true Default: "\${properties.connections.kafka.useTicketCache}" Conditional field: (config) => config.authenticationType === "GSSAPI"
Kerberos keytab file keytabFile	text Not marked required by editor	Default: "\${properties.connections.kafka.keytabFile}" Conditional field: (config) => config.authenticationType === "GSSAPI"
Kerberos principal principalName	text Not marked required by editor	Default: "\${properties.connections.kafka.principalName}" Conditional field: (config) => config.authenticationType === "GSSAPI"
OAuth JAAS config file jaasConfigFile	text Not marked required by editor	Default: "\${properties.connections.kafka.jaasConfigFile}" Conditional field: (config) => config.authenticationType === "OAUTHBEARER"
OAuth login callback handler loginCallbackHandler	text Not marked required by editor	Default: "\${properties.connections.kafka.loginCallbackHandler}" Conditional field: (config) => config.authenticationType === "OAUTHBEARER"
Request timeout (ms) requestTimeoutMilliseconds	text Not marked required by editor	Default: "\${properties.connections.kafka.requestTimeoutMilliseconds}"
Connection timeout (ms) connectionTimeoutMilliseconds	text Not marked required by editor	Default: "\${properties.connections.kafka.connectionTimeoutMilliseconds}"
Reconnect backoff (ms) reconnectBackoffMilliseconds	text Not marked required by editor	Default: "\${properties.connections.kafka.reconnectBackoffMilliseconds}"
Retry backoff (ms) retryBackoffMilliseconds	text Not marked required by editor	Default: "\${properties.connections.kafka.retryBackoffMilliseconds}"
Advanced client properties (JSON) clientProperties	text Not marked required by editor	Default: "\${properties.connections.kafka.clientProperties}"

Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library.

See Vendor Drivers and the connector guides.

Authentication and mandatory fields depend on the selected provider/mode; validate in Studio and on the target runtime.

Reference source: `frontend/src/main.tsx`

PUBSUB shared connection

Shared connections

Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

Field / key	Type / required	Details
Authentication authenticationType	select Required	Choices: Service Account JSON, Application Default Credentials, Emulator Default: "Service Account JSON"
Service account JSON (inline) serviceAccountJson	text Required	Default: "" Conditional field: (config) => config.authenticationType === "Service Account JSON" && !String(config.credentialsFile "").trim()
Service account JSON file credentialsFile	text Required	Default: "\${properties.connections.pubsub.credentialsFile}" Conditional field: (config) => config.authenticationType === "Service Account JSON" && !String(config.serviceAccountJson "").trim() Editor hint: Environment property resolving to the complete .json file path
GCP project ID projectId	text Required	Default: "" Conditional field: (config) => config.authenticationType !== "Service Account JSON" Editor hint: Derived automatically from service-account JSON
Service endpoint endpoint	text Not marked required by editor	Default: "\${properties.connections.pubsub.endpoint}" Conditional field: (config) => config.authenticationType === "Application Default Credentials"
Corporate CA certificate file (optional) caCertificateFile	text Not marked required by editor	Default: "\${properties.connections.pubsub.caCertificateFile}" Conditional field: (config) => config.authenticationType !== "Emulator" Editor hint: Only required when TLS inspection uses an internal CA
Emulator host emulatorHost	text Required	Default: "\${properties.connections.pubsub.emulatorHost}" Conditional field: (config) => config.authenticationType === "Emulator" Editor hint: localhost:8085
Ack deadline (seconds) ackDeadlineSeconds	text Not marked required by editor	Default: "\${properties.connections.pubsub.ackDeadlineSeconds}"
Connection timeout (seconds) connectionTimeoutSeconds	text Not marked required by editor	Default: "\${properties.connections.pubsub.connectionTimeoutSeconds}"
Maximum inbound message bytes maxInboundMessageBytes	text Not marked required by editor	Default: "\${properties.connections.pubsub.maxInboundMessageBytes}"
Keep-alive time (seconds) keepAliveSeconds	text Not marked required by editor	Default: "\${properties.connections.pubsub.keepAliveSeconds}"
Publisher batch maximum messages batchMaxMessages	text Not marked required by editor	Default: 100

Field / key	Type / required	Details
Publisher batch maximum bytes batchMaxBytes	text Not marked required by editor	Default: 1048576
Publisher batch delay threshold (ms) batchDelayThresholdMilliseconds	text Not marked required by editor	Default: 10
Publisher flow-control message limit flowControlMaxMessages	text Not marked required by editor	Default: 1000
Publisher flow-control byte limit flowControlMaxBytes	text Not marked required by editor	Default: 10485760

Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Service Account JSON supplies Google authentication. Verify project/topic/subscription access and IAM permissions on the target environment.

Reference source: frontend/src/main.tsx

SAP shared connection

Shared connections

Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

Field / key	Type / required	Details
Runtime adapter mode	select Not marked required by editor	Choices: mock, external Default: "external"
SAP JCo driver directory driverDirectory	text Not marked required by editor	Default: "\${properties.connections.sap.driverDirectory}" Editor hint: Blank uses C:\ProgramData\MINA Studio\drivers\sap
JCo destination name destinationName	text Not marked required by editor	Default: "\${properties.connections.sap.destinationName}" Editor hint: mina-sap
SAP release release	select Not marked required by editor	Choices: current, 720, 730 Default: "current"
Connection type connectionType	select Not marked required by editor	Choices: dedicated, logongroup, snc, sncwithlogongroup, websocket Default: "dedicated"
Application server host applicationServerHost	text Not marked required by editor	Default: "\${properties.connections.sap.applicationServerHost}"
System number systemNumber	text Not marked required by editor	Default: "\${properties.connections.sap.systemNumber}"
Client number client	text Not marked required by editor	Default: "\${properties.connections.sap.client}"
Language language	text Not marked required by editor	Default: "\${properties.connections.sap.language}"
Username username	text Not marked required by editor	Default: "\${properties.connections.sap.username}"
Password password	password Not marked required by editor	Default: "\${properties.connections.sap.password}"
Message server host messageServerHost	text Not marked required by editor	Default: "\${properties.connections.sap.messageServerHost}"
System ID systemId	text Not marked required by editor	Default: "\${properties.connections.sap.systemId}"

Field / key	Type / required	Details
Logon group logonGroup	text Not marked required by editor	Default: "\${properties.connections.sap.logonGroup}"
SAP Router sapRouter	text Not marked required by editor	Default: "\${properties.connections.sap.sapRouter}"
SNC partner name sncPartnerName	text Not marked required by editor	Default: "\${properties.connections.sap.sncPartnerName}" Conditional field: (config) => ["snc", "sncwithlogongroup"].includes(config.connectionType)
SNC library path sncLibraryPath	text Not marked required by editor	Default: "\${properties.connections.sap.sncLibraryPath}" Conditional field: (config) => ["snc", "sncwithlogongroup"].includes(config.connectionType)
SNC own name sncMyName	text Not marked required by editor	Default: "\${properties.connections.sap.sncMyName}" Conditional field: (config) => ["snc", "sncwithlogongroup"].includes(config.connectionType)
SNC quality of protection sncQop	select Not marked required by editor	Choices: , 1, 2, 3, 8, 9 Default: "\${properties.connections.sap.sncQop}" Conditional field: (config) => ["snc", "sncwithlogongroup"].includes(config.connectionType)
Program ID (inbound) programId	text Not marked required by editor	Default: "\${properties.connections.sap.programId}"
Gateway host gatewayHost	text Not marked required by editor	Default: "\${properties.connections.sap.gatewayHost}"
Gateway service gatewayService	text Not marked required by editor	Default: "\${properties.connections.sap.gatewayService}"
Maximum connections maximumConnections	text Not marked required by editor	Default: 8
Persistent outbound JVM workers outboundWorkerCount	text Not marked required by editor	Default: 2
JCo destination pool capacity poolCapacity	text Not marked required by editor	Default: 1
JCo destination peak limit peakLimit	text Not marked required by editor	Default: 1
Idle connection expiration (ms) connectionExpirationMilliseconds	text Not marked required by editor	Default: 600000
Outbound pool idle timeout (seconds) outboundPoolIdleSeconds	text Not marked required by editor	Default: 1800
Maximum distinct outbound pools maximumOutboundPools	text Not marked required by editor	Default: 16

Field / key	Type / required	Details
Maximum buffered SAP requests maxPendingEvents	text Not marked required by editor	Default: 16
JCo bridge initial heap (MB) jvmInitialHeapMb	text Not marked required by editor	Default: 64
JCo bridge maximum heap (MB) jvmMaximumHeapMb	text Not marked required by editor	Default: 512
Startup timeout (ms) timeoutMilliseconds	text Not marked required by editor	Default: "\${properties.connections.sap.timeoutMilliseconds}"
Inbound processing/reply timeout (seconds) ackTimeoutSeconds	text Not marked required by editor	Default: 300
Maximum dynamic sessions maximumDynamicSessions	text Not marked required by editor	Default: "\${properties.connections.sap.maximumDynamicSessions}"

Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Authentication and mandatory fields depend on the selected provider/mode; validate in Studio and on the target runtime.

Reference source: frontend/src/main.tsx

SAP_TID shared connection

Shared connections

Configuration

Create a shared resource in the project Resources tree, select it on activities, and use Test connection after resolving the active environment properties. Generated IDs are managed by Studio. Conditional fields below are shown only when their mode applies.

Field / key	Type / required	Details
TID management mode	select Not marked required by editor	Choices: active, disabled Default: "active"
Transaction ID storage file storageFile	text Not marked required by editor	Default: "data/sap-tids.properties" Editor hint: Durable path shared by the deployed runtime

Properties and credentials

Browse an environment property to bind a field; inspect the resolved value in Studio. Provide real credentials locally or through deployment secrets; never put secrets in documentation, source control or AI prompts. A password-style field is a UI indication, not a guarantee that all exports are redacted.

Testing and prerequisites

A connection test checks connectivity, not full application behavior, permissions for every operation, throughput or failover. Java-based connectors require their licensed driver JARs and bridge; SAP also requires a matching platform-native library. See Vendor Drivers and the connector guides.

Authentication and mandatory fields depend on the selected provider/mode; validate in Studio and on the target runtime.

Reference source: frontend/src/main.tsx

Environment property defaults

Shared connections

Defaults

These are checked-in Studio defaults, not values read from any user project. Override per environment. Password properties need secure deployment handling.

Field / key	Type / required	Details
runtime.logDirectory runtime.logDirectory	string Not marked required by editor	Default: ""
messaging.artmasQueue messaging.artmasQueue	string Not marked required by editor	Default: "ARTMAS05.IN"
advanced.logPayload advanced.logPayload	boolean Not marked required by editor	Default: false
advanced.retryEnabled advanced.retryEnabled	boolean Not marked required by editor	Default: false
advanced.retryCount advanced.retryCount	integer Not marked required by editor	Default: 3
advanced.retryIntervalSeconds advanced.retryIntervalSeconds	integer Not marked required by editor	Default: 60
advanced.groupMaxIterations advanced.groupMaxIterations	integer Not marked required by editor	Default: 10000
connections.http.baseUrl connections.http.baseUrl	string Not marked required by editor	Default: "https://api.example.com"
connections.http.host connections.http.host	string Not marked required by editor	Default: "localhost"
connections.http.port connections.http.port	integer Not marked required by editor	Default: 8080
connections.http.basePath connections.http.basePath	string Not marked required by editor	Default: ""
connections.http.scheme connections.http.scheme	string Not marked required by editor	Default: "http"
connections.http.connectorMode connections.http.connectorMode	string Not marked required by editor	Default: "both"
connections.http.authentication connections.http.authentication	string Not marked required by editor	Default: "None"

Field / key	Type / required	Details
connections.http.bearerToken connections.http.bearerToken	password Not marked required by editor	Default: "[supply securely]"
connections.http.tlsEnabled connections.http.tlsEnabled	boolean Not marked required by editor	Default: false
connections.http.certificateFile connections.http.certificateFile	string Not marked required by editor	Default: ""
connections.http.privateKeyFile connections.http.privateKeyFile	string Not marked required by editor	Default: ""
connections.http.privateKeyPassword connections.http.privateKeyPassword	password Not marked required by editor	Default: "[supply securely]"
connections.http.certificateAuthorityFile connections.http.certificateAuthorityFile	string Not marked required by editor	Default: ""
connections.http.clientAuthentication connections.http.clientAuthentication	string Not marked required by editor	Default: "none"
connections.http.tlsVersion connections.http.tlsVersion	string Not marked required by editor	Default: "TLSv1.2"
connections.http.connectTimeoutSeconds connections.http.connectTimeoutSeconds	integer Not marked required by editor	Default: 30
connections.http.timeoutSeconds connections.http.timeoutSeconds	integer Not marked required by editor	Default: 60
connections.http.username connections.http.username	string Not marked required by editor	Default: ""
connections.http.password connections.http.password	password Not marked required by editor	Default: "[supply securely]"
connections.http.proxyHost connections.http.proxyHost	string Not marked required by editor	Default: ""
connections.http.proxyPort connections.http.proxyPort	integer Not marked required by editor	Default: 8080
connections.http.verifyTls connections.http.verifyTls	boolean Not marked required by editor	Default: true
connections.ftp.host connections.ftp.host	string Not marked required by editor	Default: "ftp.example.com"

Field / key	Type / required	Details
connections.ftp.port connections.ftp.port	integer Not marked required by editor	Default: 21
connections.ftp.username connections.ftp.username	string Not marked required by editor	Default: ""
connections.ftp.password connections.ftp.password	password Not marked required by editor	Default: "[supply securely]"
connections.ftp.workingDirectory connections.ftp.workingDirectory	string Not marked required by editor	Default: "/"
connections.ftp.passiveMode connections.ftp.passiveMode	boolean Not marked required by editor	Default: true
connections.ftp.timeoutSeconds connections.ftp.timeoutSeconds	integer Not marked required by editor	Default: 60
connections.sftp.host connections.sftp.host	string Not marked required by editor	Default: "sftp.example.com"
connections.sftp.port connections.sftp.port	integer Not marked required by editor	Default: 22
connections.sftp.username connections.sftp.username	string Not marked required by editor	Default: ""
connections.sftp.password connections.sftp.password	password Not marked required by editor	Default: "[supply securely]"
connections.sftp.workingDirectory connections.sftp.workingDirectory	string Not marked required by editor	Default: "/"
connections.sftp.privateKeyFile connections.sftp.privateKeyFile	string Not marked required by editor	Default: ""
connections.sftp.privateKeyPassphrase connections.sftp.privateKeyPassphrase	password Not marked required by editor	Default: "[supply securely]"
connections.sftp.knownHostsFile connections.sftp.knownHostsFile	string Not marked required by editor	Default: ""
connections.sftp.strictHostKeyChecking connections.sftp.strictHostKeyChecking	boolean Not marked required by editor	Default: true
connections.sftp.timeoutSeconds connections.sftp.timeoutSeconds	integer Not marked required by editor	Default: 60

Field / key	Type / required	Details
connections.jdbc.driver connections.jdbc.driver	string Not marked required by editor	Default: "postgresql"
connections.jdbc.url connections.jdbc.url	string Not marked required by editor	Default: "jdbc:postgresql://localhost:5432/integration"
connections.jdbc.host connections.jdbc.host	string Not marked required by editor	Default: "localhost"
connections.jdbc.port connections.jdbc.port	integer Not marked required by editor	Default: 5432
connections.jdbc.database connections.jdbc.database	string Not marked required by editor	Default: "integration"
connections.jdbc.schema connections.jdbc.schema	string Not marked required by editor	Default: "public"
connections.jdbc.username connections.jdbc.username	string Not marked required by editor	Default: ""
connections.jdbc.password connections.jdbc.password	password Not marked required by editor	Default: "[supply securely]"
connections.jdbc.connectionMode connections.jdbc.connectionMode	string Not marked required by editor	Default: "python"
connections.jdbc.driverDirectory connections.jdbc.driverDirectory	string Not marked required by editor	Default: ""
connections.jdbc.driverClass connections.jdbc.driverClass	string Not marked required by editor	Default: ""
connections.jdbc.odbcDriver connections.jdbc.odbcDriver	string Not marked required by editor	Default: ""
connections.jdbc.authentication connections.jdbc.authentication	string Not marked required by editor	Default: "SQL Server Authentication"
connections.jdbc.encrypt connections.jdbc.encrypt	boolean Not marked required by editor	Default: true
connections.jdbc.trustServerCertificate connections.jdbc.trustServerCertificate	boolean Not marked required by editor	Default: false
connections.jdbc.serverHostname connections.jdbc.serverHostname	string Not marked required by editor	Default: ""

Field / key	Type / required	Details
connections.jdbc.httpPath connections.jdbc.httpPath	string Not marked required by editor	Default: ""
connections.jdbc.accessToken connections.jdbc.accessToken	password Not marked required by editor	Default: "[supply securely]"
connections.jdbc.clientId connections.jdbc.clientId	string Not marked required by editor	Default: ""
connections.jdbc.clientSecret connections.jdbc.clientSecret	password Not marked required by editor	Default: "[supply securely]"
connections.jdbc.catalog connections.jdbc.catalog	string Not marked required by editor	Default: ""
connections.jdbc.useCloudFetch connections.jdbc.useCloudFetch	boolean Not marked required by editor	Default: true
connections.jdbc.serviceName connections.jdbc.serviceName	string Not marked required by editor	Default: ""
connections.jdbc.sid connections.jdbc.sid	string Not marked required by editor	Default: ""
connections.jdbc.sslCaFile connections.jdbc.sslCaFile	string Not marked required by editor	Default: ""
connections.jdbc.timeoutSeconds connections.jdbc.timeoutSeconds	integer Not marked required by editor	Default: 30
connections.jdbc.minimumPoolSize connections.jdbc.minimumPoolSize	integer Not marked required by editor	Default: 1
connections.jdbc.maximumPoolSize connections.jdbc.maximumPoolSize	integer Not marked required by editor	Default: 10
connections.snowflake.mode connections.snowflake.mode	string Not marked required by editor	Default: "external"
connections.snowflake.authenticationType connections.snowflake.authenticationType	string Not marked required by editor	Default: "Username/Password"
connections.snowflake.provider connections.snowflake.provider	string Not marked required by editor	Default: "Snowflake"
connections.snowflake.account connections.snowflake.account	string Not marked required by editor	Default: ""

Field / key	Type / required	Details
connections.snowflake.username connections.snowflake.username	string Not marked required by editor	Default: ""
connections.snowflake.password connections.snowflake.password	password Not marked required by editor	Default: "[supply securely]"
connections.snowflake.warehouse connections.snowflake.warehouse	string Not marked required by editor	Default: ""
connections.snowflake.database connections.snowflake.database	string Not marked required by editor	Default: ""
connections.snowflake.schema connections.snowflake.schema	string Not marked required by editor	Default: "PUBLIC"
connections.snowflake.role connections.snowflake.role	string Not marked required by editor	Default: ""
connections.snowflake.loginTimeoutSeconds connections.snowflake.loginTimeoutSeconds	integer Not marked required by editor	Default: 60
connections.snowflake.minimumConnections connections.snowflake.minimumConnections	integer Not marked required by editor	Default: 2
connections.snowflake.maximumConnections connections.snowflake.maximumConnections	integer Not marked required by editor	Default: 8
connections.snowflake.maximumConnectionWaitSeconds connections.snowflake.maximumConnectionWaitSeconds	integer Not marked required by editor	Default: 300
connections.snowflake.serviceThreads connections.snowflake.serviceThreads	integer Not marked required by editor	Default: 8
connections.amqp.brokerType connections.amqp.brokerType	string Not marked required by editor	Default: "RabbitMQ"
connections.amqp.amqpVersion connections.amqp.amqpVersion	string Not marked required by editor	Default: "AMQP-0-9-1"
connections.amqp.hostPort connections.amqp.hostPort	string Not marked required by editor	Default: "localhost:5672"
connections.amqp.virtualHost connections.amqp.virtualHost	string Not marked required by editor	Default: "/"

Field / key	Type / required	Details
connections.amqp.username connections.amqp.username	string Not marked required by editor	Default: "guest"
connections.amqp.password connections.amqp.password	password Not marked required by editor	Default: "[supply securely]"
connections.amqp.clientId connections.amqp.clientId	string Not marked required by editor	Default: ""
connections.amqp.authenticationType connections.amqp.authenticationType	string Not marked required by editor	Default: "SAS"
connections.amqp.connectionString connections.amqp.connectionString	password Not marked required by editor	Default: "[supply securely]"
connections.amqp.tenantId connections.amqp.tenantId	string Not marked required by editor	Default: ""
connections.amqp.azureClientId connections.amqp.azureClientId	string Not marked required by editor	Default: ""
connections.amqp.clientSecret connections.amqp.clientSecret	password Not marked required by editor	Default: "[supply securely]"
connections.amqp.sharedAccessKeyName connections.amqp.sharedAccessKeyName	string Not marked required by editor	Default: ""
connections.amqp.sharedAccessKey connections.amqp.sharedAccessKey	password Not marked required by editor	Default: "[supply securely]"
connections.amqp.entityType connections.amqp.entityType	string Not marked required by editor	Default: "Queue"
connections.amqp.entityName connections.amqp.entityName	string Not marked required by editor	Default: ""
connections.amqp.entitySubscriberName connections.amqp.entitySubscriberName	string Not marked required by editor	Default: ""
connections.amqp.connectionTimeoutMsec connections.amqp.connectionTimeoutMsec	integer Not marked required by editor	Default: 30000
connections.amqp.sessionCount connections.amqp.sessionCount	integer Not marked required by editor	Default: 1
connections.amqp.idleTimeoutMsec connections.amqp.idleTimeoutMsec	integer Not marked required by editor	Default: 0

Field / key	Type / required	Details
connections.amqp.connectionRecover y connections.amqp.connectionRecover y	boolean Not marked required by editor	Default: true
connections.amqp.retryIntervalMsec connections.amqp.retryIntervalMsec	integer Not marked required by editor	Default: 3000
connections.amqp.retryAttempts connections.amqp.retryAttempts	integer Not marked required by editor	Default: 20
connections.amqp.networkRecoveryInt ervalMsec connections.amqp.networkRecoveryInt ervalMsec	integer Not marked required by editor	Default: 5000
connections.amqp.sslEnabled connections.amqp.sslEnabled	boolean Not marked required by editor	Default: false
connections.amqp.caFile connections.amqp.caFile	string Not marked required by editor	Default: ""
connections.amqp.clientCertificateFile connections.amqp.clientCertificateFile	string Not marked required by editor	Default: ""
connections.amqp.clientKeyFile connections.amqp.clientKeyFile	string Not marked required by editor	Default: ""
connections.amqp.clientKeyPassword connections.amqp.clientKeyPassword	password Not marked required by editor	Default: "[supply securely]"
connections.ems.host connections.ems.host	string Not marked required by editor	Default: "localhost"
connections.ems.port connections.ems.port	integer Not marked required by editor	Default: 7222
connections.ems.serverUrl connections.ems.serverUrl	string Not marked required by editor	Default: "tcp://localhost:7222"
connections.ems.driverDirectory connections.ems.driverDirectory	string Not marked required by editor	Default: ""
connections.ems.connectionFactoryCl ass connections.ems.connectionFactoryCl ass	string Not marked required by editor	Default: [Provider-specific class supplied by the runtime; use the exact class from your driver documentation when overriding.]
connections.ems.connectionTimeoutS econds connections.ems.connectionTimeoutS econds	integer Not marked required by editor	Default: 30

Field / key	Type / required	Details
connections.ems.destination connections.ems.destination	string Not marked required by editor	Default: "orders"
connections.ems.sessionCount connections.ems.sessionCount	integer Not marked required by editor	Default: 1
connections.ems.flowLimit connections.ems.flowLimit	integer Not marked required by editor	Default: 0
connections.ems.receiveTimeoutMs connections.ems.receiveTimeoutMs	integer Not marked required by editor	Default: 30000
connections.ems.username connections.ems.username	string Not marked required by editor	Default: ""
connections.ems.password connections.ems.password	password Not marked required by editor	Default: "[supply securely]"
connections.ems.clientId connections.ems.clientId	string Not marked required by editor	Default: ""
connections.ems.connectionFactory connections.ems.connectionFactory	string Not marked required by editor	Default: "ConnectionFactory"
connections.ems.reconnectAttempts connections.ems.reconnectAttempts	integer Not marked required by editor	Default: 3
connections.ems.connectionFactoryType connections.ems.connectionFactoryType	string Not marked required by editor	Default: "Direct"
connections.ems.messagingStyle connections.ems.messagingStyle	string Not marked required by editor	Default: "Generic"
connections.ems.queueConnectionFactory connections.ems.queueConnectionFactory	string Not marked required by editor	Default: "QueueConnectionFactory"
connections.ems.topicConnectionFactory connections.ems.topicConnectionFactory	string Not marked required by editor	Default: "TopicConnectionFactory"
connections.ems.jndiContextFactory connections.ems.jndiContextFactory	string Not marked required by editor	Default: [Provider-specific class supplied by the runtime; use the exact class from your driver documentation when overriding.]
connections.ems.jndiProviderUrl connections.ems.jndiProviderUrl	string Not marked required by editor	Default: "tcp://localhost:7222"
connections.ems.jndiUsername connections.ems.jndiUsername	string Not marked required by editor	Default: ""

Field / key	Type / required	Details
connections.ems.jndiPassword connections.ems.jndiPassword	password Not marked required by editor	Default: "[supply securely]"
connections.ems.useXa connections.ems.useXa	boolean Not marked required by editor	Default: false
connections.ems.useUfo connections.ems.useUfo	boolean Not marked required by editor	Default: false
connections.ems.sslEnabled connections.ems.sslEnabled	boolean Not marked required by editor	Default: false
connections.ems.sslTrustedCertificates connections.ems.sslTrustedCertificates	string Not marked required by editor	Default: ""
connections.ems.reconnectDelayMs connections.ems.reconnectDelayMs	integer Not marked required by editor	Default: 5000
connections.ems.heartbeatOutgoingMs connections.ems.heartbeatOutgoingMs	integer Not marked required by editor	Default: 0
connections.ems.heartbeatIncomingMs connections.ems.heartbeatIncomingMs	integer Not marked required by editor	Default: 0
connections.jms.provider connections.jms.provider	string Not marked required by editor	Default: "Generic JMS 2.0"
connections.jms.serverUrl connections.jms.serverUrl	string Not marked required by editor	Default: "tcp://localhost:61613"
connections.jms.driverDirectory connections.jms.driverDirectory	string Not marked required by editor	Default: ""
connections.jms.connectionFactoryClasses connections.jms.connectionFactoryClasses	string Not marked required by editor	Default: ""
connections.jms.connectionTimeoutSeconds connections.jms.connectionTimeoutSeconds	integer Not marked required by editor	Default: 30
connections.jms.host connections.jms.host	string Not marked required by editor	Default: "localhost"
connections.jms.port connections.jms.port	integer Not marked required by editor	Default: 61613

Field / key	Type / required	Details
connections.jms.username connections.jms.username	string Not marked required by editor	Default: ""
connections.jms.password connections.jms.password	password Not marked required by editor	Default: "[supply securely]"
connections.jms.clientId connections.jms.clientId	string Not marked required by editor	Default: ""
connections.jms.connectionFactoryType connections.jms.connectionFactoryType	string Not marked required by editor	Default: "Direct"
connections.jms.connectionFactory connections.jms.connectionFactory	string Not marked required by editor	Default: "ConnectionFactory"
connections.jms.jndiContextFactory connections.jms.jndiContextFactory	string Not marked required by editor	Default: ""
connections.jms.jndiProviderUrl connections.jms.jndiProviderUrl	string Not marked required by editor	Default: ""
connections.jms.jndiUsername connections.jms.jndiUsername	string Not marked required by editor	Default: ""
connections.jms.jndiPassword connections.jms.jndiPassword	password Not marked required by editor	Default: "[supply securely]"
connections.jms.sslEnabled connections.jms.sslEnabled	boolean Not marked required by editor	Default: false
connections.jms.sslTrustedCertificates connections.jms.sslTrustedCertificates	string Not marked required by editor	Default: ""
connections.jms.reconnectAttempts connections.jms.reconnectAttempts	integer Not marked required by editor	Default: 3
connections.jms.reconnectDelayMs connections.jms.reconnectDelayMs	integer Not marked required by editor	Default: 5000
connections.kafka.bootstrapServers connections.kafka.bootstrapServers	string Not marked required by editor	Default: "localhost:9092"
connections.kafka.clientId connections.kafka.clientId	string Not marked required by editor	Default: ""
connections.kafka.groupId connections.kafka.groupId	string Not marked required by editor	Default: "mina"

Field / key	Type / required	Details
connections.kafka.securityProtocol connections.kafka.securityProtocol	string Not marked required by editor	Default: "PLAINTEXT"
connections.kafka.saslMechanism connections.kafka.saslMechanism	string Not marked required by editor	Default: "PLAIN"
connections.kafka.username connections.kafka.username	string Not marked required by editor	Default: ""
connections.kafka.password connections.kafka.password	password Not marked required by editor	Default: "[supply securely]"
connections.kafka.requestTimeoutMilli seconds connections.kafka.requestTimeoutMilli seconds	integer Not marked required by editor	Default: 30000
connections.kafka.connectionTimeout Milliseconds connections.kafka.connectionTimeout Milliseconds	integer Not marked required by editor	Default: 10000
connections.kafka.sslCaLocation connections.kafka.sslCaLocation	string Not marked required by editor	Default: ""
connections.kafka.sslCertificateLocatio n connections.kafka.sslCertificateLocatio n	string Not marked required by editor	Default: ""
connections.kafka.sslKeyLocation connections.kafka.sslKeyLocation	string Not marked required by editor	Default: ""
connections.kafka.sslKeyPassword connections.kafka.sslKeyPassword	password Not marked required by editor	Default: "[supply securely]"
connections.kafka.schemaRegistryUrl connections.kafka.schemaRegistryUrl	string Not marked required by editor	Default: ""
connections.kafka.schemaRegistryUse rname connections.kafka.schemaRegistryUse rname	string Not marked required by editor	Default: ""
connections.kafka.schemaRegistryPas sword connections.kafka.schemaRegistryPas sword	password Not marked required by editor	Default: "[supply securely]"
connections.kafka.reconnectBackoffMi lliseconds connections.kafka.reconnectBackoffMi lliseconds	integer Not marked required by editor	Default: 50
connections.kafka.retryBackoffMillisec onds connections.kafka.retryBackoffMillisec onds	integer Not marked required by editor	Default: 100

Field / key	Type / required	Details
connections.kafka.authenticationType connections.kafka.authenticationType	string Not marked required by editor	Default: "None"
connections.kafka.useTicketCache connections.kafka.useTicketCache	boolean Not marked required by editor	Default: false
connections.kafka.keytabFile connections.kafka.keytabFile	string Not marked required by editor	Default: ""
connections.kafka.principalName connections.kafka.principalName	string Not marked required by editor	Default: ""
connections.kafka.jaasConfigFile connections.kafka.jaasConfigFile	string Not marked required by editor	Default: ""
connections.kafka.loginCallbackHandler connections.kafka.loginCallbackHandler	string Not marked required by editor	Default: ""
connections.kafka.schemaRegistryVendor connections.kafka.schemaRegistryVendor	string Not marked required by editor	Default: "Confluent"
connections.kafka.clientProperties connections.kafka.clientProperties	json Not marked required by editor	Default: "{}"
connections.pubsub.projectId connections.pubsub.projectId	string Not marked required by editor	Default: "my-gcp-project"
connections.pubsub.authenticationType connections.pubsub.authenticationType	string Not marked required by editor	Default: "Service Account JSON"
connections.pubsub.serviceAccountJson connections.pubsub.serviceAccountJson	password Not marked required by editor	Default: "[supply securely]"
connections.pubsub.credentialsFile connections.pubsub.credentialsFile	string Not marked required by editor	Default: ""
connections.pubsub.endpoint connections.pubsub.endpoint	string Not marked required by editor	Default: "pubsub.googleapis.com:443"
connections.pubsub.emulatorHost connections.pubsub.emulatorHost	string Not marked required by editor	Default: ""
connections.pubsub.ackDeadlineSeconds connections.pubsub.ackDeadlineSeconds	integer Not marked required by editor	Default: 60

Field / key	Type / required	Details
connections.pubsub.connectionTimeoutSeconds connections.pubsub.connectionTimeoutSeconds	integer Not marked required by editor	Default: 30
connections.pubsub.maxInboundMessageBytes connections.pubsub.maxInboundMessageBytes	integer Not marked required by editor	Default: 20971520
connections.pubsub.keepAliveSeconds connections.pubsub.keepAliveSeconds	integer Not marked required by editor	Default: 60
connections.pubsub.batchMaxMessages connections.pubsub.batchMaxMessages	integer Not marked required by editor	Default: 100
connections.pubsub.batchMaxBytes connections.pubsub.batchMaxBytes	integer Not marked required by editor	Default: 1048576
connections.pubsub.batchDelayThresholdMilliseconds connections.pubsub.batchDelayThresholdMilliseconds	integer Not marked required by editor	Default: 10
connections.pubsub.flowControlMaxMessages connections.pubsub.flowControlMaxMessages	integer Not marked required by editor	Default: 1000
connections.pubsub.flowControlMaxBytes connections.pubsub.flowControlMaxBytes	integer Not marked required by editor	Default: 10485760
connections.sap.applicationServerHost connections.sap.applicationServerHost	string Not marked required by editor	Default: "sap-ecc.example.com"
connections.sap.release connections.sap.release	string Not marked required by editor	Default: "current"
connections.sap.systemNumber connections.sap.systemNumber	string Not marked required by editor	Default: "00"
connections.sap.client connections.sap.client	string Not marked required by editor	Default: "100"
connections.sap.language connections.sap.language	string Not marked required by editor	Default: "EN"
connections.sap.username connections.sap.username	string Not marked required by editor	Default: ""

Field / key	Type / required	Details
connections.sap.password connections.sap.password	password Not marked required by editor	Default: "[supply securely]"
connections.sap.messageServerHost connections.sap.messageServerHost	string Not marked required by editor	Default: ""
connections.sap.systemId connections.sap.systemId	string Not marked required by editor	Default: ""
connections.sap.logonGroup connections.sap.logonGroup	string Not marked required by editor	Default: "PUBLIC"
connections.sap.sapRouter connections.sap.sapRouter	string Not marked required by editor	Default: ""
connections.sap.programId connections.sap.programId	string Not marked required by editor	Default: ""
connections.sap.gatewayHost connections.sap.gatewayHost	string Not marked required by editor	Default: ""
connections.sap.gatewayService connections.sap.gatewayService	string Not marked required by editor	Default: ""
connections.sap.maximumConnections connections.sap.maximumConnections	integer Not marked required by editor	Default: 8
connections.sap.outboundWorkerCount connections.sap.outboundWorkerCount	integer Not marked required by editor	Default: 2
connections.sap.poolCapacity connections.sap.poolCapacity	integer Not marked required by editor	Default: 1
connections.sap.peakLimit connections.sap.peakLimit	integer Not marked required by editor	Default: 1
connections.sap.connectionExpirationMilliseconds connections.sap.connectionExpirationMilliseconds	integer Not marked required by editor	Default: 600000
connections.sap.outboundPoolIdleSeconds connections.sap.outboundPoolIdleSeconds	integer Not marked required by editor	Default: 1800
connections.sap.maximumOutboundPools connections.sap.maximumOutboundPools	integer Not marked required by editor	Default: 16

Field / key	Type / required	Details
connections.sap.maxPendingEvents connections.sap.maxPendingEvents	integer Not marked required by editor	Default: 16
connections.sap.jvmInitialHeapMb connections.sap.jvmInitialHeapMb	integer Not marked required by editor	Default: 64
connections.sap.jvmMaximumHeapMb connections.sap.jvmMaximumHeapMb	integer Not marked required by editor	Default: 512
connections.sap.timeoutMilliseconds connections.sap.timeoutMilliseconds	integer Not marked required by editor	Default: 30000
connections.sap.ackTimeoutSeconds connections.sap.ackTimeoutSeconds	integer Not marked required by editor	Default: 300
connections.sap.maximumDynamicSessions connections.sap.maximumDynamicSessions	integer Not marked required by editor	Default: 1024
connections.sapTid.storageFile connections.sapTid.storageFile	string Not marked required by editor	Default: "data/sap-tids.json"

Mediation: business messages without transformation scripts

Transformation

Guide

Mediation: business messages without transformation scripts

Mediation is a separate activity in the transformation palette. Mapper and Transform remain unchanged. Use Mediation to normalize partner formats into a shared business contract using readable, deterministic rules. It does not use AI inference, contact external services, or execute user scripts.

Configuration

Open Configuration, load an example or add business fields. Choose JSON, XML or CSV for input/output. Source collection selects a record or list using a dotted path (``orders``, ``customer.orders``, ``orders.0``) or ``$`` for the entire input. Each rule reads relative to the selected record. A list is processed record by record. Enable Always return collection to preserve an array even when XML contains only one record.

Rules provide a source path, target path, action and Required flag. Target paths such as ``customer.name`` build nested objects. Duplicate or overlapping targets are rejected instead of silently overwriting data. Actions include copy, trim, uppercase, lowercase, integer, number, boolean and dictionary lookup. A constant replaces the source; a default applies to missing/null fields before conversion. Enter constants/defaults as JSON values, for example ``"Unknown"`, `0` or `true``. Required rejects absent input when no default exists. Unmapped fields are not copied automatically; optional absent fields are omitted.

Input and output

Load a target structure

Use Target structure in Configuration to select a project schema, upload a `.json`/`.xsd`` file (up to 500 KB), or paste a schema. The schema is saved as an activity-local snapshot, so exported Python uses the same contract without contacting external schema servers. Reselect the project schema to refresh the snapshot after editing it.

The field list displays dotted target paths, types and required flags. Click + Map field to add a mapping row, then supply its source. Existing rules are preserved. Arrays appear as whole-array targets; nested per-item mapping is not yet implemented. Preview and runtime validate types and required fields before serializing output. Object schemas apply to each record; an array schema validates the whole collection.

Supported JSON Schema keywords: ``type``, ``properties``, ``required``, ``items``, ``additionalProperties``, and descriptive metadata (``title``, ``description``, ``$schema``, ``$id``). References, unions, enums, patterns, numeric constraints and compositions are rejected. XSD supports one root element, sequences, inline/named non-recursive complex types and primitive types. This checks structure, not full XSD facets, namespaces or occurrence bounds. XML attributes, imports, choices and restrictions are rejected. No external references are fetched.

Input and output contract

Map previous activity outputs in Input

After loading a target schema in Configuration, open Input. The left Data pane lists available preceding activity outputs; the right tree lists target fields beneath ``targetValues``. Drag a source onto a target, type an expression such as ``${ReadOrder.id}``, or use Edit mappings for the expanded editor. These mappings are resolved for each invocation, not entered as fixed values in Configuration. An Input mapping overrides that field's recipe source; its conversion and final schema validation still apply. Unmapped fields retain existing recipe behavior. You can use only Input mappings with no recipe rules.

Map ``payload`` to select the message/collection to iterate. A directly mapped target value is resolved once per activity invocation and applies to every selected record; use record-relative recipe rules for per-record values. Whole arrays can be

mapped directly; this does not add nested per-item iteration.

Map & Test accepts a synthetic context such as ``{"activities":{"ReadOrder":{"output":{"id":"A-100"}}}`` to test ``${ReadOrder.id}``. It never reruns ReadOrder or calls a connector. Use synthetic data only. Plain upstream references work in both Studio and exported Python; advanced Input mapping operators remain subject to the exporter's existing supported-expression limits.

The Input tab's payload mapping overrides the previous activity's output. With no override, the previous output is used. JSON input can be an object/list or JSON text. XML/CSV input is text. JSON output is an object or list; XML/CSV output is text. The activity publishes the document directly, not a wrapper containing ``result``. Preview's records/rules/trace envelope is only a design-time explanation.

XML attributes become fields, repeated child elements become lists, and non-empty mixed text is stored under ``_text``. XML output uses a configurable root and ``record`` elements for top-level arrays. DTDs/entities are rejected. This is not a namespace-preserving XML round-trip editor or an XSD validator. CSV output requires scalar, flat target fields; nested targets must instead use JSON/XML. CSV values are serialized faithfully; spreadsheet consumers must apply their own formula-injection protections before opening untrusted CSV.

Try: normalize an order

Load Normalize orders, then Preview mediation. Input:

```
{ "orders": [ { "id": "A-100", "buyer": " Ana ", "amount": "42.50", "status": "N" } ] }
```

The recipe selects ``orders``, copies ``id`` to ``order.id``, trims ``buyer`` into ``customer.name``, converts ``amount`` to ``order.total``, and looks up ``N`` as ``New`` for ``order.status``.

```
[ { "order": { "id": "A-100", "total": 42.5, "status": "New", "customer": { "name": "Ana" } } } ]
```

The other built-in examples demonstrate CSV customer records with lowercase email/boolean conversion, and XML orders with typed quantities and a stable collection output. Loading an example replaces the current recipe. Preview uses the same Python implementation as execution and performs no connector actions. Preview samples are saved with the project: use synthetic data, never credentials or personal production payloads.

Advanced, errors and execution

Standard activity retry and payload logging policies apply. Avoid payload logging for sensitive messages. Studio Run/Debug uses a ``MEDIATION`` fault; exported Python raises ``MediationError``. Conversion errors identify record/rule and target, not field values. Errors include missing required sources, unknown collection paths, invalid recipes, unsupported formats, missing lookup entries, invalid XML and non-flat CSV targets. Failed execution returns no partial result.

Limits are 200 rules, 10,000 selected records, 2 MB text input and 64 levels for XML traversal. This is bounded-batch processing, not a streaming large-file processor. Split larger payloads upstream. Recursive per-child collection mapping, joins, aggregation, chained actions, schema inference and full XSD validation are not part of this initial activity.

Code and deployment

Conditional target fields in Studio

Right-click a target field in Input and select When / Otherwise. Enter the number of When branches (1-100), then select Create branches. A Choose tree appears on that field, with expandable When rows and a final Otherwise row. Edit each condition and value inline; drag source fields into an individual condition or branch value. The first matching When supplies the field value. If none matches, Otherwise supplies it. Inactive branch values are not evaluated by the Studio runtime. Existing single When/Otherwise mappings remain editable. Use Remove conditions to clear the statement from that field.

Implementation locations

- UI recipe editor: ``frontend/src/MediationStudio.tsx``; styles: ``mediation-studio.css``.
- Activity contracts/documentation: ``frontend/src/ActivityEditor.tsx``; palette: ``frontend/src/main.tsx``.
- Engine and validation: ``backend/app/mediation.py``.
- Preview: ``POST /api/mediation/test`` in ``backend/app/main.py``.

- Studio execution: Mediation branch in ``backend/app/runtime.py``.
- Raw Python export: ``backend/app/raw_python.py`` links ``application/native/mediation.py`` only when Mediation is used; dispatch is in ``raw_python_support/core.py``.
- Regression tests: ``backend/tests/test_mediation.py``.

Deploy updated Studio/runtime code before authoring or running this new activity. Re-export packages containing it for deployed runtimes. Existing installations without this activity kind cannot run it. No vendor drivers are required for Mediation itself.

Reference source: docs/MEDIATION.md

MINA documentation site and PDF

Getting started

Guide

MINA documentation site and PDF

Open Help > MINA Documentation in Studio, or visit `/help/index.html` on the same host and port as Studio. No separate documentation service is required. Installed desktop Studio uses its existing local runtime address; do not assume a fixed port.

The documentation includes a searchable, expandable index and deep links to individual topics and tabs. Use Download PDF for the complete offline manual, or Print topic for the selected topic, including all its tabs. Light and dark display modes are supported.

Coverage and accuracy

Activity Configuration, Input, Output, Advanced and Errors sections are generated from Studio's base contracts and documentation definitions. Tasks, groups, mapping, functions, shared connections, deployment, debugging, administration and security guides are included. Project-selected schemas, provider modes and JDBC parameters can change the fields shown in Studio: the reference does not substitute for those project-specific schemas. Provider prerequisites and production qualification still apply.

Only explicitly selected product source files and guides are read. The generator does not inspect project data, environment credentials or user payloads. Existing implementation limitations are documented, not claimed as implemented.

Updating the guide

From the repository root, install the documentation authoring dependencies into your chosen Python environment:

```
python -m pip install -r scripts/documentation-requirements.txt
cd frontend
npm run docs:build
npm run docs:check
node --test scripts/documentation.test.mjs
```

If Python is not on PATH, set `MINA_DOCS_PYTHON` to its executable path before running `docs:build`. The Python dependencies are build-time only, not required to read the site or PDF.

`npm run build` checks that the generated documentation matches the selected source files. After changing those sources, regenerate the guide. The site and PDF are copied from `frontend/public/help` to `frontend/dist/help` by Vite and included in the normal Studio distribution. Rebuild the frontend and installer to deliver updated documentation to installed machines. Keep the whole help directory together when copying it for offline use; opening `index.html` directly also works without a server.

The guide is local to the Studio host. Making it reachable on a shared enterprise URL requires publishing these static assets through your approved web server and access controls; this change does not create a public internet endpoint.

Reference source: docs/DOCUMENTATION.md

Activity debugging and testing

Studio

Guide

Activity debugging and testing

Job Data tree

The ribbon's Job Data window shows processes, call activities, and nested subtask activities in an expandable tree (initially expanded). Select a process or activity to inspect its captured input and output. Data is keyed by process and activity, not by a flattened activity ID. Completed calls record the actual called task, including dynamically resolved calls. Unexecuted activities are labelled as not captured. Repeated invocations show the latest captured values for that task/activity; this is not a per-invocation execution-history viewer.

Windows installations must be rebuilt and updated with the new installer to receive these UI/backend changes; restarting an older installation is insufficient.

During an active debug session, a dedicated Debug ribbon appears underneath the Studio ribbon. It provides Continue, Pause, Stop, Step In/Over/Out, Jump In/Out, Run to Selected, Breakpoints, Watches, Variables, Evaluate, Job Data, Edit Payload, Preview Inputs, Test Activity, and Mock Output. Data/tool buttons open the relevant editor section; real tests and mocks still require explicit submission there. Commands are enabled according to session state. The ribbon disappears when the session completes, fails, or stops; it is not shown during ordinary Run mode.

Choose Run -> Start Activity Testing to create a paused debug session and open the test bench. Unlike ordinary Debug, this does not automatically run the flow or start event listeners. The existing Debug Tools window also contains the bench.

Use the searchable process/activity tree on the left of the test bench. Expand a process or subprocess and select its activity; enter the input in the right pane. Each target retains its own input draft while the dialog remains open. Selecting a Call Sub Task row selects the call activity itself; expanding its arrow reveals the called subprocess and its activities. Nested calls can be expanded the same way, and selecting a nested activity targets its owning subprocess. Search includes nested activities. Circular references stop with a notice. Dynamic calls show a configured fallback, if any, labelled as potentially different from the runtime target; unresolved targets are not guessed.

Selecting a process targets its unique entry activity (or Start); Run once still executes only that entry activity, not the entire process. If the entry is ambiguous, select an activity explicitly. Independent tests can target another process in the debug project without moving the paused flow. Mock stepping is restricted to the current paused process and activity.

Select an activity and enter a JSON payload (including arrays, strings, numbers, or null), or choose raw text/XML. The test payload is available as both ``input`` and ``last`` for this independent test. Earlier activity outputs and environment properties remain available for mappings.

- Preview inputs resolves configuration and mappings without executing the activity.

- Run once (real) executes the selected activity without advancing the paused

flow. Inspect resolved inputs, output, logs, duration, and failure details.

- Use mock output & step supplies the payload as the current paused activity's

output, skips its actual connector execution, and follows its normal transitions. Use this to simulate a receiver or downstream dependency without contacting it.

- Optional field overrides are a JSON object keyed by the activity's configuration

field names, such as ``message`` or ``topic``. They are applied after mappings as literal values. Preview first to verify the inputs.

- Optional assertions use runtime conditions, for example ``${last.count} > 0``.

Results distinguish execution failures from assertion failures.

The variable editor accepts whole ``input``, ``last``, and ``vars`` replacements, as well as nested paths such as ``input.customer.id``. Replacing ``input`` does not also replace ``last``; edit each explicitly when needed. ``vars`` must remain an

object.

Safety and scope

Real tests use the configured connections and can publish, acknowledge, write, or call external systems. Use test environments and credentials. They are not an external-side-effect sandbox or an automatic rollback mechanism. Independent tests do not inherit a paused group's database transaction or lock context; use normal flow stepping to test those lifecycles.

Single-activity tests default to a ten-second timeout. A timeout cannot guarantee cancellation of work already submitted to a provider or blocking vendor driver. Mocks bypass the activity, not the surrounding group's execution semantics. Input overrides apply to configuration-driven activity fields; special task or interface mapping behavior still follows the activity's runtime implementation.

The most recent full result and the last 30 result summaries are retained for the debug session. Test payloads and mock results do not modify or export into the project. This bench complements breakpoints, watches, expression evaluation, variable editing, and flow stepping; real-provider integration and load testing still require the target infrastructure.

Reference source: docs/DEBUG_TESTING.md

MINA Studio debugging

Studio

Guide

MINA Studio debugging

MINA Studio supports interactive debugging for ordinary tasks and continuous event starters.

Execution controls

- Continue runs until completion, an enabled breakpoint, a handled exception when pause-on-error is enabled, or a manual pause.
- Step In, Step Over, and Step Out navigate activities and Call Sub Task frames.
- Run to selected continues until the activity selected on the canvas is about to execute.
- Stop cancels the debug driver and listener, rolls back open transaction groups, and releases owned native processes.

Debug Tools

Open Debug Tools from the debug bar to use the following features:

- Add an optional condition to any canvas breakpoint. Conditions use the same expression syntax as transitions, for example ``${last.amount} > 100`` or ``contains(${last.status}, "FAIL")``.
- Enable or disable pause-on-error for handled error and retry paths.
- Add watch expressions such as ``${input.orderId}``, ``${last.customer.id}``, or ``${vars.retryCount}``.
- Inspect the current input, previous output, process variables, process context, call stack, group iterations, and last exception.
- Evaluate an expression without modifying the running task.
- While paused, change a value under ``input.*``, ``last.*``, or ``vars.*``. JSON values are typed; other input is stored as text.

Environment properties and connector credentials are intentionally excluded from the live-variable editor.

Continuous starters

An EMS/JMS, Kafka, Pub/Sub, SAP, timer, or file listener displays ready · waiting for event until an event arrives. Breakpoints, watches, and pause-on-error remain active for every event. Stopping the session also stops the listener and prevents it from re-arming.

Reference source: docs/DEBUGGING.md

Data and file utilities

Studio

Guide

Data and file utilities

Open Window -> XML Viewer, Window -> JSON Viewer, or Window -> Compare Files. These tools use a dedicated Studio workspace above the orchestration editor, so a utility can remain open without changing the active Task.

Large-file behavior

The desktop application gives the UI an opaque file handle rather than unrestricted filesystem access. It reads only the selected 256 KB, 1 MB, or 4 MB window. Previous and Next move through the file without retaining earlier windows. Browser mode applies the same behavior with ``Blob.slice()``.

Each active window is editable. Desktop saves stream the untouched prefix and suffix through a temporary file and then replace the original, so even a very large file is not assembled in renderer memory. Navigation is locked while edits are unsaved. Browser mode downloads the edited result because web pages cannot overwrite the selected local file directly.

The utilities do not require an existing file. New XML, New JSON, New left, and New right create scratch documents where content can be typed or pasted directly. Save As converts a scratch document into a filesystem-backed document. Ctrl+S saves the editor that currently has focus.

Complete XML or JSON files up to 16 MB can be validated and pretty-printed as a whole. Larger files use a tolerant pretty printer for the current bounded window, avoiding an unbounded parse tree or full-file string in renderer memory.

XML Viewer

- Opens XML, XSD, and WSDL files.
- Creates blank XML documents for direct typing or clipboard paste.
- Pretty-prints well-formed complete documents.
- Decode `<` and `>`; changes ``<`` and ``>`` into visible angle brackets in the display.
- Wraps long or minified lines by default so payload content stays visible; wrapping can be toggled from the toolbar.
- Searches the current file window.
- Saves edits or formatting back to the selected desktop file and provides Revert for unsaved changes.

JSON Viewer

- Opens JSON and JSON Lines files.
- Creates blank JSON documents for direct typing or clipboard paste.
- Pretty-prints and validates complete JSON documents.
- Uses bounded-window formatting for very large payloads.
- Wraps long or minified lines by default.
- Searches the current file window.
- Saves edits or formatting back to the selected desktop file and provides Revert for unsaved changes.

Compare Files

- Accepts any two file types.
- Allows either side to be a newly created document instead of an imported file.
- Uses side-by-side aligned text rows for text files.

- Marks added, removed, and changed rows separately.
- Automatically displays binary or unknown content as hexadecimal and printable ASCII.
- Supports a differences-only view and synchronized window navigation.
- Allows individual difference rows-or all differences in the current window-to be selected and copied left-to-right or right-to-left.
- Provides a two-pane editing mode and independent Save left and Save right actions.
- Detects external file changes before saving and refuses to overwrite newer content.
- Renders at most 10,000 comparison rows per window to prevent unusually short-line files from creating an unbounded number of UI elements.

Reference source: docs/FILE_UTILITIES.md

Runtime logging

Operations

Guide

Runtime logging

Studio records Run, Debug, listener, job, activity, transition, retry, exception, and application lifecycle events in the Execution / Debug panel. The disk button in that panel reloads saved records after a run or after Studio restarts.

Each environment properties file contains the user-configurable property:

```
`runtime.logDirectory`
```

Set it independently in ``local.properties``, ``dev.properties``, ``qa.properties``, ``pre.properties``, and ``production.properties``. Run, Debug, listeners, and Stop use the selected runtime environment and write beneath that environment's configured directory. For example, setting the development value to ``D:\IntegrationLogs\dev`` produces:

```
`D:\IntegrationLogs\dev\<project-name>\<project-name>.log`
```

Relative values are resolved beneath the Mina data directory. Environment variables and ``~`` are expanded. When the property is blank, each project has an isolated structured log at the default location:

```
`<Mina data directory>/logs/<project-name>/<project-name>.log`
```

Log and audit timestamps use Arizona time by default (``America/Phoenix``, shown with the ``-07:00`` offset). Set ``MINA_LOG_TIMEZONE`` to another IANA timezone when a deployment requires a different display timezone. Scheduler and token expiry calculations continue to use UTC internally.

The active file automatically rolls at 10 MB. Four numbered archives are retained by default (``<project-name>.log.1`` through ``<project-name>.log.4``), so an individual file never grows without bound. Each line is UTF-8 JSON and includes the project, timestamp, level, message, and available run, correlation, task, activity, duration, and exception context. Failed package builds also write a ``packaging`` record with a traceback to this log, and the API returns the log path with the error. The exact resolved path is shown in Studio's Execution / Debug panel after clicking the disk button.

Packaged desktop installations place the Mina data directory under the Studio user-data location configured through ``MINA_DATA_DIR``. Administrators may override logging without changing code:

- ``MINA_RUNTIME_LOG_DIR``: process-wide fallback root when the active environment's ``runtime.logDirectory`` property is blank.
- ``MINA_PROJECT_LOG_MAX_BYTES``: maximum bytes per active or archived file; default ``10485760`` (10 MB).
- ``MINA_PROJECT_LOG_BACKUP_COUNT``: number of rolling archives; default ``4``.

Payload bodies are written only when the activity's Automatic payload logging option is enabled. Passwords and secrets should never be deliberately mapped into log messages.

Reference source: docs/RUNTIME_LOGGING.md

MINA Control Plane guide

Operations

Guide

MINA Control Plane guide

Purpose and operating model

MINA Control Plane is the self-hosted management plane for MINA data planes, capabilities, applications, resources, access assignments, observability, and immutable deployment packages. A data plane represents an on-premises runtime host or Kubernetes runtime boundary. Capabilities are provisioned into a data-plane namespace, while applications are deployed to a selected data plane, namespace, and compatible Integration Runtime capability.

MINA separates control and data planes and provides namespace-scoped capabilities and applications, platform resources, role assignments, health/heartbeat inventory, application lifecycle, audit, and observability dashboards.

Organization and delivery-team isolation

The built-in Technology Team is the permanent Control Plane owner. It registers data planes, provisions capabilities, configures platform resources, creates delivery teams, assigns principals, issues or revokes delivery credentials, and can govern all assets. Set `MINA\_ADMIN\_API\_KEY` for its production credential.

Each data delivery team must be assigned one or more exclusive `{data plane, namespace}` scopes. The same namespace cannot be assigned to two active delivery teams. Packages, extracted package storage, deployments, environment requirements, encrypted secrets, runtime logs, lifecycle operations, and application observability carry an immutable `teamId`. Backend authorization returns `404` when another delivery team probes an asset identifier, preventing both access and asset discovery.

Technology Team can upload a package on behalf of a delivery team by selecting its name beside Upload package. Delivery automation uses a one-time credential issued from Delivery teams and sends it in `X-Control-Plane-Key`. Only a SHA-256 hash of that credential is stored. An Application Manager token can upload, deploy, manage secrets, run lifecycle operations, and inspect its own team assets. An Application Viewer token is read-only. Delivery credentials cannot access Control Plane overview, data-plane registration, capabilities, resources, team/access administration, global monitoring, or audit APIs.

For production, configure corporate identity-provider groups at the reverse proxy or identity gateway and exchange their authenticated team identity for scoped Control Plane credentials. Never expose the Technology Team key to delivery pipelines.

Control Plane does not rebuild a Studio project. A package already contains the selected starter tasks and every reachable called task, shared connection, XSD/JSON schema, resource, property profile, and deployment descriptor selected during packaging. Cloud packages keep their generated Docker/Kubernetes descriptors and can be assigned only to a Kubernetes data plane. On-premises packages can run locally through the configured command adapter.

Install and start

Development

```
cd D:\Integration-tool\MINA\administrator
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
$env:MINA_ADMIN_DATA_DIR = "D:\MINAAdmin\data"
.\.venv\Scripts\python.exe run_admin.py
```

Open `http://localhost:9080`. OpenAPI is available at `http://localhost:9080/docs`.

Windows distribution

Build with `scripts/build-administrator.ps1 -Version 2.4.0`, extract the generated versioned ZIP, and run `MinaAdministrator.exe`. The output is a one-directory native Windows distribution, so copy the complete extracted

directory-not only the `.exe`. Use `bin\minaadmin.cmd start|stop|status|run` for service-style local control.

Linux distribution

Build on Linux with `scripts/build-administrator.sh 2.4.0`, extract the generated tarball under `/opt/mina/control-plane`, set `MINA_ADMIN_HOME`, and use `bin/minaadmin start|stop|status|run`. PyInstaller output is operating-system specific: build the Linux bundle on Linux and deploy the entire extracted directory.

Container

```
docker build -f Dockerfile.administrator --build-arg MINA_VERSION=2.4.0 -t mina-administrator:2.4.0 .
docker run -d --name mina-admin -p 9080:9080 \
-e MINA_ADMIN_API_KEY='replace-me' \
-e MINA_ADMIN_SECRET_KEY='retrieve-from-secret-manager' \
-v mina-admin-data:/var/lib/mina/administrator \
mina-administrator:2.4.0
```

Configuration

Variable	Default	Meaning
<code>MINA_ADMIN_HOST</code>	<code>0.0.0.0</code>	HTTP bind address
<code>MINA_ADMIN_PORT</code>	<code>9080</code>	HTTP port
<code>MINA_ADMIN_DATA_DIR</code>	<code>administrator/data</code>	Repository, state, encrypted secrets, logs, and audit location
<code>MINA_ADMIN_API_KEY</code>	empty	If set, <code>/api/*</code> except health requires <code>X-Admin-Key</code>
<code>MINA_ADMIN_SECRET_KEY</code>	generated local key	Stable encryption passphrase; supply from a secret manager in clustered/production installs
<code>MINA_ADMIN_RUNTIME_COMMAND</code>	empty	Administrator-approved runtime command template
<code>MINA_ADMIN_MAX_PACKAGE_MB</code>	<code>250</code>	Maximum uploaded archive size
<code>MINA_ADMIN_MAX_EXPANDED_MB</code>	<code>1024</code>	Maximum expanded package size
<code>MINA_ADMIN_MAX_PACKAGE_FILES</code>	<code>10000</code>	Maximum archive members

The runtime command supports `{application}`, `{package}`, `{environment}`, `{deployment_id}`, and `{instance_id}` placeholders. The desktop installer includes a separate `MINAWorker` executable for this purpose. Example:

```
export MINA_ADMIN_RUNTIME_COMMAND='mina-runtime --application {application} --environment {environment}'
```

On Windows desktop installations, configure the worker executable rather than the Studio sidecar:

```
$worker = 'C:\Program Files\MINA Studio\resources\runtime\MINAWorker\MINAWorker.exe'
$command = "\"$worker\" --application \"{application}\" --environment \"{environment}\""
[Environment]::SetEnvironmentVariable('MINA_ADMIN_RUNTIME_COMMAND', $command, 'Machine')
```

The runtime receives `MINA_APPLICATION_DIR`, `MINA_ENVIRONMENT`, `MINA_DEPLOYMENT_ID`, `MINA_INSTANCE_ID`, and decrypted deployment secret values in its process environment. The command comes only from trusted Administrator configuration; package contents cannot provide an executable command.

Package and deployment workflow

- In Studio Packaging, select the starter tasks, environments, target, archive format (`.mpkg`, `.zip`, `.tar.gz`, or `.ear`), and deployment files. Only selected starter tasks and their recursively called Sub Tasks are packaged. Legacy `.ifpkg` uploads remain supported during the compatibility period.

- Select Export archive for an offline bundle, or enter the Control Plane URL, credential, team, data plane, namespace, capability, deployment environment, and required secrets and select Deploy to Control Plane. Studio builds the archive once, uploads those exact bytes, creates the deployment, and reports the Control Plane deployment ID/state. Credentials and secret values are transient and are not saved in the project or package.
- Alternatively, upload an exported archive in Applications. Administrator rejects traversal paths, links/devices, duplicate paths, oversized expansion, unsupported formats, missing project/task artifacts, and invalid manifests.
- Review checksum, target, profiles, selected task metadata, and secret requirements.
- For local on-premises deployment, Start uses ``MINA_ADMIN_RUNTIME_COMMAND``. Generated packages contain Linux ``install.sh`/`deploy.sh`` and Windows ``install.ps1`/`deploy.ps1`/`start.ps1`` assets. If the command adapter is not configured, startup fails visibly rather than reporting a false running state.
- For cloud deployment, the Control Plane records the desired application, environment, replicas, namespace, and capability. The registered Kubernetes data-plane agent applies the generated Dockerfile, ConfigMap, Secret, Deployment, Service, HPA, and Kustomize assets; Studio does not incorrectly invoke the local command adapter for cloud targets.
- Use Details for instance PIDs and logs. Stop performs normal termination; Kill is forced; Restart stops and recreates desired instances; Undeploy removes deployment secrets and the active inventory record.

For a private CA, keep TLS verification enabled and supply the CA PEM path in Studio. Disabling verification is provided only for isolated development systems.

Legal lifecycle transitions are enforced. Package deletion is blocked while any non-undeployed deployment references it.

Application health is evaluated per deployment. For the local command adapter, Control Plane verifies each managed runtime PID. Remote and Kubernetes agents report ``HEALTHY``, ``DEGRADED``, ``UNHEALTHY``, or ``UNKNOWN`` in the ``deploymentHealth`` object of their data-plane heartbeat; Control Plane does not infer application health merely because the data plane is online. Health checks can be disabled per deployment.

Starter Task start/stop changes the deployment's desired starter set. A running local deployment is restarted with ``MINA_ENABLED_STARTERS`` containing only enabled task IDs. Remote and Kubernetes runtime agents reconcile the same desired state and may report their observed task state with the health heartbeat. Whole-application lifecycle state and individual Starter Task state remain separate.

Control-plane model and screens

- Overview: fleet, capability, deployment, request, and recent activity summaries.
- Data planes: local, remote on-premises, and Kubernetes registrations; namespace, capacity, tag, tunnel, heartbeat, and health inventory.
- Capabilities: namespace-scoped capability provisioning and status. Application deployment requires an Integration Runtime capability.
- Applications: searchable package/deployment repository; archive task and Starter Task inventory; health, instance details and logs; deploy, start, stop, configure, redeploy, undeploy, and safe package deletion; deployment and configuration revision history; individual Starter Task desired-state controls.
- Environments & secrets: exportable packaged profiles, JSON profile editing/upload, optional immediate redeployment, and required secret names. Password values are rejected from profile uploads because secret values remain encrypted and deployment-scoped.
- Observability: control-plane request/error totals plus data-plane and application CPU, memory, instance, and state telemetry.
- Resources: reusable global or data-plane-scoped resource definitions. Secret-valued properties are masked in list responses and audit entries.
- Access control: platform and team principals with Owner, Team Admin, Capability Manager, Application Manager, and Application Viewer roles, optionally scoped to a data plane and namespaces.
- Delivery teams: exclusive namespace allocation, asset counts, and one-time scoped automation credentials. Control Plane access remains disabled for delivery teams.

- Audit trail: provisioning, registration, deployment, resource, access, secret, and lifecycle operations with time, outcome, target, and detail.

REST API summary

- ``GET /api/health`, `/api/control-plane/overview`, `/api/monitoring`, `/api/observability`, `/api/audit``
- ``GET|POST /api/data-planes`; `GET|DELETE /api/data-planes/{id}`; `POST /api/data-planes/{id}/heartbeat``
- ``GET|POST /api/capabilities`; `DELETE /api/capabilities/{id}``
- ``GET|POST /api/resources`; `DELETE /api/resources/{id}``
- ``GET|POST /api/access/principals``
- ``GET|POST /api/teams`; `PUT|DELETE /api/teams/{id}``
- ``POST /api/teams/{id}/tokens`; `DELETE /api/teams/{id}/tokens/{tokenId}``
- ``GET /api/session``
- ``GET /api/applications``
- ``GET|POST /api/packages`; `GET|DELETE /api/packages/{artifact}/{version}``
- ``GET /api/packages/{artifact}/{version}/tasks``
- ``GET|PUT /api/packages/{artifact}/{version}/environments/{environment}``
- ``GET|POST /api/deployments`; `GET /api/deployments/{id}``
- ``PUT /api/deployments/{id}/configuration`; `GET /api/deployments/{id}/health``
- ``POST /api/deployments/{id}/starters/{taskId}/{start|stop}``
- ``PUT /api/deployments/{id}/secrets``
- ``POST /api/deployments/{id}/{start|stop|restart|kill|undeploy}``
- ``GET /api/deployments/{id}/logs``
- ``GET /api/revisions/{package|deployment}/{id}``

Security, recovery, and troubleshooting

- Put Control Plane behind TLS/reverse proxy and set an API key. Restrict the data directory to the service identity.
- Permission enforcement is performed on every backend API call. UI filtering is informational and is not the security boundary.
- Back up the entire data directory together with the external ``MINA_ADMIN_SECRET_KEY``. Losing or changing the key makes stored secrets unreadable.
- Rotate a secret with the secret API while stopped, then restart. The API returns secret names/configuration state only.
- A ``FAILED`` deployment preserves its error and logs. Correct the adapter or configuration and select Restart.
- ``No runtime adapter is configured`` means package validation and deployment creation are working, but ``MINA_ADMIN_RUNTIME_COMMAND`` has not been supplied.
- Control Plane PID reconciliation detects local processes that exited unexpectedly. Data-plane registration and heartbeat APIs provide the management-plane inventory. Remote command execution still requires a trusted data-plane agent/tunnel; this repository does not silently execute remote commands or claim a remote application is running without that adapter.

Reference source: docs/ADMINISTRATOR_GUIDE.md

Control Plane operator workspace

Operations

Guide

Control Plane operator workspace

The Control Plane home page summarizes running applications, data-plane connectivity, unhealthy/stopped deployments, and recent operations. The Applications workspace lets an operator search and filter deployments and packages, then inspect an application without leaving the list.

The application inspector exposes lifecycle actions, health and instances, environment/instance/health-check configuration, write-only secret updates, Starter Task controls, runtime logs, revision history and revert, configuration comparison, backup, and restore. Existing package upload, archive inspection, and deployment flows remain available.

The Telemetry page shows CPU, memory, and disk readings reported by agents, with bounded trends (up to 720 samples per data plane, sampled no more than once every 15 seconds). It also shows task outcomes and recent Control Plane request errors. Blank readings mean the agent has not reported that metric; they are not treated as zero.

Operators can create or delete alert rules for offline data planes, failed deployments, unhealthy applications, and CPU/memory/disk thresholds. Resource rules report `NO\_DATA` when a host has not sent the selected metric. Rules are evaluated in the Control Plane UI/API; external notification delivery is not implemented by these rules.

Python data-plane agents install `psutil` to report host CPU, memory, and disk. If `psutil` is unavailable, the agent still reports disk space and its local process count. A remote on-premises Python agent forwards a bounded tail of each instance log. Kubernetes pod-log forwarding is not yet implemented; local deployments are read from the Control Plane host.

Reference source: [docs/control-plane-operator-workspace.md](#)

Python archive modes

Deployment

Guide

Python archive modes

Studio exposes two distinct Python exports. Both contain Python application files and `manifest.json` for Control Plane package metadata; neither includes project, task, or resource JSON descriptors.

- Export direct Python / Deploy direct Python compiles supported task transitions into async Python state machines. The generated task source contains direct `if/elif` flow decisions, inline group conditions/loops, `asyncio.gather` for success fan-out, and typed Python references for property and activity bindings. It does not bundle the MINA workflow engine. The exporter rejects unsupported semantics with activity names instead of silently substituting an engine-backed implementation. Supported paths include Start, End, Scheduler, Log, Mapper (including functions, conditions, and repeating targets), Call Sub Task, global and transition Catch, Throw, Rethrow, General activities (Empty, Assign, Sleep, Checkpoint, shared variables, and External Command), File, FTP/SFTP, hosted and outbound HTTP/REST/SOAP, HTTP Response, XML/JSON/flat-data parse and render, Excel Read, Transform, Python/Java Invoke, JDBC, Snowflake, AMQP, Kafka, Pub/Sub, EMS/JMS send, receive, request/reply and reply, and supported SAP operations and IDoc parsing. Scheduler, Kafka, Pub/Sub, EMS, JMS, SAP, HTTP, REST, and SOAP starters remain active until stopped. SAP IDoc listeners support direct JCo and EMS/JMS/Kafka messaging sources. Single-entry/single-exit If, For Each, Iterate, While, Repeat, Repeat-on-Error, JDBC transaction, and critical-section groups compile to explicit Python control flow and may be nested with mixed group types. Client-acknowledge JMS receivers use a persistent bridge session and Confirm Message can acknowledge an active delivery.

- Python compatibility archive (engine-backed) retains the full Python runtime engine and the existing wider activity/group behavior. It remains deployable but is not a full direct-code conversion. The package manifest records `pythonSource.mode: engine` so consumers can tell which implementation they received.

The standard `.mpkg` and `.ear` exports remain available. Python archives require separately licensed vendor drivers where the underlying connection needs them (for example SAP JCo or JMS/EMS). A Python source package does not replace those vendor libraries. Legacy `.ifpkg` packages remain importable.

Engine-backed Python archives select connector modules from the packaged activities and groups instead of copying every engine file. All included Sub Tasks participate in selection, and JDBC transaction groups include the JDBC adapter. The exporter follows adapter imports transitively (including Java bridge dependencies and the JDBC adapter's Snowflake support). Shared scheduling, model, mapping/expression, and timestamp modules remain in every engine-backed archive. Optional connectors load only when executed; unused shared connection definitions do not cause their connector modules to be bundled. Standard JSON-based EAR archives do not contain this Python engine.

Direct archives are capability-linked from the persisted project model and selected task closure. The linker never checks project names, sample identifiers, known canvas layouts, or a catalog of previously exported applications. It records the exact activity operations and group types in `application/capabilities.py`, removes unreachable execution branches, specializes connector functions to the operations used by the project, and omits unreferenced activity modules, connector modules, native adapters, inbound servers, shared resources, schemas, and properties. Referenced shared resources are followed transitively, as are XSD `schemaLocation` and JSON Schema `$ref` dependencies, so dependency pruning does not produce an incomplete package. Its registry is required to cover every activity operation advertised by the direct exporter; adding a future supported Studio activity without linker metadata fails the exporter and its coverage test instead of silently generating an incomplete application. A property-driven resource, schema, or SAP messaging-provider selection conservatively retains compatible candidates because the selected object can change between packaged environments. Packaging never removes something required by a selected environment merely to make the archive smaller.

For a successfully compiled direct archive, run `python exported.pypkg --task TASK_ID --input '{"key":"value"}'` on a standard Python host. No extraction or source edit is needed. Alternatively, extract the archive and use `python run.py` as a Python-script job entry point (including a Databricks Python script task); pass `--environment PROFILE` when multiple profiles are packaged, and `--task TASK_ID` for a single task. Without `--task`, the generated application runs the selected starters; receiver starters keep polling until stopped. If no environment is specified, the first packaged profile is used.

Run ``python run.py --check`` (or ``python exported.pympkg --check``) in the target data plane before deployment. The generated preflight checks only the dependencies used by that application, returns machine-readable JSON, and exits with code ``2`` when a required package, Java runtime, or external project artifact is missing. The Control Plane manifest also exposes the preflight command and dependency inventory.

For qualification against real providers, run ``python run.py --qualify-task TASK_ID --input '{"test":true}' --iterations 1000 --concurrency 10``. Optional thresholds include ``--max-error-rate``, ``--max-p95-ms``, and ``--max-memory-growth-mb``; ``--duration-seconds`` turns the same command into a bounded soak test. The command exercises the actual exported task and produces JSON evidence containing throughput, success/failure counts, latency percentiles, bounded error samples, and Python memory growth. It exits with code ``3`` when a threshold fails. Provider failover is tested by running this command while performing an approved broker/database/SAP failover in the test environment.

Real connectors require runtime provisioning, not source edits. Set ``MINA_JAVA_BRIDGE_HOME`` to the installed Java bridge directory containing ``classes``, and ``MINA_DRIVER_HOME`` to a directory containing ``sap/`` and/or ``jms/`` licensed vendor JARs. Rebuild the bridge from this source before deploying client-acknowledge JMS flows. Place the SAP JCo native library in the SAP driver directory or on the platform native library path. A Java runtime must be available via ``MINA_JAVA`` or ``PATH``. Depending on the generated preflight inventory, install ``aiokafka``, ``google-cloud-pubsub`` and ``google-auth``, ``openpyxl``, ``paramiko``, ``snowflake-connector-python``, ``pika``, or ``azure-servicebus``. Provide connection URLs, credentials, and target-network access through the selected environment profile or environment variables. Missing packages and artifacts are reported before application startup.

The direct export is not a blanket compatibility promise for every future Studio feature. Grouped parallel fan-out still requires an explicit top-level branch boundary because a deterministic in-group join is not represented in the saved graph. Live ECC/JCo, EMS/JMS provider, AMQP, Snowflake, failover, and soak results cannot be asserted without those systems; use the generated qualification command in the target test environment and retain its evidence before approving production. The automated suite covers hosted REST, JMS request/reply correlation, SAP-over-Kafka, mixed nested groups, mock/in-memory providers, SQLite transactions, and bridge compilation. The saved ``order-integration`` project passes direct compiler static validation, which is not itself a live SAP qualification. Export reports unsupported graph behavior instead of generating a package with silently different behavior.

Reference source: docs/PYTHON_EXPORT.md

Vendor driver installation

Deployment

Guide

Vendor driver installation

MINA Studio bundles its own minimal Java runtime and connector bridge. Vendor-licensed JARs are intentionally not distributed in the installer.

The default machine-wide driver root on Windows is:

```
C:\ProgramData\MINA Studio\drivers
```

Set `MINA\_DRIVER\_HOME` to override the complete root, or enter a Driver JAR directory in an individual shared connection. Source checkouts also search `D:\Integration-tool\MINA\drivers` as a development fallback.

EMS / JMS

Place the EMS client files in:

```
C:\ProgramData\MINA Studio\drivers\jms\tibjms.jar
C:\ProgramData\MINA Studio\drivers\jms\jakarta.jms-api-2.0.3.jar
```

For older EMS releases, use the matching `jms-2.0.jar` supplied with that EMS installation. Add `tibjmsufo.jar` for unshared-state failover. Do not mix JARs from different EMS releases.

The EMS direct factory class is supplied by the runtime default. If overriding it, use the exact fully qualified class name from your licensed driver documentation. Java class names are provider identifiers and must not be renamed.

Direct mode uses the EMS URL, username, and password. JNDI mode additionally uses the configured initial-context factory, provider URL, credentials, and connection-factory JNDI name.

Microsoft SQL Server

Download the Microsoft JDBC Driver for SQL Server and place the Java 11-compatible JAR in:

```
C:\ProgramData\MINA Studio\drivers\jdbc\sqlserver\mssql-jdbc-<version>.jre11.jar
```

The driver class is `com.microsoft.sqlserver.jdbc.SQLServerDriver`. SQL Server Authentication is fully Java-based and does not need ODBC. Windows Integrated Authentication additionally requires Microsoft's architecture-matched authentication DLL; username/password authentication is the portable default.

Oracle Database

Place the Oracle JDBC driver in:

```
C:\ProgramData\MINA Studio\drivers\jdbc\oracle\ojdbc11.jar
```

Optional Oracle companion JARs such as `orai18n.jar`, `ucp.jar`, or Oracle PKI libraries can be put in the same directory when the selected Oracle features require them. The driver class is `oracle.jdbc.OracleDriver`. Thin JDBC mode does not require Oracle Instant Client.

Build prerequisite

The installer build machine needs JDK 17 or newer because `desktop:installer` compiles the bridge and creates the bundled Java runtime. End-user machines do not need Java installed.

Build the bridge alone with:

```
powershell -ExecutionPolicy Bypass -File scripts\build-java-bridge.ps1
```

The connection-test response reports `loadedJars`, making it possible to verify exactly which vendor files were loaded.

Reference source: docs/VENDOR_DRIVERS.md

MINA environment variables

Deployment

Guide

MINA environment variables

This is the reference list of environment variables recognized by the current MINA source, Studio sidecar, and Control Plane. Variable names are case-sensitive on Linux and Kubernetes. Values containing passwords, private keys, API keys, or tokens must be supplied by the operating system, service manager, or cloud secret store; do not commit them to a project or container image.

Runtime and Studio sidecar

Variable	Default	Scope and purpose
<code>MINA_PORT</code>	<code>8787</code>	Backend HTTP port used by <code>run_sidecar.py</code> . Studio normally assigns this automatically.
<code>MINA_LOG_LEVEL</code>	<code>info</code>	Uvicorn process log level, for example <code>debug</code> , <code>info</code> , <code>warning</code> , or <code>error</code> .
<code>MINA_DATA_DIR</code>	<code>backend/data</code>	Runtime data root for projects, SAP TID files, and default project logs.
<code>MINA_RUNTIME_LOG_DIR</code>	<code><data-dir>/logs</code>	Process-wide fallback log root when <code>runtime.logDirectory</code> is blank.
<code>MINA_PROJECT_LOG_MAX_BYTES</code>	<code>10485760</code>	Maximum size of each project log file before rotation.
<code>MINA_PROJECT_LOG_BACKUP_COUNT</code>	<code>4</code>	Number of rotated project log files retained.
<code>MINA_LOG_TIMEZONE</code>	<code>America/Phoenix</code>	IANA display timezone for runtime, debugger, and audit timestamps. Arizona is emitted with <code>-07:00</code> .
<code>MINA_WORKER_THREADS</code>	<code>auto</code>	Runtime worker-thread setting; otherwise environment property <code>advanced.workerThreads</code> or <code>advanced.threadCount</code> is used.
<code>MINA_APPLICATION_DIR</code>	unset	Application directory supplied to a deployed runtime by the Control Plane.
<code>MINA_ENVIRONMENT</code>	<code>local</code> or deployment value	Active runtime environment.
<code>MINA_DEPLOYMENT_ID</code>	unset	Deployment identifier supplied by the Control Plane.
<code>MINA_INSTANCE_ID</code>	unset	Runtime instance identifier supplied by the Control Plane.
<code>MINA_BUILD_VERSION</code>	<code>source</code>	Build/version value included in startup diagnostics.
<code>MINA_STATIC_DIR</code>	bundled static directory	Override location of backend-served frontend assets.
<code>MINA_DRIVER_HOME</code>	platform search locations	SAP JCo/native driver directory override. Must contain matching <code>sapjco3.jar</code> and native library.
<code>MINA_JAVA</code>	automatic Java lookup	Java executable override for the SAP/JMS Java bridge.
<code>MINA_JAVA_BRIDGE_HOME</code>	repository/bundled bridge location	Java bridge installation directory override.
<code>MINA_AI_MODEL</code>	<code>gpt-5</code>	AI Builder model name when <code>OPENAI_API_KEY</code> is configured.
<code>OPENAI_API_KEY</code>	unset	Enables the OpenAI AI Builder provider. Store as a secret.

The Electron Studio sets `MINA\_PORT`, `MINA\_DATA\_DIR`, `MINA\_LOG\_LEVEL`, and `MINA\_BUILD\_VERSION` for its child runtime. It also supports these Studio-only process variables:

Variable	Purpose
`MINA_PYTHON`	Python executable override for development sidecar startup.
`MINA_DEV_URL`	Frontend development URL loaded by Electron instead of the local production URL.

Control Plane

Variable	Default	Scope and purpose
`MINA_ADMIN_HOST`	`0.0.0.0`	Control Plane HTTP bind address.
`MINA_ADMIN_PORT`	`9080`	Control Plane HTTP port.
`MINA_ADMIN_DATA_DIR`	`administrator/data`	Control Plane state root: packages, deployments, machines, secrets, audit, and logs. Mount this directory persistently.
`MINA_ADMIN_API_KEY`	unset	Technology Team API key. When set, protected APIs require `X-Admin-Key` or `X-Control-Plane-Key`. Store in a secret manager.
`MINA_ADMIN_SECRET_KEY`	generated local key	Stable encryption key for stored deployment secrets. Required from a durable secret store for production and clustered deployments.
`MINA_ADMIN_RUNTIME_COMMAND`	unset	Trusted command template used to start an on-premises runtime. Supports `{application}`, `{package}`, `{environment}`, `{deployment_id}`, and `{instance_id}`.
`MINA_ADMIN_MAX_PACKAGE_MB`	`250`	Maximum uploaded package size in megabytes.
`MINA_ADMIN_MAX_EXPANDED_MB`	`1024`	Maximum expanded package size in megabytes.
`MINA_ADMIN_MAX_PACKAGE_FILES`	`10000`	Maximum archive member count.
`MINA_ADMIN_VERSION`	packaged value or `development`	Control Plane version override.
`MINA_ADMIN_WEB`	bundled `web` directory	Override location of Control Plane frontend files.

The Linux service wrapper also recognizes:

Variable	Default	Purpose
`MINA_ADMIN_HOME`	`/opt/mina/control-plane`	Installed Control Plane directory.
`MINA_ADMIN_PID_DIR`	`/opt/mina/run`	PID file directory used by `bin/minaadmin`.
`MINA_ADMIN_LOG_DIR`	`/opt/mina/logs/control-plane`	Service-wrapper stdout/stderr log directory.

Build and packaging variables

Variable	Purpose
`MINA_VERSION`	Default version passed to Studio or Control Plane build scripts when no explicit version argument is supplied.

Variable	Purpose
`WIN_CSC_LINK` / `WIN_CSC_KEY_PASS_WORD`	Windows Electron signing certificate and password. Keep both outside source control.
`CSC_LINK` / `CSC_KEY_PASSWORD`	Electron Builder fallback names for Windows signing.

`MINA\_VERSION` and signing variables are build-time settings; they are not needed on an installed runtime machine.

Automatically injected deployment variables

When the Control Plane starts an on-premises deployment, it injects `MINA\_APPLICATION\_DIR`, `MINA\_ENVIRONMENT`, `MINA\_DEPLOYMENT\_ID`, and `MINA\_INSTANCE\_ID`. It also injects the deployment's decrypted secret values using their configured names. These values should not be manually duplicated in a package or checked into source control.

Windows

For the current PowerShell session:

```
$env:MINA_DATA_DIR = 'C:\ProgramData\MINA\runtime-data'
$env:MINA_RUNTIME_LOG_DIR = 'D:\IntegrationLogs'
$env:MINA_LOG_TIMEZONE = 'America/Phoenix'
$env:MINA_DRIVER_HOME = 'C:\SAP\jco'
python -m uvicorn app.main:app --host 127.0.0.1 --port 8787
```

For a persistent machine-level setting, run PowerShell as Administrator and use `[Environment]::SetEnvironmentVariable('NAME', 'VALUE', 'Machine')`. Restart the service or application after changing environment variables.

Linux and systemd

Create a root-readable environment file such as `/etc/mina/runtime.env`:

```
MINA_DATA_DIR=/opt/mina/runtime/data
MINA_RUNTIME_LOG_DIR=/opt/mina/logs/runtime
MINA_LOG_TIMEZONE=America/Phoenix
MINA_DRIVER_HOME=/opt/mina/drivers
MINA_ENVIRONMENT=dev
```

Reference it from the service unit:

```
[Service]
EnvironmentFile=/etc/mina/runtime.env
```

Protect files containing secrets with `chmod 600` and set the service account as owner. Run `systemctl daemon-reload` and restart the service after changes.

Docker and Kubernetes

Use ordinary configuration values for paths and ports, and use Docker/Kubernetes Secrets for credentials:

```
env:
  - name: MINA_ENVIRONMENT
    value: dev
  - name: MINA_LOG_TIMEZONE
    value: America/Phoenix
  - name: MINA_DATA_DIR
    value: /var/lib/mina/runtime
  - name: MINA_ADMIN_API_KEY
    valueFrom:
      secretKeyRef:
        name: mina-admin-secrets
        key: api-key
  - name: MINA_ADMIN_SECRET_KEY
    valueFrom:
      secretKeyRef:
```

```
name: mina-admin-secrets
key: encryption-key
```

Persist `MINA\_DATA\_DIR` and `MINA\_ADMIN\_DATA\_DIR` on volumes. For SAP JCo, provide the licensed matching JCo JAR/native library through a protected volume or image layer and set `MINA\_DRIVER\_HOME` to that directory. Do not print secret environment values in startup diagnostics.

Operational notes

- Environment variables are read during process startup; restart the affected process after changing them.
- Relative paths are resolved from the process/application data location and should be replaced with absolute paths in production.
- `MINA\_LOG\_TIMEZONE` changes display timestamps only. Scheduler and token expiry calculations continue to use UTC internally.
- Existing log files are not rewritten when the timezone changes.
- To inspect effective non-secret settings on Windows use `Get-ChildItem Env:`; on Linux use `env` or `systemctl show <service> --property=Environment`.

Reference source: docs/ENVIRONMENT_VARIABLES.md

SAP ECC integration runtime guide

Connectors

Guide

SAP ECC integration runtime guide

MINA uses the licensed SAP Java Connector (JCo) supplied by the customer. The connector is not redistributed with Studio. This implementation interoperates with SAP ECC through public RFC/JCo contracts. MINA is an independent product and does not include third-party proprietary connector implementations.

MINA itself is not SAP-certified merely because it uses JCo. SAP permits JCo to connect external applications to SAP in both directions, but the customer must download it from SAP, comply with named-user/solution licensing, and must not redistribute the connector. Record the JCo, Java, OS, and CPU architecture reported by Test Connection as part of every production approval.

Supported integration patterns

Pattern	Runtime behavior
IDoc Listener	Persistent registered JCo server, tRFC TID duplicate protection, bounded delivery queue, one workflow per IDoc in a package, enclosing transaction commit only after every IDoc succeeds
RFC/BAPI Listener	Persistent registered JCo server for the selected function, request/reply export and table response mapping, rollback when the task fails or omits Reply
Invoke RFC/BAPI	Persistent, bounded JVM worker pool; JCo destination pooling; import, changing, nested structure, and table parameters; request/reply, tRFC, and qRFC transport
Post IDoc / IDoc Reader posting	XML or rendered input is converted to EDI_DC40 and EDI_DD40 and sent with `IDOC_INBOUND_ASYNCHRONOUS`; tRFC or qRFC is selected at transport level
IDoc Parser / Converter / Renderer	Metadata-driven, byte-position fixed-width conversion with physical E2 to logical E1 normalization, configurable code page, strict/replace invalid-byte policy, and delimiter-safe flat records
Read Table	`RFC_READ_TABLE` field, filter, paging, and named-record output
Dynamic Connection	A dedicated persistent JVM and thread-bound JCo context across activities; explicit commit/end, rollback on failure/expiry, bounded session count, and duplicate-session rejection
IDoc acknowledgment / confirmation	Invokes the customer-configured SAP acknowledgment RFC because this function is solution-specific

ALE distribution is implemented through the IDoc listener/posting paths. SAP partner profiles, ports, process codes, message types, RFC destinations, and authorizations remain SAP Basis configuration and are not generated by Studio.

Required runtime files

Install a matching x64 JCo distribution in one of these locations:

```
C:\ProgramData\MINA Studio\drivers\sap\sapjco3.jar
C:\ProgramData\MINA Studio\drivers\sap\sapjco3.dll
```

On Linux use `libsapjco3.so` next to `sapjco3.jar`, or select an explicit SAP driver directory in the shared connection. The packaged Java runtime and JCo native library must have the same architecture. Do not mix files from different JCo releases.

Use a currently supported SAP JCo 3.1 patch and a JDK/JRE listed for that patch in SAP Note 2786882. As of September 2026 SAP publishes JCo 3.1.14 and lists Java 8, 11, 17, 21, and 25; re-check SAP's support page when qualifying a build.

ECC configuration checklist

- Create the RFC user with only the required S_RFC and IDoc authorizations.
- Configure SM59 for the Studio Program ID, gateway host, and gateway service.
- Configure WE21, WE20, message types, process codes, and partner profiles for ALE/IDoc traffic.
- For qRFC, configure and monitor the queue in SMQ1/SMQ2 and set a non-empty SAP queue name in the activity.
- Make each concurrently deployed registered server identity unambiguous.
- Keep the TID store on durable writable storage. Never share one TID file between unrelated Program IDs.
- Validate ``STFC_CONNECTION``, fetch the actual IDoc/RFC metadata, and save the selected metadata with the project before packaging.

Resource and failure behavior

- Listener event queues and stderr history are bounded. Backpressure reaches the JCo callback instead of permitting unbounded Python heap growth.
- Cancellation does not leave blocked Python executor threads. Runtime shutdown closes native listener processes and removes their temporary descriptors.
- The JCo server waits for the workflow result. Success commits the delivery; task failure, cancellation, missing RFC Reply, or reply timeout rolls it back so SAP can retry according to its transaction policy.
- TID state is persisted atomically. A previously committed or in-flight TID is rejected as a duplicate; SAP confirmation removes it.
- Single-call BAPI auto-commit executes ``BAPI_TRANSACTION_COMMIT`` in the same JCo context as the BAPI call.
- A Dynamic Connection with ``transactional=true`` pins all calls carrying its ``sessionID`` to one worker/JCo context. ``contextEnd=true`` closes the context; with ``autoCommit=true`` it first executes ``BAPI_TRANSACTION_COMMIT``. Failure, expiry, or normal termination without ``commitOnTerminate=true`` rolls back.
- External outbound calls use persistent JVM workers. ``outboundWorkerCount`` controls Python-side worker concurrency; ``poolCapacity``, ``peakLimit``, and ``connectionExpirationMilliseconds`` configure the JCo destination pool in each worker. Changing any connection setting, including credentials, creates a distinct pool identity without storing plaintext credentials in its key.

Flat IDoc and messaging modes

The built-in parser reads canonical EDI_DD40 byte fields (``SEGNAM``, ``MANDT``, ``DOCNUM``, ``SEGNUM``, ``PSGNUM``, ``HLEVEL``, and ``SDATA``) and EDI_DC40 control fields. Set ``idocEncoding`` to the actual sender code page. Keep ``invalidCharacterPolicy=strict`` for production unless replacement characters are an accepted business rule. When SDATA may contain CR/LF, configure a unique ``idocRecordDelimiter``; that delimiter is authoritative and embedded line breaks remain data. With no delimiter, canonical 1063-byte records are split by width.

If the separately licensed ``sapidoc3.jar`` is installed, Test Connection reports its presence and version. Selecting the SAP JIDocLib parser engine validates IDoc XML with SAP's repository-backed ``IDocXMLProcessor``; flat EDI_DD40 input continues to use the built-in byte parser because SAP documents the public IDoc class-library processor as an XML processor.

IDoc Listener supports these sources:

- ``NoMessaging``: the direct persistent JCo RFC server, including durable TID handling and SAP transaction acknowledgment.
- ``EMS`` or ``JMS``: consumes one queue message through the selected shared connection and publishes its body to the parser. Client acknowledgment is exposed through the standard Confirm Message activity.
- ``Kafka``: consumes one topic record through the selected Kafka resource and publishes its value to the parser. Manual offset confirmation uses the same Confirm Message contract.

For broker-backed modes, select ``messagingResourceId`` and ``messagingDestination``. SAP-to-broker transaction atomicity is owned by the upstream bridge; MINA does not claim an XA transaction spanning SAP tRFC and an independent EMS/JMS/Kafka broker.

Production certification

Unit and mock tests verify serialization, lifecycle, transaction routing, and bounded queues, but they cannot certify a particular ECC/JCo combination. Before production, run an environment test with the exact JCo version and architecture:

- sustained IDoc batches containing the largest expected SDATA payloads;
- concurrent RFC/BAPI calls at the configured maximum connection count;
- SAP and Studio restart during in-flight tRFC/qRFC deliveries;
- duplicate TID, reply timeout, malformed IDoc, network partition, and expired credential scenarios;
- a 24-72 hour soak while recording process RSS, JVM heap/GC, thread count, handle count, queue depth, throughput, and SAP SM58/SMQ1/SMQ2 state.

Do not accept the connector for production until those measurements remain stable and every failed transaction is demonstrably recoverable.

The repository includes a repeatable qualification runner and an intentionally non-secret example configuration:

```
Copy-Item docs\sap-soak-config.example.json sap-soak-config.local.json
# Edit the local copy with QA-only credentials and parameters, then run:
backend\.venv\Scripts\python.exe scripts\sap-production-soak.py `
  --config sap-soak-config.local.json --output sap-soak-report.json
```

Set ``terminateWorkerEveryCalls`` to a positive number for controlled worker-JVM failure injection. Run separate 24-72 hour profiles for sRFC, transactional BAPI, tRFC/qRFC IDoc posting, and inbound listeners. The generated report holds throughput, failures, latency percentiles, and worker process IDs; correlate those IDs with OS/JVM monitoring and SAP SM58/SMQ1/SMQ2 evidence. Never commit the local configuration because it contains credentials.

Reference source: docs/SAP_INTEGRATION.md

EMS/JMS sender reuse in Run mode

Connectors

Guide

EMS/JMS sender reuse in Run mode

Run previously included the unique job execution ID in the persistent sender cache key. Each incoming event therefore opened another Java worker/EMS connection. After eight cached configurations, additional jobs fell back to one-shot JVMs. Debug used a stable session ID and did not normally encounter this problem.

The runtime now gives EMS/JMS an application-scoped sender identity, separate from the job ID. Jobs keep unique IDs for logging and tracing. Subtasks and parallel branches inherit their caller's sender scope; Debug retains its own session scope. Different applications and runtime instances remain isolated. Connection settings still participate in the cache key, so changed credentials/settings are not ignored.

Studio project Stop cancels execution and force-closes that application's senders. Restart can establish a fresh sender. Idle eviction, bounded caching, serialized JMS session access, and the existing no-replay-on-uncertain-send behavior remain.

The fix applies to Windows and Linux execution through WorkflowRuntime, including new engine-based exports. Rebuild/reinstall Studio and regenerate/redeploy engine archives to replace older bundled code. Direct Python connector exports use a different connection path and are not modified by this fix.

Verification uses the actual Java bridge and a local test JMS provider: twelve separate Run jobs retain one sender, application Stop is isolated, restart works, and provider failures are not automatically replayed. Debug, scheduling, runtime, and engine-export regression suites also pass. Real EMS throughput still requires measurement against the target broker; no fixed delivery latency is guaranteed.

For diagnosis inspect ``senderReused``, ``providerSendMs``, and ``publishTiming`` (``queueMs``, ``bridgeMs``, ``totalMs``) in send output. A healthy warm send should reuse its sender. Upstream event rate, broker persistence, network latency, and payload logging can still constrain end-to-end application throughput.

Reference source: [docs/EMS_RUN_CONNECTION_REUSE.md](#)

Kafka publishing performance

Connectors

Guide

Kafka publishing performance

Studio and engine-backed exports reuse Kafka producers and run publishing on a dedicated four-thread executor. Long-running receivers, SAP/JCo calls, and other default-executor work therefore do not hold up Kafka enqueue operations. Producer cleanup closes this executor with the runtime. Direct Python exports reuse an asynchronous producer per connection/configuration and event loop.

Publish output includes `publishLatencyMs` plus:

- `publisherQueueMs` (Studio/engine): time waiting for a Kafka worker thread.
- `producerSetupMs`: producer lookup/creation time; direct Python includes initial broker connection setup and waiting for the creation lock.
- `sendAndWaitMs`: serialization/enqueue/delivery work after producer lookup; this is not a pure network-latency measurement.
- `producerReused` (direct Python): whether the existing asynchronous producer was reused.

With `waitForDelivery` enabled, completion includes broker acknowledgement. Kafka transactions also wait for delivery and commit in the Studio engine. With it disabled, `queued: true` and `deliveryConfirmed: false` indicate local queue acceptance, not confirmed broker delivery. This change does not alter acknowledgement settings or transaction semantics.

For remaining delays, compare the first publish with subsequent publishes using the same connection. High setup time on the first direct Python publish can involve metadata, DNS, TLS, or authentication. High send/wait time can include configured linger, queue backpressure, broker replication, retries, and network latency. Capture these timing fields alongside the effective producer settings and broker metrics before changing reliability options. Rebuild installed Studio and regenerate exported Python archives to use the updated implementation.

The automated tests cover thread-pool saturation, producer reuse, queued and acknowledged sends, and transaction initialization reuse. They do not establish throughput or latency against a real Kafka broker.

Reference source: docs/KAFKA_PUBLISHING.md

MINA security review - 2026-09-22

Security

Guide

MINA security review - 2026-09-22

Outcome and scope

Production blockers found, particularly for shared/multi-team deployments. Reviewed source at commit `676c0bf` with the existing working-tree changes present. This is a security-focused review across Studio/runtime, debugger, Control Plane, local/Kubernetes agent, Electron IPC, package handling, connector execution, secret storage, and selected dependencies. It is not an exhaustive certification that all code or dependencies are safe. No application security fixes were applied in this review. Only this report was added.

Checks used static source inspection, mocked in-process authorization checks, synthetic credentials, installed-package metadata, upstream advisories, and `npm audit --json --ignore-scripts`. No real credentials were printed, no live deployments were modified, and no destructive or resource-exhaustion tests ran.

Severity below assumes the affected service is reachable or the stated attacker role exists. Firewall, reverse proxy, OS permissions, and deployment topology can reduce exposure, but do not fix the underlying application boundary.

Findings

F1 - Critical: unauthenticated Control Plane owner access by default

Evidence: `administrator/app/main.py:66,141-157` uses an empty API key by default and returns the Technology Team Owner identity when no credential is provided. `administrator/run\_admin.py:6` binds to `0.0.0.0:9080` by default.

Impact: Anyone able to reach an unconfigured instance can obtain governance and deployment privileges, including access to agent deployment responses that contain decrypted deployment secrets. The risk is not restricted to localhost.

Validation: With `API\_KEY` mocked to empty, an anonymous synthetic request resolved to `roles=['Owner']`. No live server configuration was changed.

Fix: Fail closed when credentials are missing. Restrict initial bootstrap to loopback plus a one-time credential; require authenticated setup before exposing network access. Keep management APIs behind TLS and an authenticated boundary.

F2 - Critical: deployed application code inherits management credentials and host privileges

Evidence: `administrator/app/main.py:1238-1243` copies the full administrator environment into application subprocesses. `administrator/agent/main.py:173-184` does the same for agent deployments. Agent configuration reads `MINA\_AGENT\_KEY` from that environment (`administrator/agent/main.py:39`). Raw Python deployments are started under the same OS identity, not a separate tenant security boundary.

Impact: A malicious or compromised deployed application can read management credentials such as the admin API key, secret-encryption key, or agent key when configured via environment. Agent endpoints currently require a Technology Team credential, not a narrowly scoped per-plane identity. Consequently an inherited credential can expose other teams' deployments and secrets. The same OS identity also permits access to other application files accessible to the service account; clearing environment variables alone is insufficient isolation.

Validation: Source-confirmed inheritance and privileged agent authorization. No real credential extraction or malicious deployment was attempted.

Fix: Explicit child-process environment allowlist, per-plane scoped credentials, and isolated low-privilege application identities/containers with separate filesystem access. Never expose management or encryption keys to application code. Treat packages, scripts, and vendor drivers as executable code requiring approval.

F3 - High: deployment move bypasses team namespace authorization

Evidence: ``administrator/app/main.py:1735-1748`` checks deployment ownership, the target plane's registered namespaces, and the capability, but omits ``team_can_use_namespace``. Initial deployment creation checks this permission at ``administrator/app/main.py:1160``.

Impact: An Application Manager can move its stopped deployment to a registered namespace not assigned to its team, bypassing the intended deployment-placement boundary. Availability of valid plane/capability IDs is required.

Validation: A mocked Team A request successfully moved its own deployment to ``team-b``; a mocked ``team_can_use_namespace`` returning false was never called. All persistence/audit functions were mocked; no real deployment moved.

Fix: Apply the same team/namespace/capability authorization to move, redeploy, restore/revert, and start. Recheck immediately before execution as permissions may change after deployment creation. Add negative cross-team route tests.

F4 - High: Studio management/runtime APIs have no authentication boundary

Evidence: ``backend/app/main.py:575-617`` exposes project listing, creation, editing, and execution without authentication dependencies. The middleware near line 80 controls caching, not access. Workflows can execute Python and commands (``backend/app/runtime.py:2030,2089``).

Impact: A caller who can access the runtime HTTP endpoint can read projects and configure/execute operations with the runtime account's permissions. The packaged sidecar binds to loopback (``backend/run_sidecar.py``), which limits remote exposure but does not authenticate other local users/processes. Exposing this same API for browser access without an authenticated proxy broadens the risk. Cross-origin browser exploitation was not established in this review.

Validation: Unauthenticated in-process requests to project listing and a synthetic memory-only connection test both returned HTTP 200. No shell commands or real connector operations were invoked through these endpoints.

Fix: Per-launch authenticated desktop session credentials, Origin/Host checks, and explicit authenticated user access/RBAC for browser deployments. Keep untrusted workflow code out of the privileged Studio/management process.

F5 - Medium: project exports and workspace files contain plaintext secrets

Evidence: ``backend/app/store.py:46-58`` serializes resource configurations and property values without encryption. ``backend/app/main.py:689-700`` writes full project/resource/property models into the project export without redaction.

Impact: Shared ``mpackage`` exports and workspace backups can disclose database, EMS, cloud, and other connector credentials. This concerns project exports; it does not imply every deployment archive's secret handling is identical.

Validation: A synthetic password in a mocked project resource was present in the exported ``project.json``. No real project secret was read or displayed.

Fix: Redact secret values by default on project export, preserve secret references, and make any sensitive export an explicit protected operation. Use OS-backed or managed secret storage and restrict workspace ACLs.

F6 - High: Studio project import has unbounded memory/decompression exposure

Evidence: ``backend/app/main.py:1475-1494`` calls ``await file.read()`` with no application limit and decompresses manifest/project ZIP entries without checking uncompressed size or compression ratios. By contrast, Control Plane and agent archive handlers implement explicit member and expanded-size limits.

Impact: A reachable import endpoint can be used to consume large amounts of memory/CPU using oversized or highly compressed uploads, disrupting Studio and its runtime. A reverse-proxy compressed-body limit alone is insufficient.

Validation: Source-confirmed. No ZIP bomb or large-memory attack was executed.

Fix: Limit upload bytes, entry count, expanded size and per-entry size before materialization; use bounded streaming reads, concurrency limits, and quotas.

F7 - Medium: installed Python web dependencies have known advisories

Evidence: The inspected backend environment has FastAPI ``0.115.12``, Starlette ``0.46.2``, and python-multipart ``0.0.20``. Both backend and administrator requirements pin FastAPI ``0.115.12`` and python-multipart ``0.0.20``.

- Starlette's multipart rollover can block the event loop: CVE-2025-54121,

fixed in `0.47.2`. The installed `0.46.2` is affected. [Maintainer advisory](https://github.com/Kludex/starlette/security/advisories/GHSA-2c2j-9gv5-cj73).

- python-multipart has Content-Disposition parameter interpretation issues

covered by CVE-2026-53537, fixed in `0.0.30`. [Maintainer advisory](https://github.com/Kludex/python-multipart/security/advisories/GHSA-vffw-93wf-4j4q).

- The separate python-multipart arbitrary-file-write advisory CVE-2026-24486

requires non-default `UPLOAD\_DIR`/`UPLOAD\_KEEP\_FILENAME` settings. Those settings were not found in the reviewed application, so this is not being claimed as a demonstrated arbitrary-file-write exploit here.

[Advisory](https://github.com/advisories/GHSA-wp53-j4wj-2cfg).

Fix: Upgrade to a mutually compatible, patched FastAPI/Starlette/multipart set and regression-test uploads, forms, and packaged builds. Scan resolved Python dependencies and shipped artifacts in CI rather than only requirements text.

Additional hardening observations

- Electron uses context isolation, disabled Node integration, and sandboxing,

which are positive. However, file-write IPC accepts caller-provided paths (`frontend/electron/main.cjs:158-170`) without sender-frame authorization, and navigation/window-opening restrictions were not found. A compromised renderer could abuse the bridge. A renderer compromise was not demonstrated here.

- Control Plane UI stores its credential in localStorage. Reduce exposure with

scoped/session credentials and a suitable CSP; do not assume HTML escaping alone provides a complete browser security boundary.

- TLS enforcement, firewall rules, Windows ACLs, Unix permissions, Kubernetes

RBAC/network policies, and supply-chain/signing controls need deployment-level verification. They cannot be certified from this source checkout.

- Existing archive path/member checks, Control Plane encrypted secret storage,

hashed team tokens, ownership checks on many routes, and default SSH host-key rejection are useful controls. Their presence does not mitigate the specific bypasses above.

Dependency-check results and limitations

`npm audit --json --ignore-scripts` completed successfully and reported zero known advisories across 478 dependency entries. This is not a claim that the frontend or Electron bridge is vulnerability-free.

`pip-audit` is not installed in the backend environment. No package installation or application dependency mutation was performed. Python checks were limited to installed metadata and selected verified upstream advisories, not a full resolved dependency audit. Vendor SAP/EMS/JDBC JARs, native libraries, bundled Java/Python, and the actual Windows installer were not fully audited.

Recommended order

- Before shared deployment: close anonymous Control Plane access and isolate

application processes from management credentials and filesystem privileges.

- Fix namespace authorization on move and audit every lifecycle route.

- Authenticate Studio APIs and restrict network exposure.

- Patch the web dependency stack and bound import/upload processing.

- Redact/export secrets safely, harden Electron IPC/navigation, and add security

regression tests plus complete Python/JAR/native artifact scans.

No live-server compromise was demonstrated or required. The findings are enough to block a claim that the current product is safe for untrusted multi-team hosting.

Web dependency security update

Security

Guide

Web dependency security update

The Studio backend, browser runtime, and Administrator now pin the same reviewed web stack: FastAPI 0.141.1, Starlette 1.6.0, and python-multipart 0.0.32. This replaces FastAPI 0.115.12 and python-multipart 0.0.20 and explicitly pins Starlette instead of relying on an indirect dependency.

Upstream references:

- [Multipart upload rollover denial of service](https://github.com/Kludex/starlette/security/advisories/GHSA-2c2j-9gv5-cj73)
- [URL-encoded form limits](https://github.com/Kludex/starlette/security/advisories/GHSA-82w8-qh3p-5jfq)
- [Windows StaticFiles UNC handling](https://github.com/Kludex/starlette/security/advisories/GHSA-wqp7-x3pw-xc5r)
- [Multipart disposition parsing](https://github.com/Kludex/python-multipart/security/advisories/GHSA-vffw-93wf-4j4q)

Applying the update

For source installations, stop the service, install its requirements with its own Python environment, then restart it. From the repository root on Windows:

```
& .\backend\.venv\Scripts\python.exe -m pip install -r backend/requirements.txt
& .\backend\.venv\Scripts\python.exe scripts/verify-web-dependencies.py
& .\administrator\.venv\Scripts\python.exe -m pip install -r administrator/requirements.txt
& .\administrator\.venv\Scripts\python.exe scripts/verify-web-dependencies.py
```

For a browser-only installation use `backend/requirements-browser.txt` instead. For packaged installations, rebuild and redistribute the desktop/Administrator installer. Already installed executables retain their old bundled dependencies. Existing deployment environments must update their own Python dependencies too. Do not disable certificate verification to install dependencies.

Windows packaging scripts now verify the installed web versions before bundling; dependency-install failures stop packaging rather than reusing stale packages.

Verification and limits

Security regressions cover multipart upload functionality, offloaded disk rollover, URL-encoded field count/size enforcement, consistent dependency pins, and stale build rejection. Backend runtime and Administrator tests pass with the new stack. This is not a full security certification or remediation of other audit findings. Request/body size limits, authentication, and deployment hardening remain necessary.

Local environment caveats: the Administrator virtual environment points to a missing Python 3.11 installation, so Administrator tests were run with the updated backend environment. Repair/recreate that environment before building it. The backend Python 3.14 environment also reports platform compatibility issues for httptools, PyYAML, watchfiles, and websockets during `pip check`; these packages were not changed by this update. A clean supported build environment and installer smoke test are still required before release.

Reference source: docs/WEB_DEPENDENCY_SECURITY.md